COLLEGIUM DA VINCI

Faculty of Applied Sciences

Major: INFORMATION TECHNOLOGY
first-cycle studies

Vitalii Sakhno, 28696.
Artem Dzhula, 28701.
Oleksii Fronin, 28685.

Web and mobile application for providing moving services to the customers
Aplikacja webowa i mobilna do świadczenia usług przeprowadzkowych klientom

Engineering thesis

Thesis supervised by
dr inż. Adam Czajka

Poznan, 2026

Table of contents

Introduction

The digital economy has changed transportation forever. Nowadays, 'Uber-style' applications are the new normal for both people and businesses. Building digital tools for things like apartment moves or city deliveries is a necessary fix for the problems of old-school logistics.

This project offers a platform of web and mobile application for providing moving services to the customers. It also covers the entire journey of building software from the initial idea to final design. Everything was started by outlining the theoretical part of the project that included a proper breakdown of industry challenges, personal motivations and reasons for multiple decisions that were realized in the project. After that functional requirements were defined. They provided a system with behaviours for different user scenarios. We also described all the stages of project creation starting with the design and finishing with technical implementation of main elements. A brief summary took its place at the end of the thesis.

The choice of moving services as the main topic of the project is driven by several critical factors where urban logistics and software architecture meet.

Traditional logistics is often stuck in a rigid system of fixed schedules and set routes. The aim was to build something adaptive. Whether you are moving a few boxes from a small apartment or hauling a massive load between cities, our platform lets you customize the service to fit your specific needs, not the other way around.

Logistics usually involves a messy mix of phone calls, email and different apps. This solved this by creating one unified interface. By putting the customer and the driver in the same digital space, a chaotic process turned into a simple, transparent dashboard where everyone knows exactly what is happening.

A common nightmare in moving is ordering a truck for a sofa and having a small car showing up instead. In order to stop this guessing game, a smart filtering system was built.

When logistics are unorganized, prices are often unpredictable. By using software to model the distance and the vehicle type upfront, we turned an unreliable service into a professional operation. This gives the user a clear, cost-effective price before the move even starts, removing any hidden surprises.

1. Current state of knowledge

1.1 Project motivation

The decision to undertake development of the ‘Gazelka’ platform was driven by the specific engineering and logistical challenges in the urban transport sector. The justification for this work stands on the reason that unlike the standard system of postal logistics, the niche of residential and small scale commercial movings presents certain technical problems that have not been solved by the current market yet. In order to resolve this kind of problem our choice was to deploy a cross-platform architecture consisting of web and mobile applications.

Theoretical research into urban transport shows that logistics of residential moving remains fragmented. This sector is characterised by a high density of small and independent services, operating without a unified information framework. The lack of unity in this point leads to significant resource waste such as empty runs or mismatched vehicle usage where capacity of the fleet does not align with the actual market demand [32].

A big issue in this field is the lack of clever information between the driver and the customer. In most cases, the driver has all the information about the trip, while the customer is left in the dark. This basically creates a ‘trust gap’ where misunderstandings about price and time often lead to arguments. It was concluded that the best way to solve the problem is a dgital platform. When both the driver and the customer see the same real-time data on their screens such as the exact price or order status - the process becomes more transparent.

Nowadays people move more often for work or education but moving house is still considered one the most difficult events for individuals. From our perspective, this stress is mostly caused by a lack of reliable software. Many people still have to rely on old-fashioned phone calls and manual search. [2]. Therefore, the goal of this project is creating a simple and modern framework that changes moving from a chaotic task into a predictable service. By replacing manual haggling with an app-based model, the process is supposed to become easier for every user.

1.2 Market research and competitive analysis

Before diving into the development of the 'Gazelka' platform, it was essential to evaluate how the modern logistics market actually functions. To be honest, it was interesting to notice a sharp increase in the ride hailing market [26], considering where it mostly fails the average user. Currently, the industry is dominated by massive digital platforms but their focus is often split in ways, that leave specific niches underserved.

Figure 1.1 Ride-Hailing Market (source: [1])

Take Uber Freight [33], for example. To understand why they are that dominant, you have to look at how the parent company basically reinvented the entire ride-hailing market. From our perspective, Uber's original success came from its ability to use real-time GPS tracking and dynamic pricing to connect regular drivers with passengers through a simple app. This concept proved that you can easily eliminate the midleman (like traditional taxi dispatchers) and significantly lower costs.

Naturally, they tried to port this success over to the logistics sector with Uber Freight, aiming to connect carriers with shippers just as easily as they manage to connect riders and drivers. But here is the catch: while their ride-hailing app is perfect for a person moving between 2 points, their freight setup is strictly designed for massive B2B operations - it works great for shipping a couple of IKEA boxes but it is almost impossible for a regular person, who just wants to move a wardrobe or some office equipment to use it. This a reason to conclude that Uber Freight handles the truck, but it does not handle the heavy lifting which is a major pain for users.

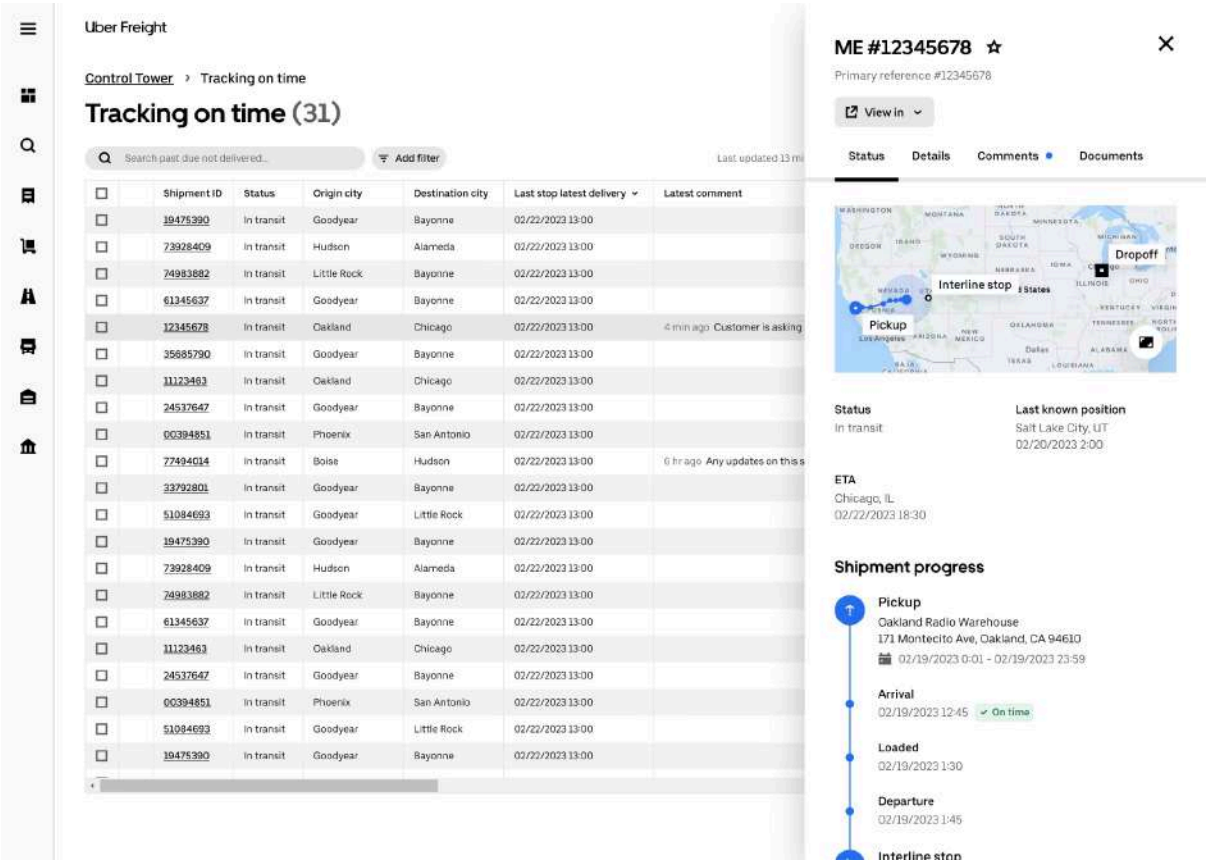


Figure 1.2 Uber Freight - Ride schedulement (source: [33])

When other players were observed, we noticed Lalamove [24] which is extremely popular in the Asian 'on-demand' delivery sector. The business model uses a similar ride-hailing logic specifically to lorries and motorcycles, but there is a fundamental diference in their target audience. From what was seen, Lalamove is primarily optimized for small-scale retail logistics and e-commerce deliveries, meaning it is just not for everyone. At the same time, our project is fully about full-service relocation.

They actually excel at moving a single palette or a few IKEA [19] boxes across the city within a few hours. But when it comes to a full-scale residential move, we noticed significant limitations of the platform. Lalamove treats its drivers just as transporters - they do not provide a robust way to coordinate them with customers or specify the handling of fragile, oversized furniture like grand pianos or heavy wardrobes. Basically, it is a great tool for moving stuff but a poor tool for 'moving a home', which is a much more complex logistical operation that requires more than just a cargo bed.

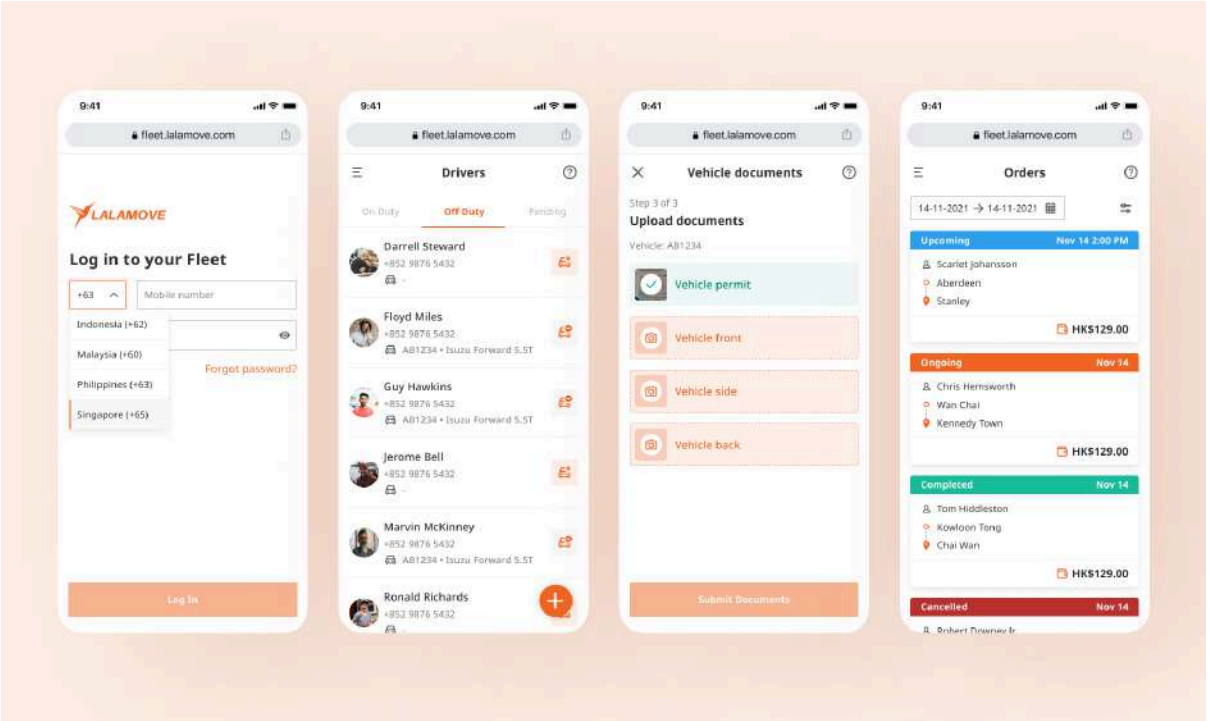


Figure 1.3 Lalamove - Basic interface (source: [3])

On the European side of the market, AnyVan [1] (often reffered to in similar contexts as a general 'AnyMove' model) takes a completely different approach by using a bidding-based marketplace. Instead of a booking system, it allows users to list their job details and wait for various independent transporters to bid on it. While this can lower prices, it introduces what is called 'human uncertainty' because the bid system generally does not seem very reliable, especially when it comes to transportations.

In our opinion, this bedding system often turns the process into a guessing game. A user might get a cheap quote but they have no real guarantee about the driver's reliability until they show up. There is also the issue of hidden costs - since the price is not set by a central alghoritm but by individual negotiation, being able to change at the last minute and is a huge step back compared to 'Gazelka' that is represented as a project of the next generation. Taking this view into account, we conclude that 'AnyVan' is a very manual project. It lacks the seamless, software-driven experience, where the system knows exactly which vehicle and crew is needed based on the cargo data provided.

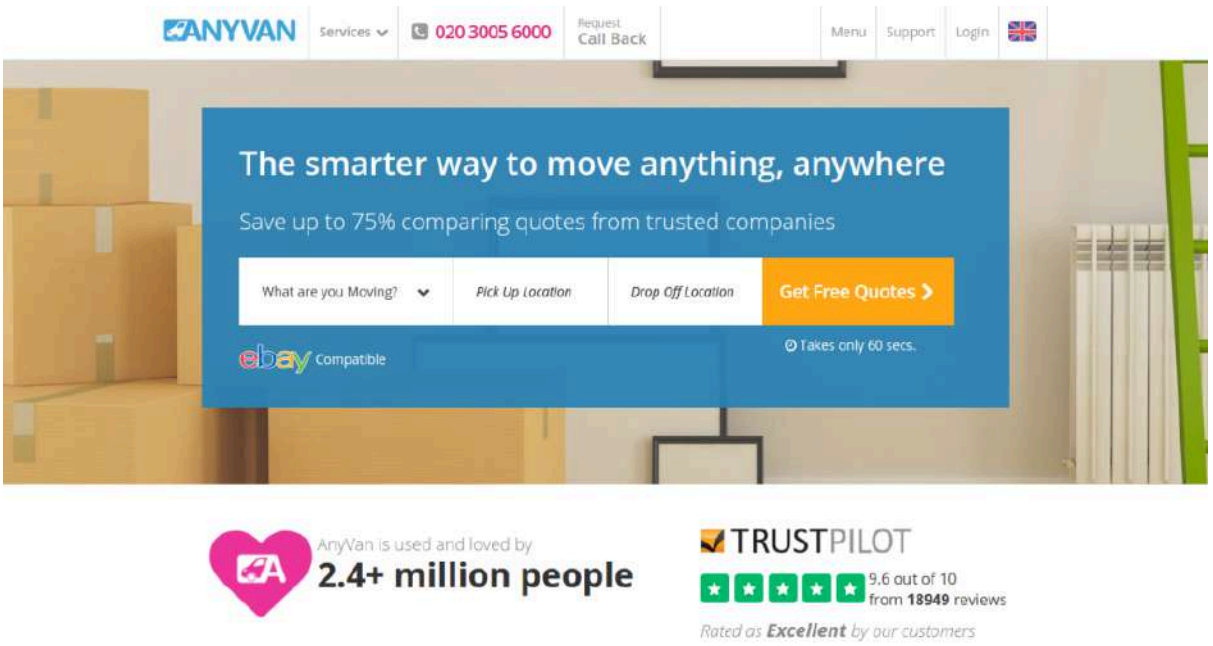


Figure 1.4 AnyVan - Booking Screen (source: [1])

2. Aim of the work

2.1 Goal of the project

The principal aim of this project is to develop a complete full-stack web and mobile applications for an instant delivery service, created in order to effortlessly connect customers and drivers. The application named ‘Gazelka’ is designed to meet the challenges of modern urban environment by providing an intuitive platform that streamlines order placement, driver assignment, price valuation and communication. By implementing current technologies and libraries, the project aims to create a flexible solution that boosts user experience, operational efficiency and reliability in service delivery.

The comprehensive goal is building a durable system that enables customers to request deliveries quickly and drivers to accept and fulfill these criteria, while endorsing protected authentication, accurate pricing and instant updates. Specifically, the project aims the next objectives:

1. User onboarding and verification. Provide secure login and registration, password recovery algorithm for the both roles including role-precised features such as vehicle registration for drivers;
2. Order creation and management. Allow customers to generate orders with specific location selection using interactive maps and provide distance calculations;
3. Driver pipeline. Permit drivers to view and approve accessible orders and dynamically update their statuses;
4. Enforce real-time and notifications. Apply an internal chat for straight messaging between customers and drivers based on real-time order delivery. Push notifications to track order’s progress;
5. Security. Ensure data authentication, provide strictly role-based approach and generally reduce errors in order fulfillment in order to improve user satisfaction on such a challenging market.

2.2 Scope of work

The development stage of the project focuses on providing the core functionalities while excepting advanced features like payment integration or administration panel.

Design

- Figma interface wireframes. Creates as a visualisation of the project model;
- Design system. Includes a deep argumentation of corporative styles chosen for the project starting background colors and finishing font styles;
- Future scalability. Coming up with potential project improvements as it develops in a competitive environment.

Front-end

- User interfaces for login/registration, email verification, password recovery, order creation and other key features of the project;
- Order creation with map-based location selection, distance computation and price estimation;
- Driver specific screens for available orders, their acceptance and status renew;
- Real-time features such as connection for chat messages and notifications push;
- Proper error handling.

Back-end

- Authentication end-points for login/registration, token issuance, email approval and password reset;
- Order handling. Creation, acceptance, status changing and fees logic;
- Chat endpoints. Message sending, history fetching and real-time chat distribution;
- Notification center. Background services for new order alerts and upcoming reminders integrated with FCM;
- Database interactions. User account management, orders, chats, notification settings and FCM tokens.

Tests

- Functions;
- Interfaces;
- Requests;
- Improvements

The project assumes a local operation on city-based screening and uses development tools/environment (emulator, local host, console logging). Future work can expand it to production deployment, adaptability improvements and additional platforms.

2.3 Responsibilities delegation

The platform was carried out by a team of three. The division of duties was agreed upon at the start of the work and is present in the below table.

Table 2.1 Duties division (source: *Developed by the authors*)

Duty	**Vitalii**	**Artem**	**Oleksii**
Mobile application	5%	35%	65%
Web application	70%	25%	5%
Design	30%	5%	65%
Project management	20%	40%	40%
Tests	10%	40%	50%

Documentation	80%	10%	10%
Server	5%	85%	10%

3. Methodology

3.1 Functional requirements

They define the specific behaviour of the system by detailing the exact functions and services the application must provide. In this project, these requirements cover the entire logistics cycle. Each point in the functional specification is designed to address a specific user intention. We have engineered the system to anticipate the actions of two primary actors that are the customer and the driver. The project documentation organizes these interactions into specific tables to clarify each perspective. Tables *3.1 - 3.8* represent the user's experience as a customer, tables *3.9 -3.12* - as a driver and tables *3.13 - 3.19* describe common functions for both actors.

Table 3.1 Register customer function (source: *Developed by the authors*)

Name	**Description**
Function name	Register customer.
Description	Account creation according to the chosen role of a customer.
Input data	Name and surname, email, password, phone number, verification code.
Source of input data	Registration form.
Result of function	A new record is created in the database.
Requirements	All fields in the form to be filled.
Purpose	Access to the platform’s features.
Initial condition	The user does not have an account in the system.
Final condition	The user has an account in the system.
Side effects	Account information is placed in the database.
Reason	Ability to track orders and higher security.

Table 3.2 Create order function (source: *Developed by the authors*)

Name	**Description**
Function name	Create order.
Description	Allows user to book a ride by entering specific details of the order.
Input data	Pick-up and drop-off locations, date and time, vehicle and cargo types.
Source of input data	User input through mobile or web application.
Result of function	An order is placed in the logistics system.
Requirements	Stable internet connection and customer account.
Purpose	To place and confirm transportation requests from users.
Initial condition	Service is available and the user is logged in.
Final condition	A confirmed and queued order for driver assignment.
Side effects	The order is displayed on the customer's dashboard.
Reason	Services easy transportation booking for customers.

Table 3.3 Create order function (source: *Developed by the authors*)

Name	**Description**
Function name	Terminate order.
Description	Allows customers to cancel the order before its initialization.
Input data	Cancel order tab click event.
Source of input data	Order details page.
Result of function	The order is deleted from the panel. Its status gets cancelled in the system.
Requirements	The order is neither in progress nor completed.
Purpose	Flexibility in order management.
Initial condition	The order is pending.
Final condition	The order is deactivated.
Side effects	The record is deleted.
Reason	Prevents unnecessary initialization of the order by a driver.

Table 3.4 Edit customer's personal information function (source: *Developed by the authors*)

Name	**Description**
Function name	Edit customer's personal information.
Description	Allows customers to adopt certain changes into personal information.
Input data	Name/surname, email or phone number.
Source of input data	Settings page.
Result of function	Changed personal data of a user.
Requirements	Customer account.
Purpose	Actualization of information about customers.
Initial condition	User is logged in.
Final condition	Inputs with changed data now replace the latest information about the user.
Side effects	The current record is updated in the database.
Reason	Improvements in user experience.

Table 3.5 Select vehicle type function (source: *Developed by the authors*)

Name	**Description**
Function name	Select vehicle type.
Description	Allows users to choose a specific truck based on cargo requirements.
Input data	Vehicle class picture click event.
Source of input data	Create order page.
Result of function	Selected vehicle logic is applied to the draft order.
Requirements	List of vehicle options.
Purpose	Matching the cargo size with appropriate van capacity.
Initial condition	One of the vehicles selected by default.
Final condition	The new selection is highlighted.
Side effects	Change of estimated price.
Reason	Transparency in prising.

Table 3.6 Select cargo type function (source: *Developed by the authors*)

Name	**Description**
Function name	Select cargo type.
Description	Allows users to choose a specific cargo type based on its size.
Input data	Cargo type title tab click event.
Source of input data	Create order page.
Result of function	Selected cargo type logic is applied to the draft order.
Requirements	List of cargo type options.
Purpose	Clarification of cargo requirements.
Initial condition	One of the cargo types selected by default.
Final condition	The new selection is highlighted.
Side effects	Change of estimated price.
Reason	Transparency in prising.

Table 3.7 Select cargo type function (source: *Developed by the authors*)

Name	**Description**
Function name	Select date and time.
Description	Allows users to choose a specific timeframe for the order to be completed.
Input data	Date and time from the calendar.
Source of input data	Create order page.
Result of function	The ride is scheduled.
Requirements	Input fields for date and time.
Purpose	Clarification of order details.
Initial condition	Fields for date and time are empty.
Final condition	Date and time fields are filled.
Side effects	None.
Reason	Prevention of creating orders in the past.

Table 3.8 Select pick-up/drop-off location function (source: *Developed by the authors*)

Name	**Description**
Function name	Select pick-up/drop-off location.
Description	Allows customers to add addresses when creating an order.
Input data	Coordinates of location from map.
Source of input data	Order creation page.
Result of function	Location inputs are fulfilled.
Requirements	Active session, customer account.
Purpose	To add information about the order.
Initial condition	The user is on the order creation page.
Final condition	The user is on the order details page.
Side effects	Locations added to the order.
Reason	Flexibility.

Table 3.9 Register driver function (source: *Developed by the authors*)

Name	**Description**
Function name	Register driver.
Description	Account creation according to the chosen role of a driver.
Input data	Name and surname, email, password, phone number, verification code, city of service and vehicle information.
Source of input data	Registration form.
Result of function	A new record is created in the database.
Requirements	All fields in the form filled.
Purpose	Access to the platform’s features.
Initial condition	User is an anonymous on the page.
Final condition	User has an account and can create an order.
Side effects	Account information is placed in the database.
Reason	Ability to track orders and higher security.

Table 3.10 Accept order function (source: *Developed by the authors*)

Name	**Description**
Function name	Accept order.
Description	Allows drivers to choose an order from the list within the city of service.
Input data	Accept order tab click event.
Source of input data	Order details page.
Result of function	An order is placed in the logistics system.
Requirements	Stable internet connection, driver account and suitable vehicle type.
Purpose	To place and confirm transportation requests from users.
Initial condition	Service is available and the driver is logged in.
Final condition	A confirmed order waiting for initialization
Side effects	Order is displayed on driver’s dashboard
Reason	Services easy transportation booking for customers

Table 3.11 Edit drivers personal information function (source: *Developed by the authors*)

Name	**Description**
Function name	Edit driver's personal information.
Description	Allows drivers to adopt certain changes into personal information.
Input data	Name/surname, email, phone number, vehicle information, city of service.
Source of input data	Settings page
Result of function	Changed personal data of a driver
Requirements	Driver account
Purpose	Actualization of information about driver.
Initial condition	The driver is logged in.
Final condition	The latest information about the driver.
Side effects	The record is updated in the database.
Reason	Improvements in user experience.

Table 3.12 Cancel order function (source: *Developed by the authors*)

Name	**Description**
Function name	Cancel order.
Description	Allows the driver to cancel the order but only after its acceptance.
Input data	Cancel order tab click event.
Source of input data	Order details page.
Result of function	The order is deleted from the dashboard.
Requirements	Driver account.
Purpose	To reduce driver workload.
Initial condition	The driver is on the hashboard.
Final condition	Driver is released.
Side effects	The order is deleted from the database.
Reason	Adjustment to different circumstances.

Table 3.13 User login function (source: *Developed by the authors*)

Name	**Description**
Function name	Login user.
Description	Validates credentials and establishes a secure session.
Input data	Email and password.
Source of input data	Login page.
Result of function	Redirection to the dashboard in the active session.
Requirements	Valid account.
Purpose	Protection of user data.
Initial condition	The user is unauthorized.
Final condition	The user is logged in by its token.
Side effects	Previous session data is cleared.
Reason	Standard security requirement.

Table 3.14 Send message function (source: *Developed by the authors*)

Name	**Description**
Function name	Send message.
Description	Message transmission via WebSockets.
Input data	Text string.
Source of input data	Chat message input.
Result of function	Message appears on the receiver’s screen without page reload.
Requirements	Active SignalR connection, authorized session.
Purpose	Coordination during the order being completed.
Initial condition	Chat is open and WebSocket connection is established.
Final condition	Message displayed in UI bubble.
Side effects	None.
Reason	Clarification in order details.

Table 3.15 Access to the order history function (source: *Developed by the authors*)

Name	**Description**
Function name	Access to the order history.
Description	Displays list of completed transportations.
Input data	Orders tab click event.
Source of input data	Main menu.
Result of function	List of orders rendered into visual cards.
Requirements	Database connection, user’s past orders.
Purpose	Past orders details are retrieved.
Initial condition	The customer is on the dashboard.
Final condition	Historical data is displayed.
Side effects	Network request.
Reason	Record-keeping.

Table 3.16 Enable notifications function (source: *Developed by the authors*)

Name	**Description**
Function name	Enable notifications.
Description	Activities receivement of notifications.
Input data	None.
Source of input data	Settings tab click event.
Result of function	Notifications toggle is switched on.
Requirements	Active session.
Purpose	Keep users notified about changes in their orders.
Initial condition	User is on the settings panel.
Final condition	Notifications are turned on.
Side effects	None.
Reason	Clarify the moving process.

Table 3.17 Recover password function (source: *Developed by the authors*)

Name	**Description**
Function name	Recover password.
Description	Allows users to change password.
Input data	Verification code and new password.
Source of input data	Recover password tab click event.
Result of function	Password is changed.
Requirements	Active session and user profile.
Purpose	To improve user experience.
Initial condition	The user is on the login or settings panel.
Final condition	The user gains access to the account.
Side effects	Password updated.
Reason	Human factor.

Table 3.18 Verify email function (source: *Developed by the authors*)

Name	**Description**
Function name	Verify email.
Description	Verifies user credentials.
Input data	Verification code.
Source of input data	Email with details received.
Result of function	The credential is verified.
Requirements	Active session.
Purpose	To create a user account.
Initial condition	The user is on the registration or settings page.
Final condition	The user is on the verification page.
Side effects	A record created.
Reason	Security.

Table 3.19 Reverse geocoding address function (source: *Developed by the authors*)

Name	**Description**
Function name	Reverse geocoding addres.
Description	Converts map coordinates selected by the user into a readable street addres.
Input data	Latitude and longitude coordinates.
Source of input data	Map click event.
Result of function	Text string with addresses fills the inputs.
Requirements	Active internet connection, Nominatim availability.
Purpose	Elimination of manual typing errors.
Initial condition	The user is on the Create order page and input fields are empty.
Final condition	Input fields containing the exact addreses according to the map marker.
Side effects	Price recalculation.
Reason	Automating complex data entry.

3.2 Use case modelling

This kind of diagrams helps us look at the system's functions from the user's point of view. For our project this approach was used to set clear boundaries and define how two main actors, that are the customer and the driver interact, with the system.

By mapping out these boundaries, we ensured our development stayed focused on the most important features. This structure helped us identify exactly how each actor starts different processes, allowing us to build a precise and efficient environment tailored to their needs.

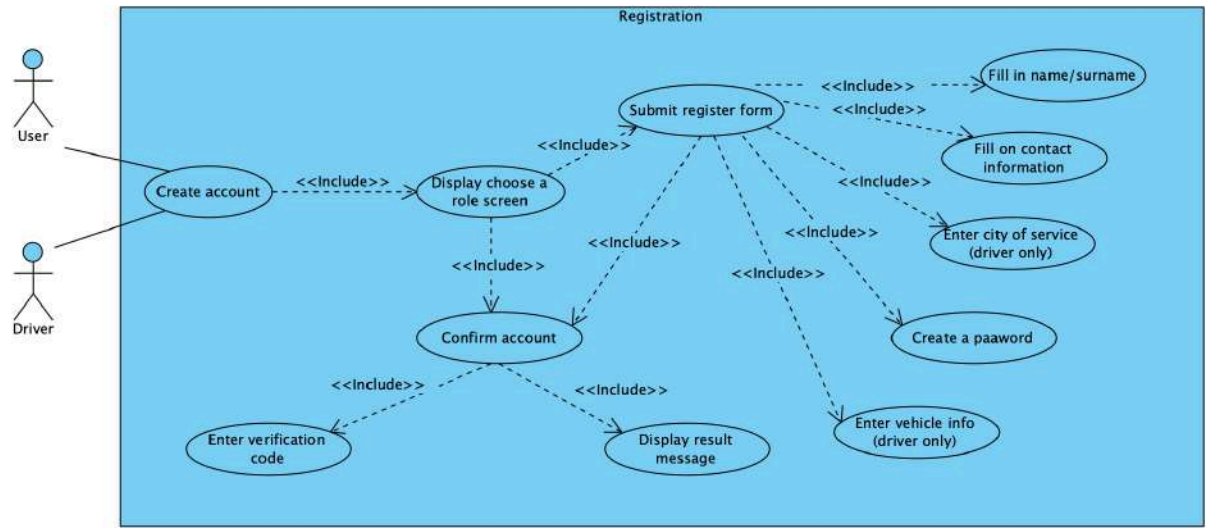


Figure 3.1 Registration use case (source: *Developed by the authors*, VisualParadigm)

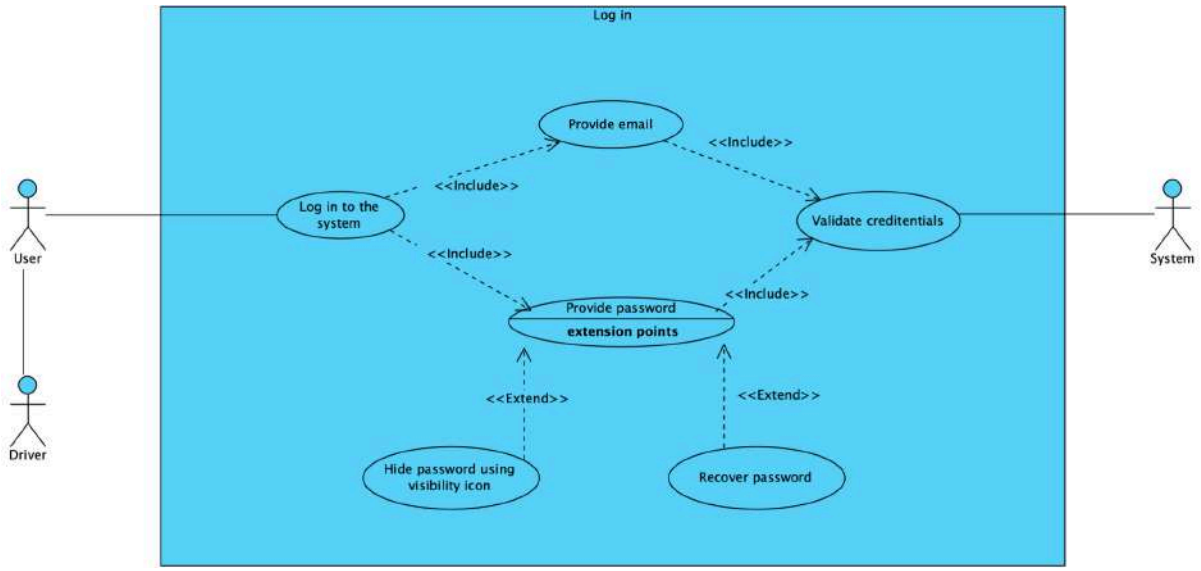


Figure 3.2 Login use case (source: *Developed by the authors*, VisualParadigm)

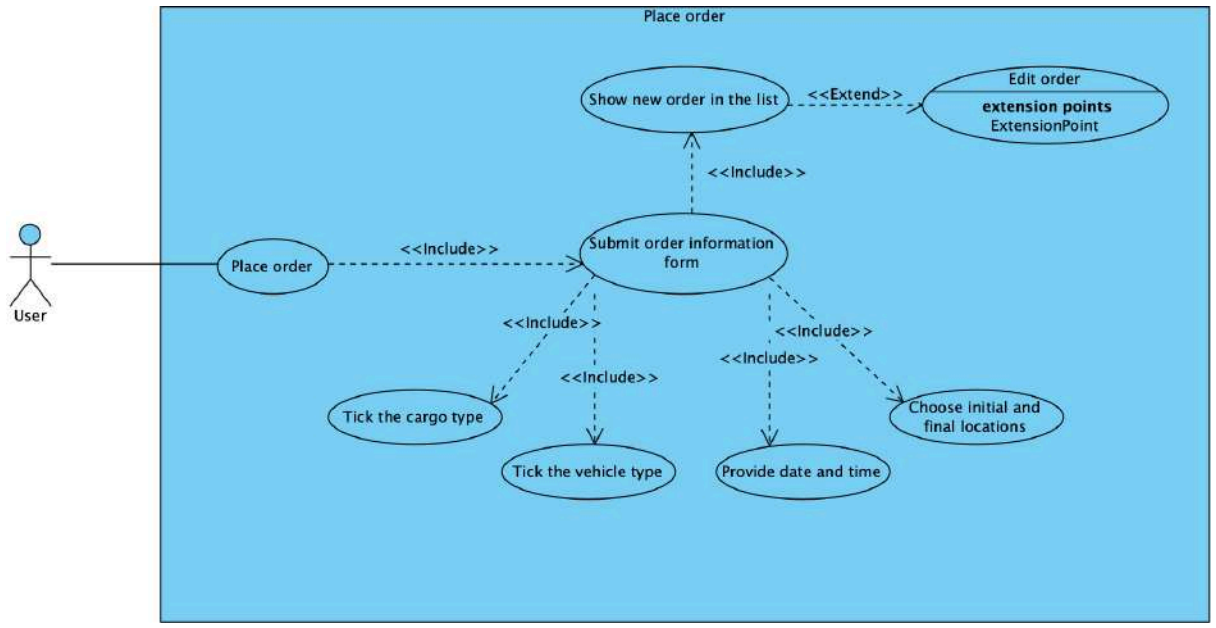


Figure 3.3 Place order use case (source: *Developed by the authors*, VisualParadigm)

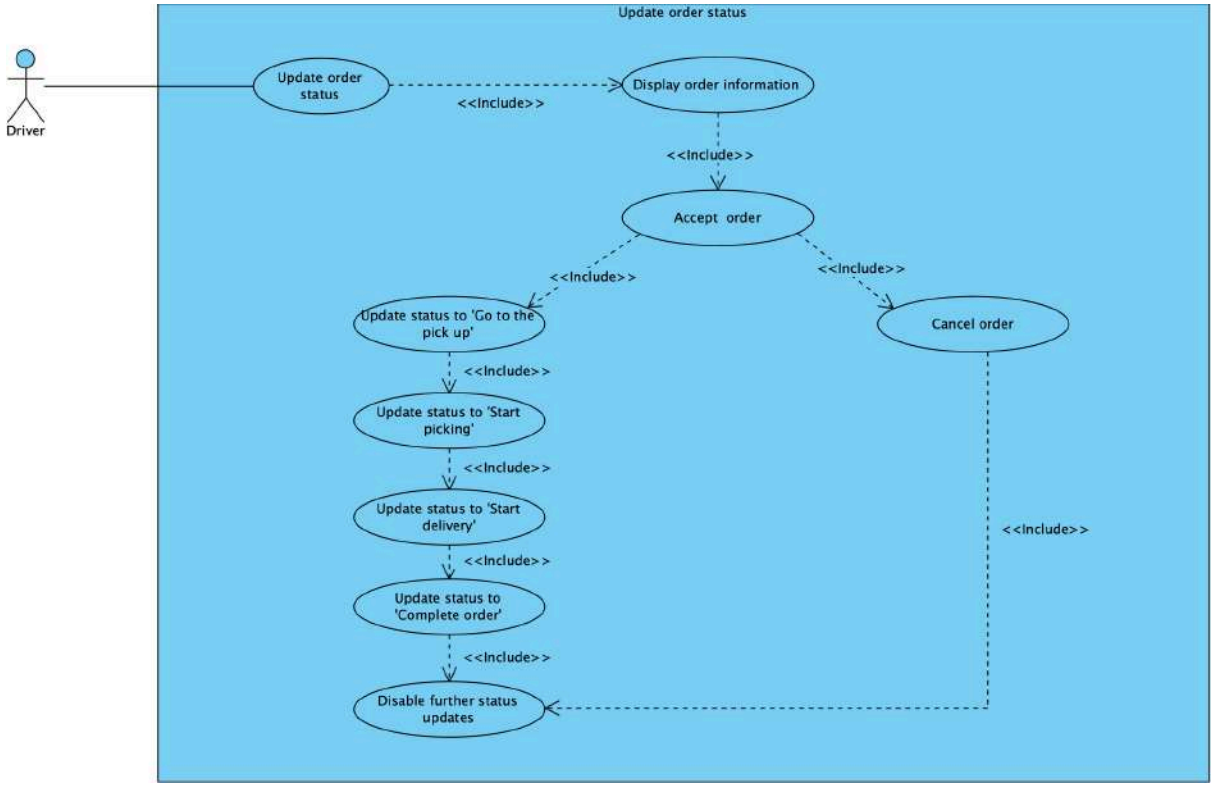


Figure 3.4 Update order status use case (source: *Developed by the authors*, VisualParadigm)

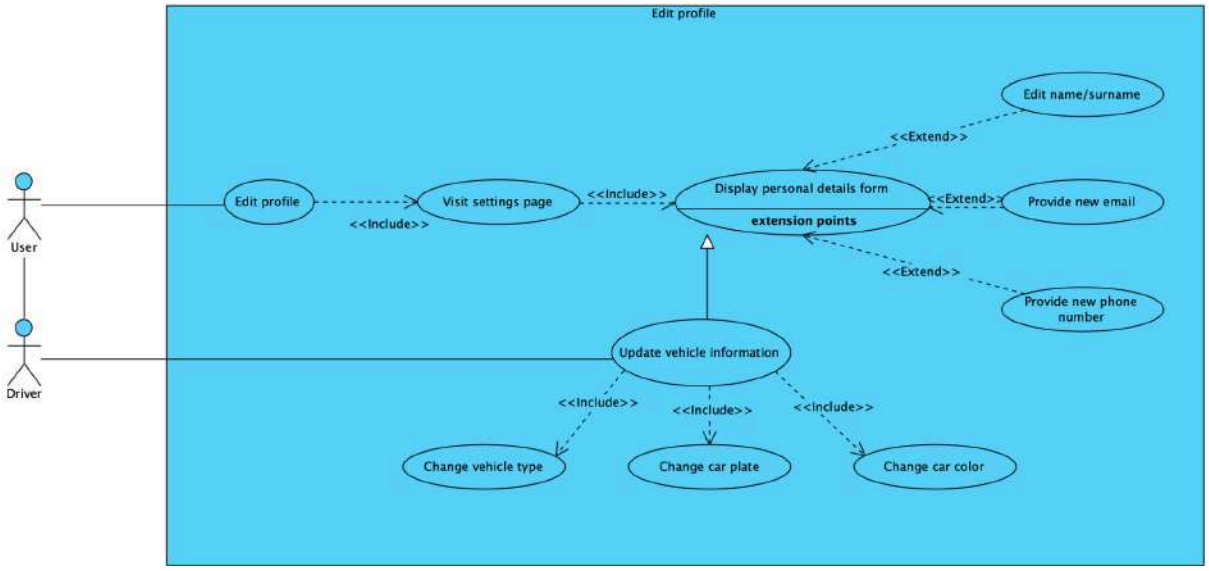


Figure 3.5 Edit profile use case (source: *Developed by the authors*, VisualParadigm)

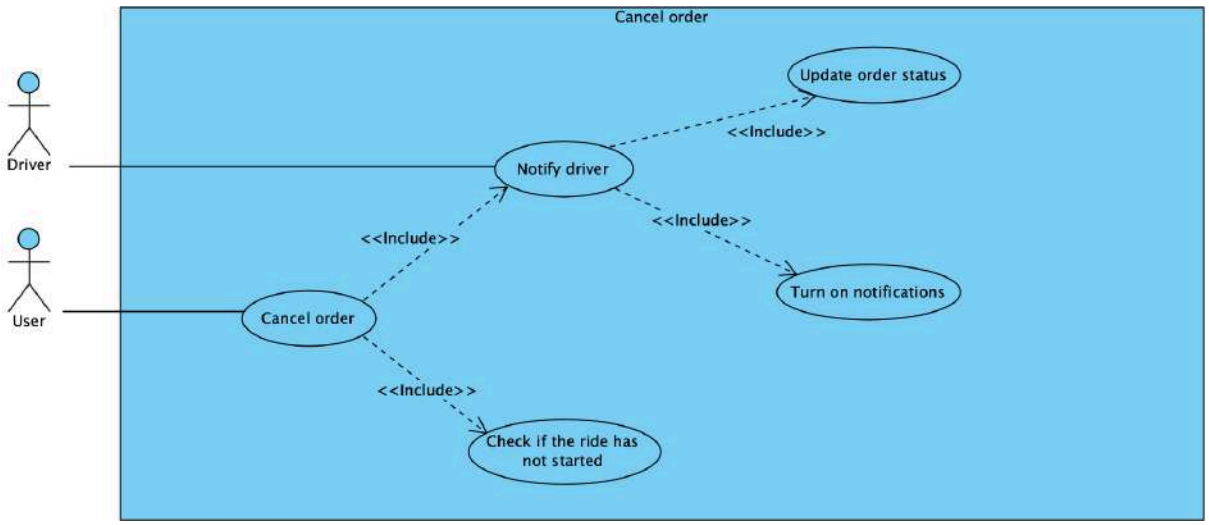


Figure 3.6 Cancel order use case (source: *Developed by the authors*, VisualParadigm)

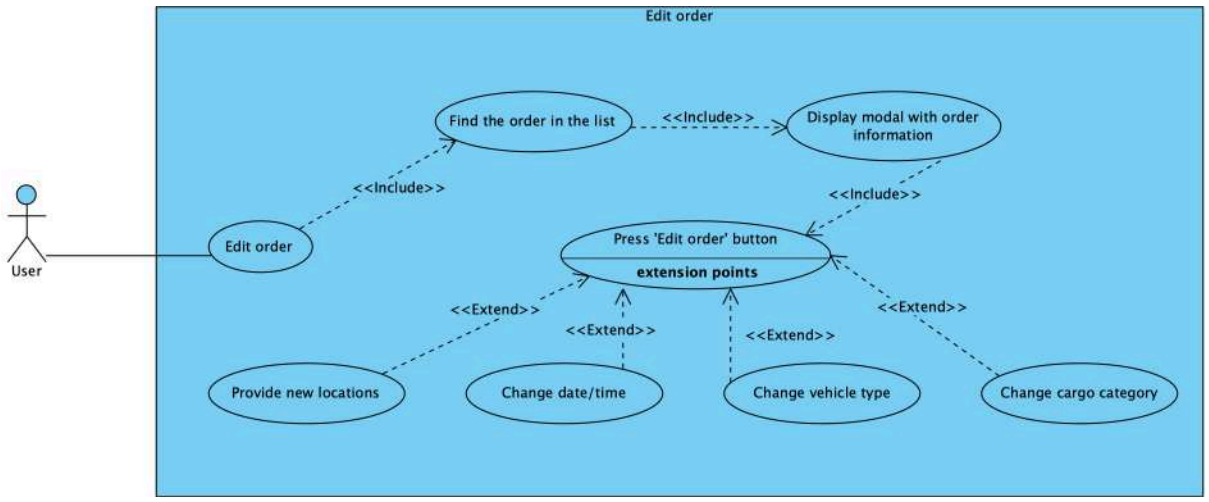


Figure 3.7 Edit order use case (source: *Developed by the authors*, VisualParadigm)

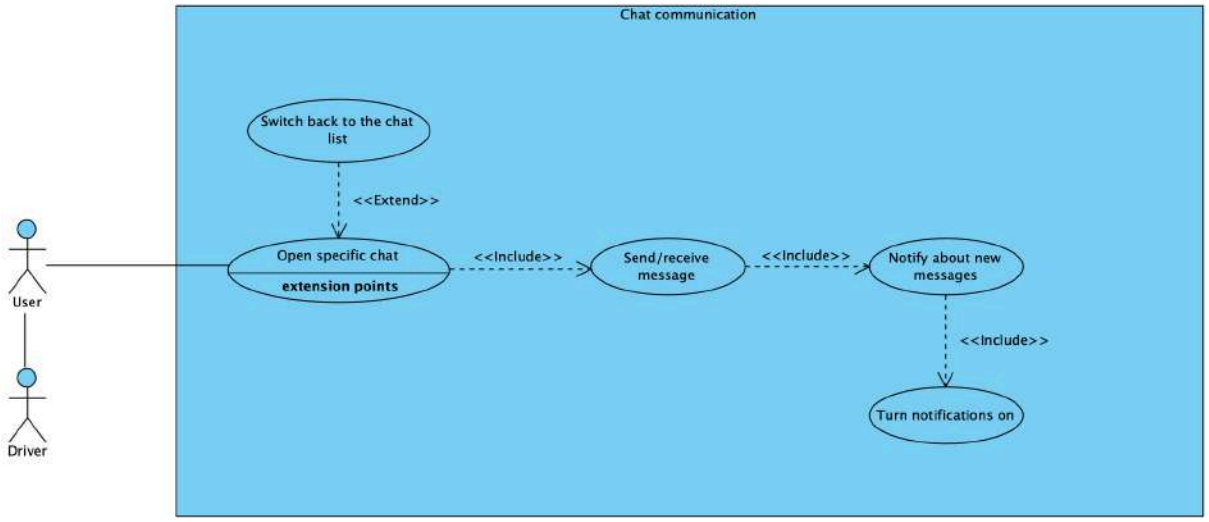


Figure 3.8 Chat communication use case (source: *Developed by the authors*, VisualParadigm)

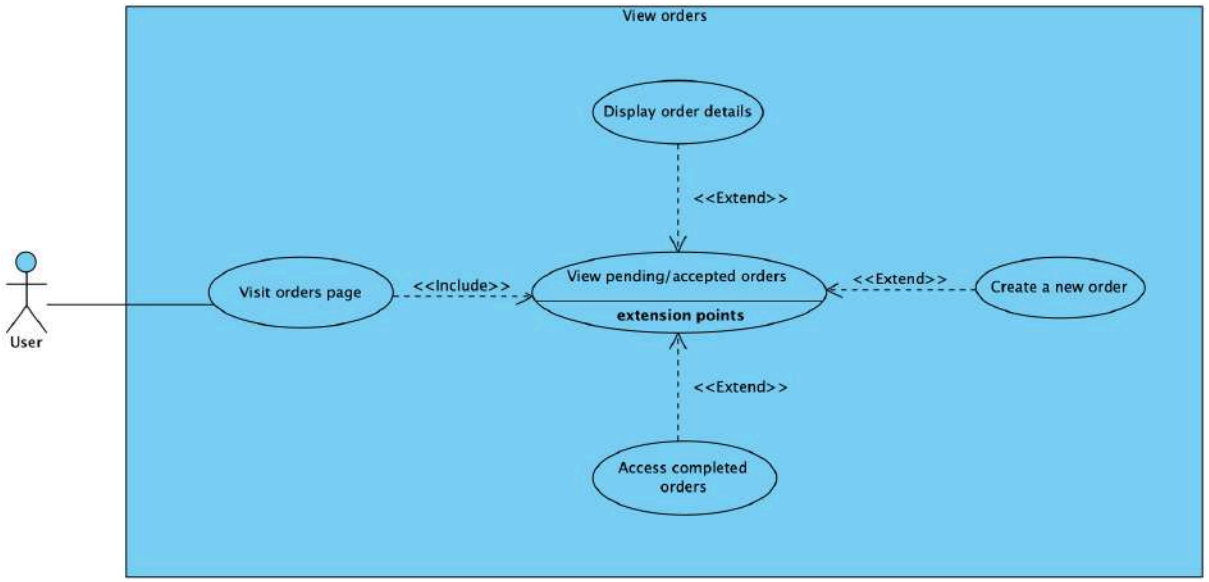


Figure 3.9 View orders use case (source: *Developed by the authors*, VisualParadigm)

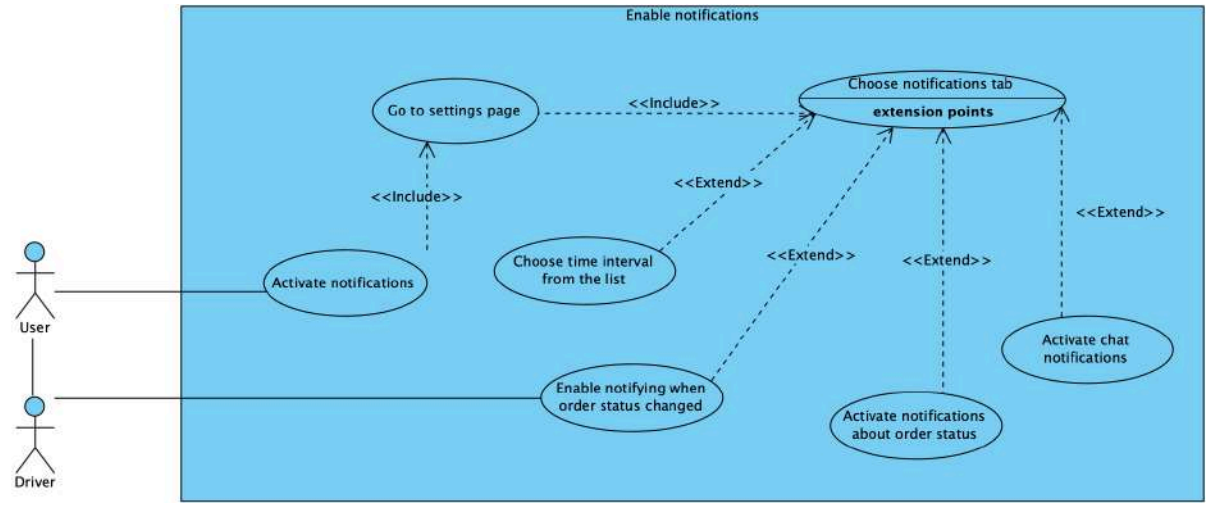


Figure 3.10 Enable notifications use case (source: *Developed by the authors*, VisualParadigm)

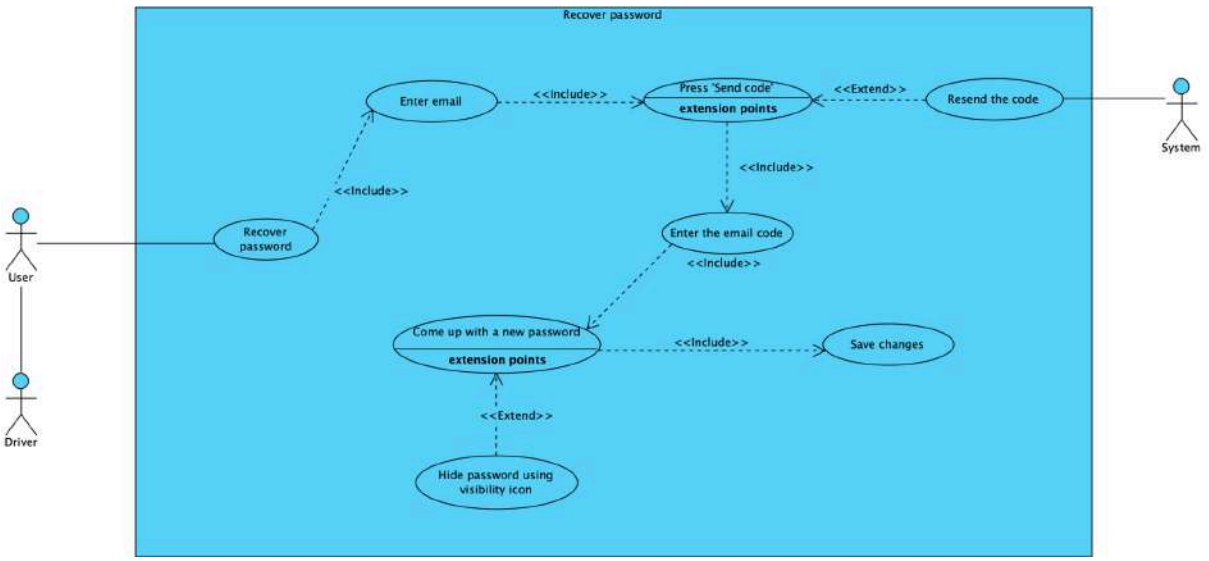


Figure 3.11 Recover password use case (source: *Developed by the authors*, VisualParadigm)

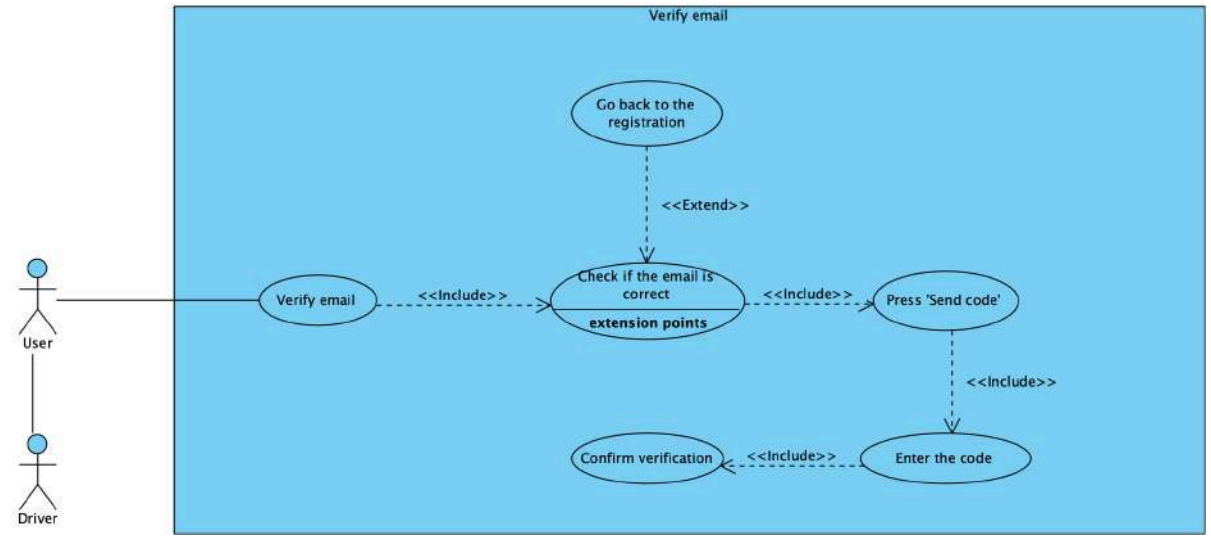


Figure 3.12 Verify email use case (source: *Developed by the authors*, VisualParadigm)

3.3 Class modelling

Class diagrams were also used in the development process to map out the structure of the system. This sort of diagram defined our main categories such as customer, driver, vehicle, order management, etc. This provides a technical specification for our database and back-end code, making sure the data stays organized and accurate on all the stages of the platform's future growth.

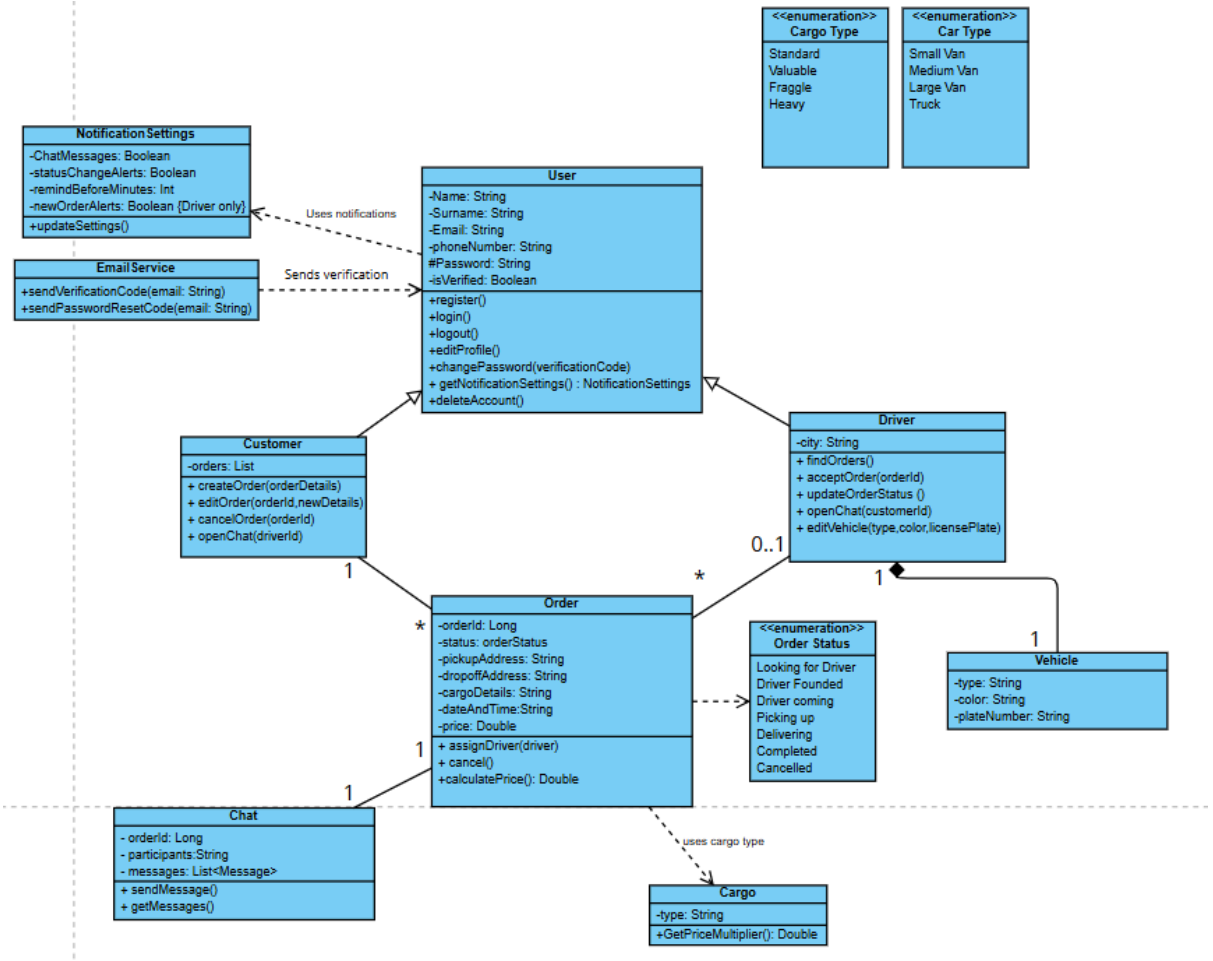


Figure 3.13 Class diagram (source: *Developed by the authors*, VisualParadigm)

3.4 Object modelling

Object diagrams were essential for us because they show exactly how the system looks at a specific moment when running different operations. They were used in order to test how users and external services interact in real-world scenarios. This allowed to double-check the rules set in the class diagram (for instance, how many users can connect at once) and fix any logic gaps before they could become problems.

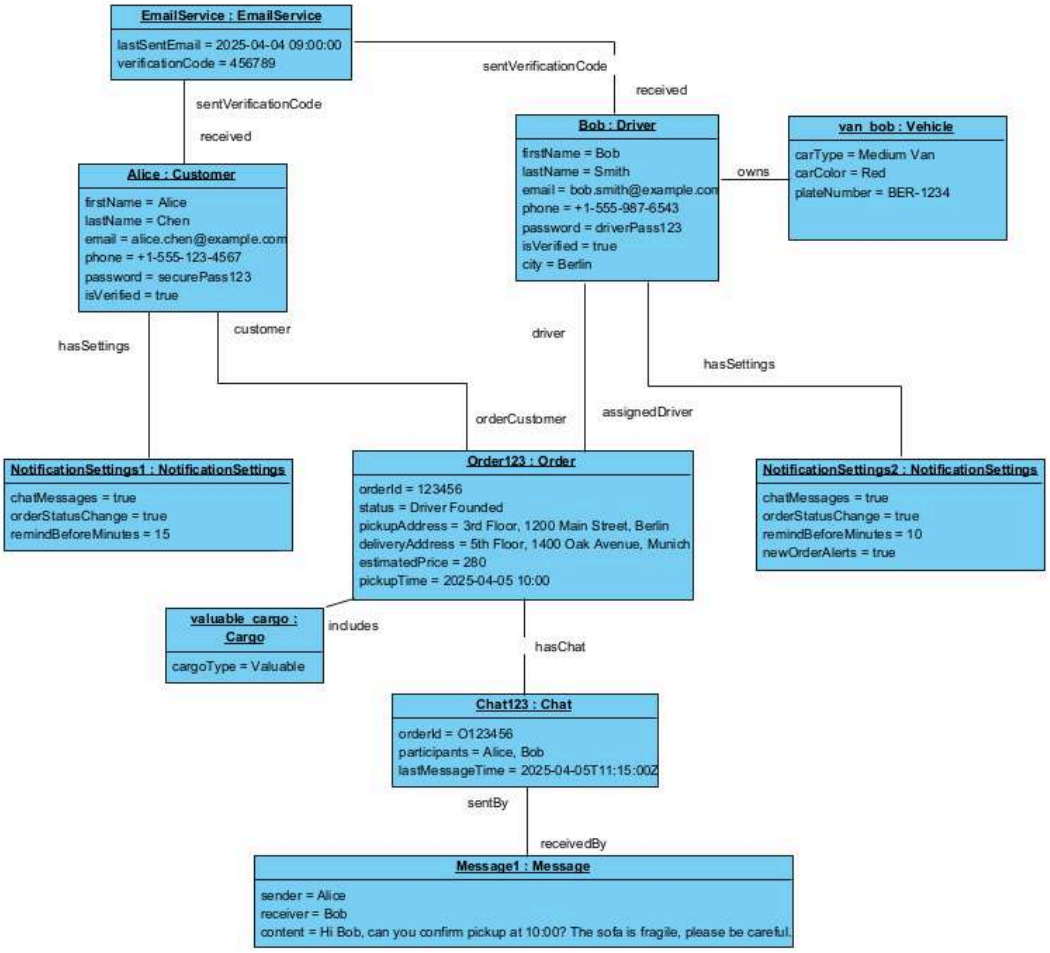


Figure 3.14 Object diagram (source: *Developed by the authors*, VisualParadigm)

3.5 Component modelling

Component diagrams were utilized to illustrate how our software is organized into modular parts. This has actually helped us to clearly separate the client side (what customers and drivers see) form the backend services (for example, authentication and order managanemt). By defining these parts, the system was made decoupled, which means we can develop, test or update specific parts without breaking the rest.

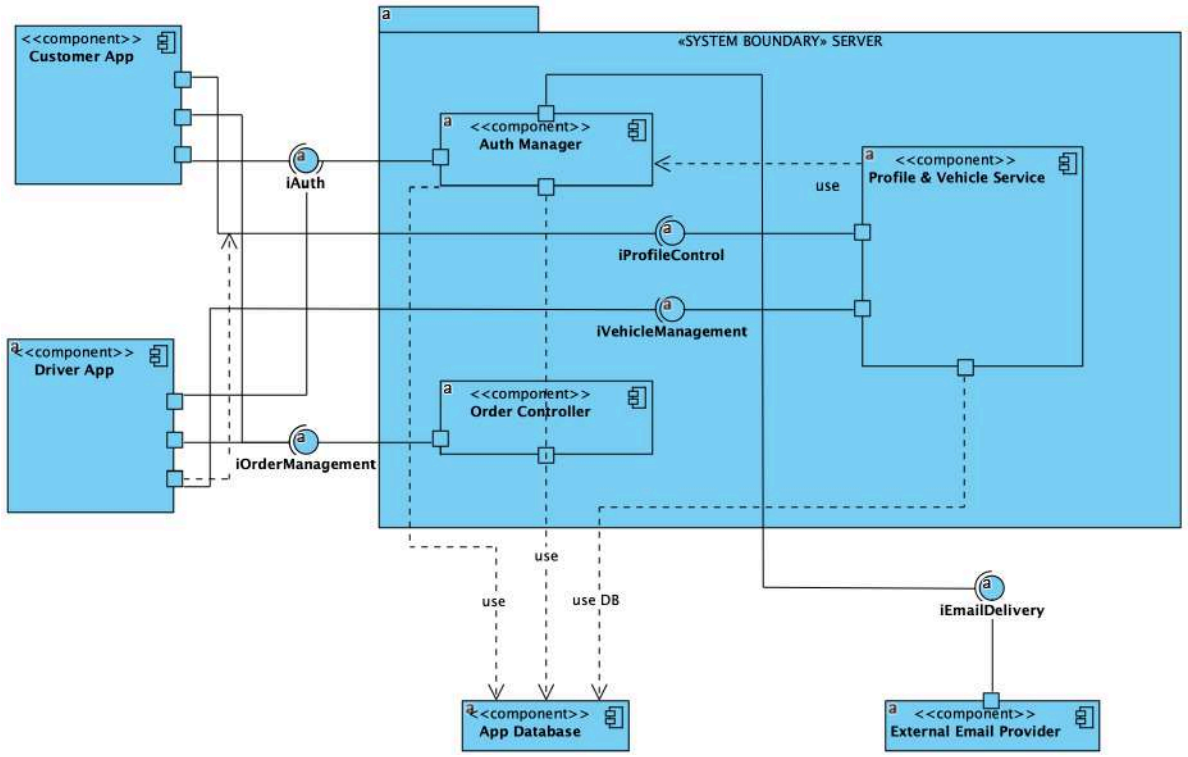


Figure 3.15 Component diagram (source: *Developed by the authors*, VisualParadigm)

3.6 Deployment modelling

Deployment diagrams were utilized in order to map out how the software runs on mobile devices and the cloud. We specifically traced the paths for secure data exchange between these locations and this step gave us a clear definition for the hardware and server settings that are need to host the system.

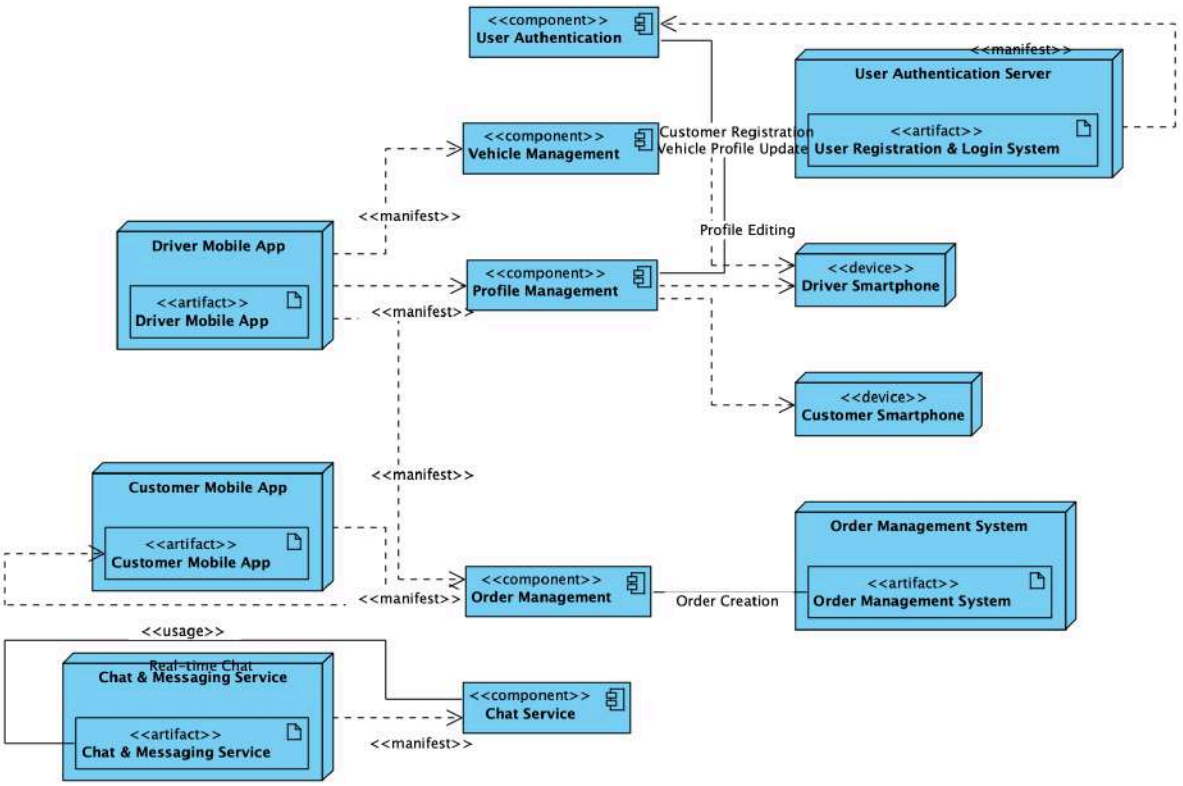


Figure 3.16 Deployment diagram (source: *Developed by the authors*, VisualParadigm)

4. Description of the project implementation

4.1 Implementation framework

This chapter details the actual building of the ‘Gazelka’ platform. It basically documents how the theoretical UML models that were developed in the previous chapter turned into a fully functional software system. To manage the complexity of a dual-user platform, a phased development approach was used that allowed to prioritize core features as well as to build a strong foundation.

To clearly structure our project, the development cycle was divided into certain stages. It started from justification of the selected programming languages and development tools that formed the technical foundation of our project. After that it moved on to a detailed look at the server architecture, data persestence and realisation of core business logic. Later on the user interface was outlined and experienced implementation for both the customer and the driver. Nextly, main communication protocols were explained and API contracts that enabled seamless data exchange within different levels of system components. Closer to the end, the implementation stage of ‘Gazelka’ was launched and tested all the functions from customer and driver applications. Our goal was to verify system stability and performance along with fixing different sorts of bags that, in our opinion, could provide the users with a less intuitive interface. Finally, the ready configuration was checked and hosted the platform into a live and operational environment.

4.2 Client side development

Initial wireframes

The user interface of the ‘Gazelka’ project was developed iteratively, starting from low-fidelity wireframes and progressing to high-fidelity mockups with consistent branding, improved typography and refined user flows.

A pair of representative design frames illustrate this evolution. The initial prototype stage and the final polished design. These frames were created in Figma [10], allowing for visual comparison of layout and interaction clarity.

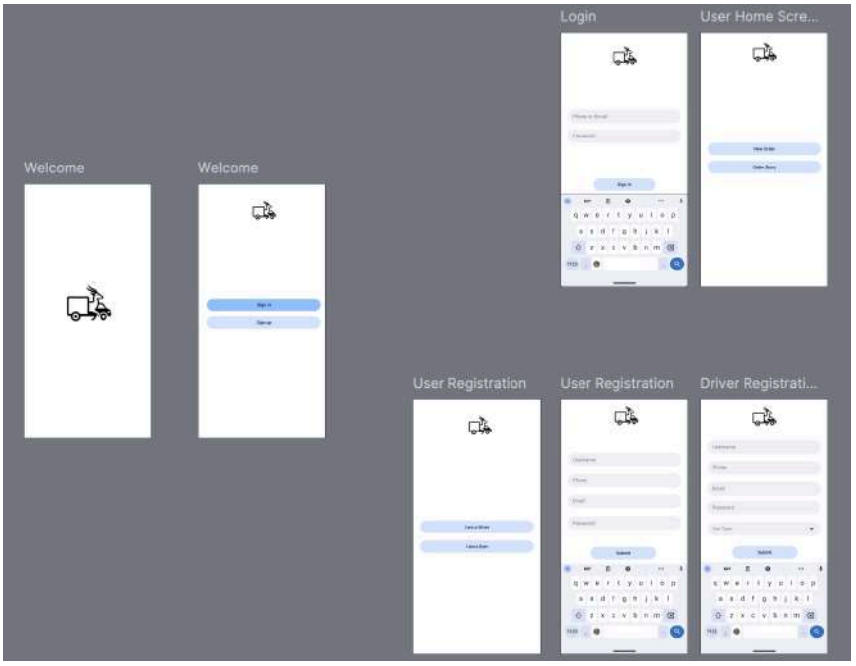


Figure 3.17 The first wireframes of mobile application (source: *Developed by the authors*, Figma)

The first frame represents the earliest conceptual stage of mobile application design. It features a minimalistic welcome screen with a single centered icon. After that it moves on to the Registration and Login buttons. The layout is extremely sparse, with no branding colors, no typography hierarchy and without any additional elements.
The purpose of this stage was to validate the basic onboarding structure and overall application flow without investing the detailed visuals. This version served as a quick proof-of-concept, focusing solely on functional placement of key actions.

Final wireframes

The final wireframes represent mature and production-ready design. It features a consistent yellow-black color scheme, full user registration and login flows for both customer and driver, clear typography and visual feedback. The design also i includes polished buttons, consistent icons usage and a logical branching structure.

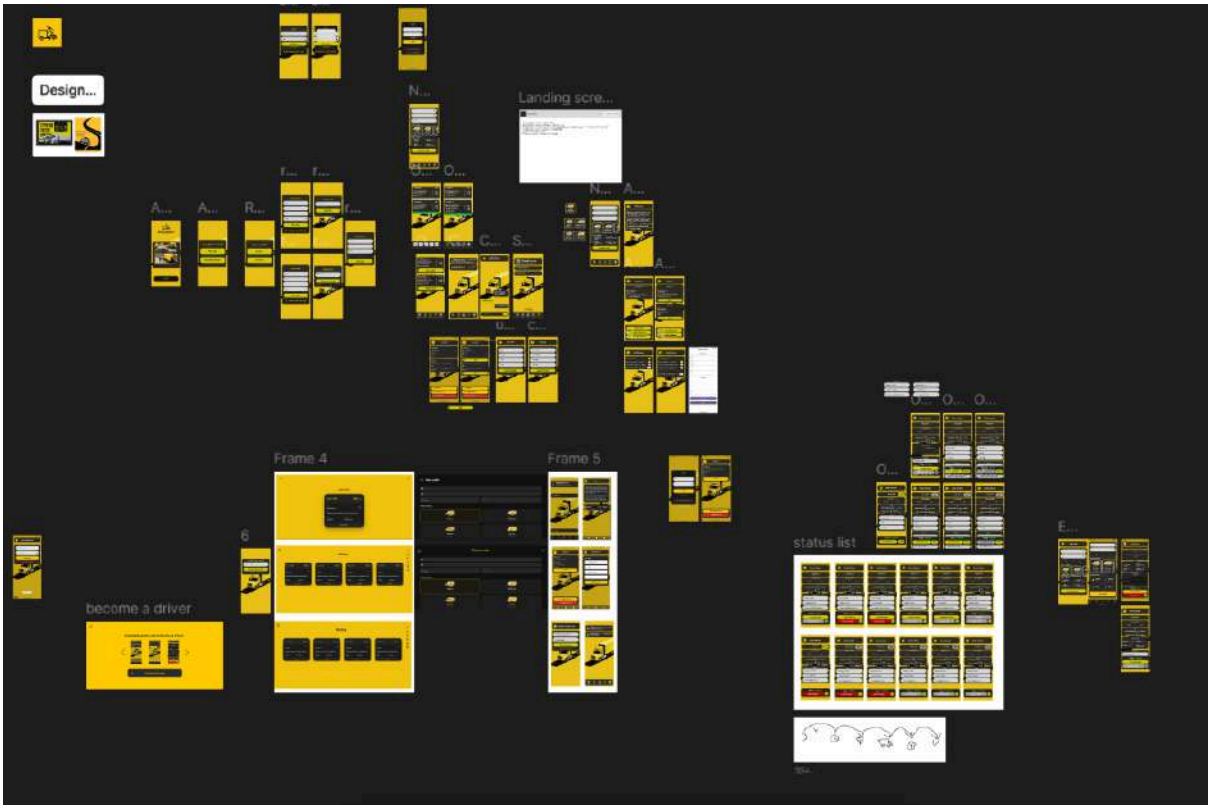


Figure 3.18 Final wireframes of web and mobile applications (source: *Developed by the authors*, Figma)

Design system

Table 4.1 Design schema of the application (source: *Developed by the authors*)

Element	**Realisation**
Primary brand color	Yellow (#FCD12A).
Secondary brand colors	Black (#000000) and dark-gray (#121212).
Text color	White (#FFFFFF) and black (#000000).
Typography	Inter and Poppins, sans-serif (400, 500, 700 weights).
Spacing scale	4px multiples.
Corner radius	12px for buttons, 16px for modals.
Icons	Custom car+gazel based logo and material icons set.

4.3 Technology stack of web application

For the core of the interface. HTML5 [16], CSS3 [5] and JavaScript(ES6+) [20] were implemented as a foundation. In spite of the fact that these are the average in front-end development, the architectural decisions made around them are what drive 'Gazelka' performance. The application is organized as a SPA (Single Page Application) [30] with a custom-built vanilla JS router that actively fetches and injects HTML section files, fully avoiding any heavy framework overhead.

For session management, localStorage [25] to remain JWT authentication [22] was chosen. That allowed for a constant session without hitting the server on every single page transition. Additionally, sessionStorage was used to cache draft order information such as the preferred transport type and edit mode state.

This way, if a user guides back or temporarily leaves the order form, they do not drop their progress and we noticeably reduce redundant API calls to the back-end.
Geography is definitely the core of any logistics system. Due to this, Google Maps API [14] was implemented as the main mapping part. It was joined with Google built in Geocoder [12] for invert geocoding. This is that instead of a user having to manually type in long and complex addresses, the system automatically settles the accurate street address based on the coordinates selected on the map. This step productively removes the headache of manual data entry, reducing human error factor and ensuring that delivery locations are precise from the very start.

Chat was another essential area where the most existing transportation application break. The simultaneous exchange between customer and driver was addressed through SignalR 7.0.7 [29]. This was specifically chosen in order to enable a real-time chat system via WebSockets [35] which is far exceptional to standard polling methods where the application has to demand updates from the server every few seconds. All in all, that setup permits for an instant exchange information.

The application communicates with the back-end through a RESTful API [27] using JSON [21] as the key data layout for all exchanges. So as to assure that the project remains a private environment for both of the participants, a security level based on JWT was utilised with Bearer [4] tokens saved in LocalStorage to maintain a constant yet secure session. In practice, this means the system can approve user identity on every request without making them re-enter their personal information as they navigate between the map, chat and order history while keeping their sensitive data protected.

Passwords are never kept in plain symbols. The backend uses BCrypt [3] to hash passwords before recording them to the database. It assures a robust protection against unauthorized access in case of data breach.

4.4 Technology stack and environment of mobile application

Mobile application is built using Kotlin [23] as the main programming language. It was chosen for its modern syntax, strong type safety and integration with Android systems. This allowed to make a clean codebase.

In order to provide map-based interactions, Google Maps API, similarly to the web application, was integrated into the mobile version. This extension handles an interactive map display, marker placement and route calculations between pick-up and drop-off places. At the same time, Google Places API [bibliographyG] powers the city picking aspect for drivers, enabling them to search and select their service city during registration. This combination omits the need for manual address entry, decreases input

errors and ensures proper geocoding and routing from the very start of the order process.

Real-time chat functionality is implemented using SignalR that creates a persistent WebSocket connection between the mobile app and its back-end, allowing instant message delivery between customers and drivers without constant polling.

Push notifications were managed through Firebase Cloud Messaging [11]. It delivers real-time alerts for main events of order stages, including coming reminders and chat messages. Notifications contain deep-link data (type, orderId and chatId), so that tapping a push opens the proper screen in the app.

Authentication and communication with the back-end rely on JWT tokens issued by the server. After successful login, the token is privately stored in the device SharedPreferences and automatically attached as a Bearer token to every protected API request. This permits constant sessions without requiring the user to re-authenticate on every screen change.

Email verification and password reset streams use Gmail SMTP [bibliographyG] through the back-end. During registration and recovery, the server sends a confirmation or reset code to the user email address. The mobile application then hints the user to provide this code, finishing the secure onboarding process.

Local data persistence is handled by sharedPreferences [28]. It handles the local database for caching order, chat history, user preferences and notification settings, ensuring the app stays operational offline and decreases needless network requests after the initial sync.

4.5 Architecture of web application

As mentioned earlier, the web application stands on three main technologies of HTML, CSS and JavaScript. Below they are shown in the context of the organisational part of this project.

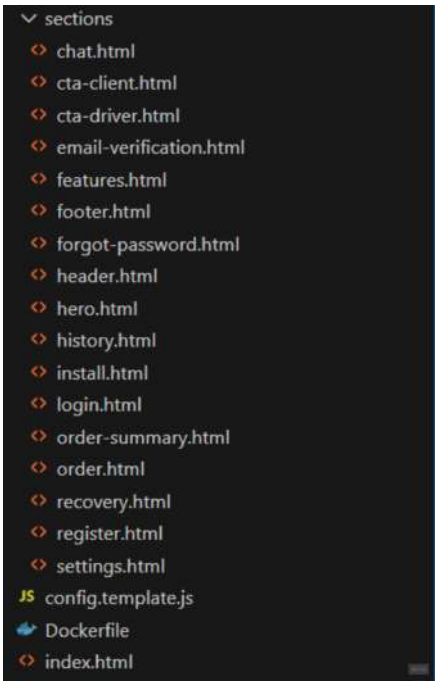


Figure 4.1 HTML structure - Web application (source: *Developed by the authors*, VisualStudioCode)

All the HTML components of the web application are located within the sections folder of the project. Each HTML file describes a separate section as a part of single-page application.

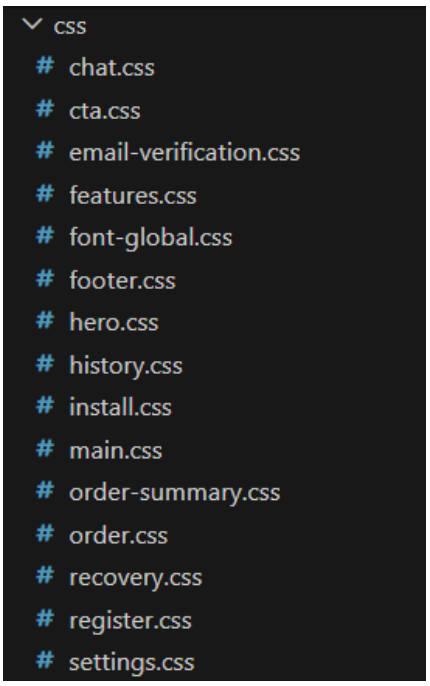


Figure 4.2 CSS structure - Web application (source: *Developed by the authors*, VisualStudioCode)

Similarly to the HTML sections structure, styles are stored within the CSS folder that contains separated styles sheets for each of the sections. It is also necessary to mention that each style's file name corresponds to markup's file name and it was made in order to simplify the overall navigation.

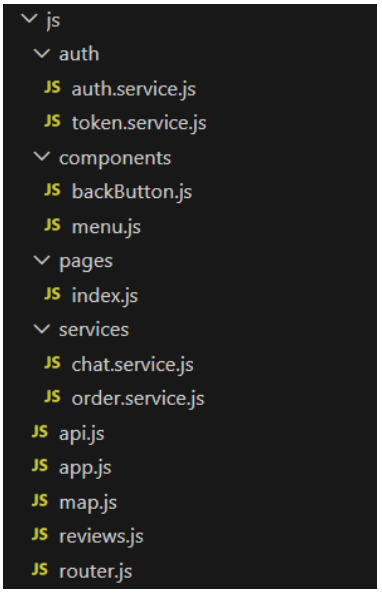
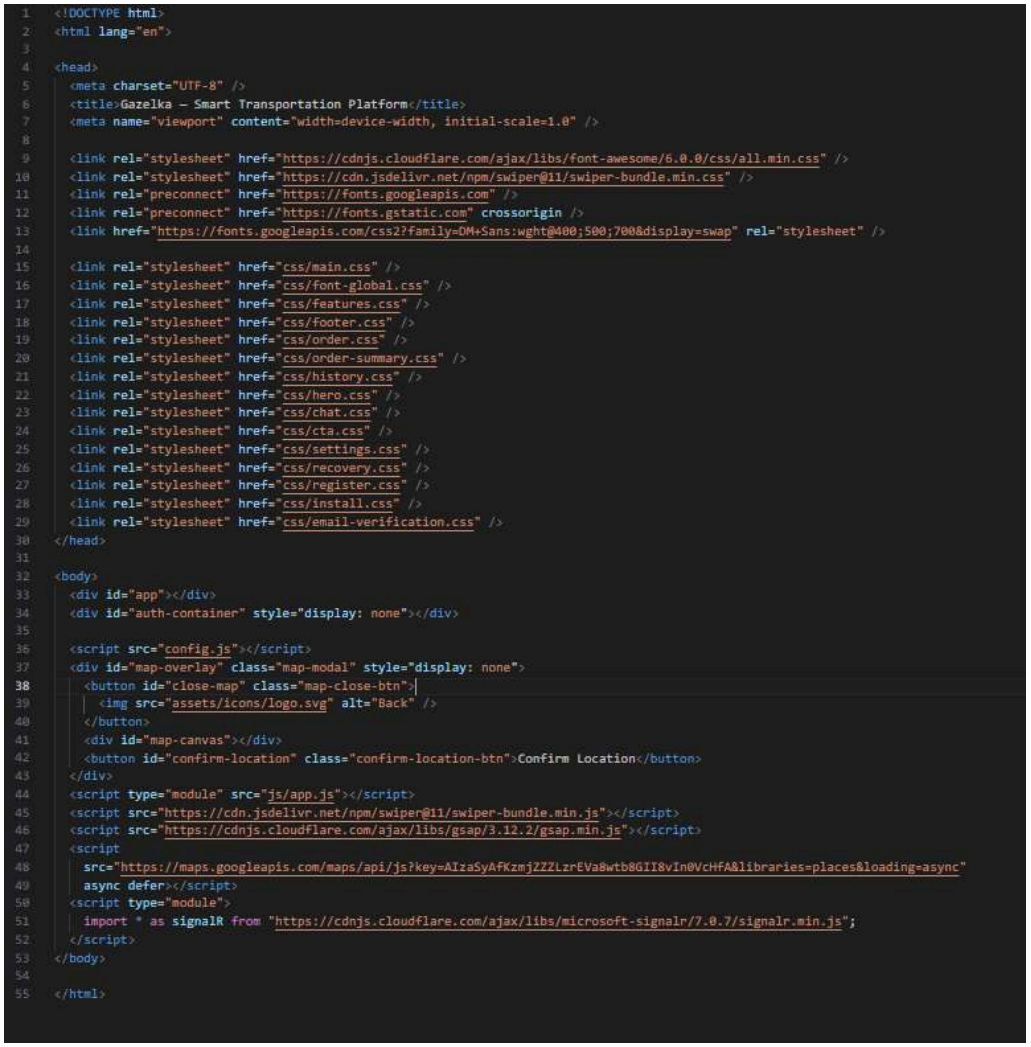


Figure 4.3 JavaScript structure - Web application (source: *Developed by the authors*, VisualStudioCode)

JavaScript contents are divided a little bit more carefully due to their diversity. Within the main folder there are three additional ones and each of them is responsible for certain logics such as user authentication, components, providing services and the main index.js file that serves the whole JavaScript hierarchy.

The index.html file (Figure 4.4) is the main entry point for the whole web application. It serves as a minimal, clean and modular single-page application shell that leads all necessary styles, scripts and external libraries. It then renders the dynamic content into the app id of the body tag (line 33 of the Figure 4.4).



```
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5    <meta charset="UTF-8" />
6    <title>Gazelka — Smart Transportation Platform</title>
7    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
8
9    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" />
10   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/swiper@11/swiper-bundle.min.css" />
11   <link rel="preconnect" href="https://fonts.googleapis.com" />
12   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
13   <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;700&display=swap" rel="stylesheet" />
14
15   <link rel="stylesheet" href="css/main.css" />
16   <link rel="stylesheet" href="css/font-global.css" />
17   <link rel="stylesheet" href="css/features.css" />
18   <link rel="stylesheet" href="css/footer.css" />
19   <link rel="stylesheet" href="css/order.css" />
20   <link rel="stylesheet" href="css/order-summary.css" />
21   <link rel="stylesheet" href="css/history.css" />
22   <link rel="stylesheet" href="css/hero.css" />
23   <link rel="stylesheet" href="css/chat.css" />
24   <link rel="stylesheet" href="css/cta.css" />
25   <link rel="stylesheet" href="css/settings.css" />
26   <link rel="stylesheet" href="css/recovery.css" />
27   <link rel="stylesheet" href="css/register.css" />
28   <link rel="stylesheet" href="css/install.css" />
29   <link rel="stylesheet" href="css/email-verification.css" />
30 </head>
31
32 <body>
33   <div id="app"></div>
34   <div id="auth-container" style="display: none"></div>
35
36   <script src="config.js"></script>
37   <div id="map-overlay" class="map-modal" style="display: none">
38     <button id="close-map" class="map-close-btn">
39       <img src="assets/icons/logo.svg" alt="Back" />
40     </button>
41     <div id="map-canvas"></div>
42     <button id="confirm-location" class="confirm-location-btn">Confirm Location</button>
43   </div>
44   <script type="module" src="js/app.js"></script>
45   <script src="https://cdn.jsdelivr.net/npm/swiper@11/swiper-bundle.min.js"></script>
46   <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/gsap.min.js"></script>
47   <script
48     src="https://maps.googleapis.com/maps/api/js?key=AIzaSyAfKzmjZZZLzrEVa8wtb8GII8vIn0VcHfA&libraries=places&loading=async"
49     async defer></script>
50   <script type="module">
51     import * as signalR from "https://cdnjs.cloudflare.com/ajax/libs/microsoft-signalr/7.0.7/signalr.min.js";
52   </script>
53 </body>
54
55 </html>
```

Figure 4.4 Index HTML - Web application (source: *Developed by the authors*, VisualStudioCode)

4.6 Architecture of mobile application

Comparing to the web application, that is just an addition to the mobile, the second one has a much more complicated structure due to specification and, mainly, because of containing most of the core functions.

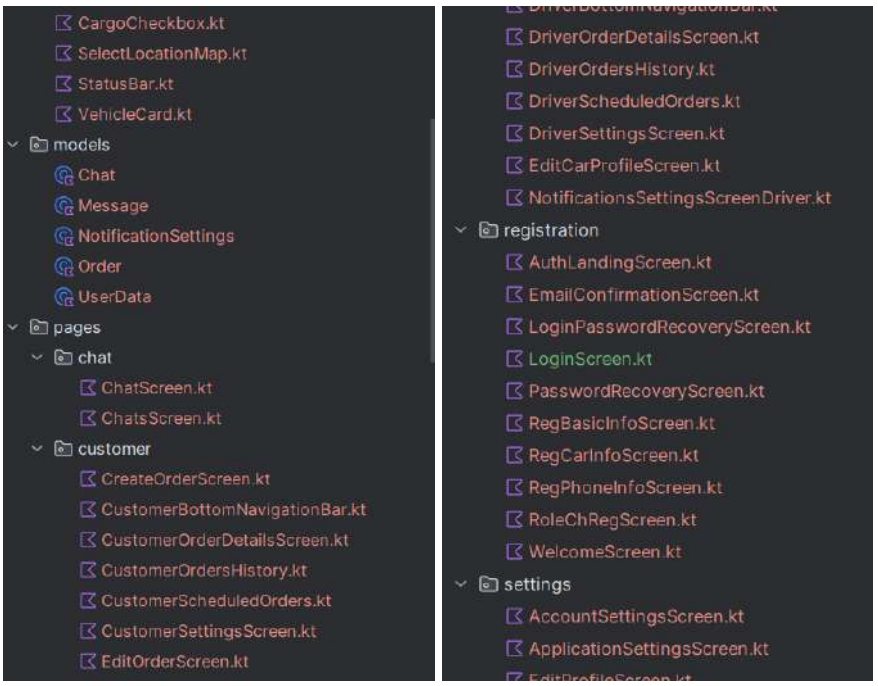


Figure 4.5 Structure of customer and driver components. Kotlin - Mobile application (source: *Developed by the authors*, VisualStudio)

Structure services are carried out to a separate folder for better code management.

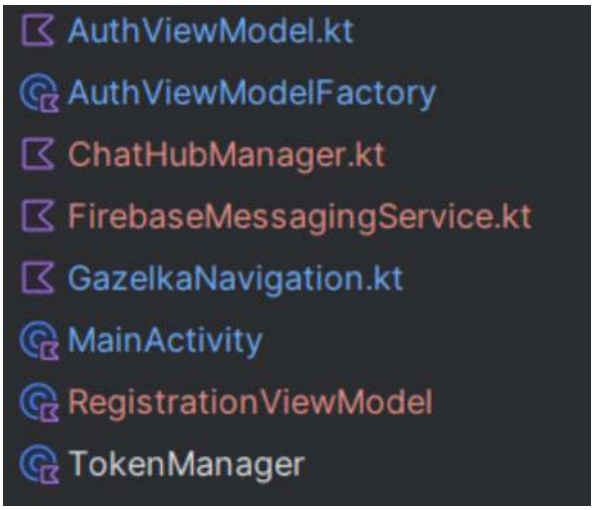


Figure 4.6 Structure services. Kotlin - Mobile application (source: *Developed by the authors*, VisualStudio)

The whole back-end is brought out to another folder group. There, the code snippets are identically placed according to their functionalities (for example, Services, Models, Helpers etc.). The first group of folders on Figure 4.x mostly represents DTO (Data Transfer Organisation) [9] structure, while the other group focuses on services organisation in the project.

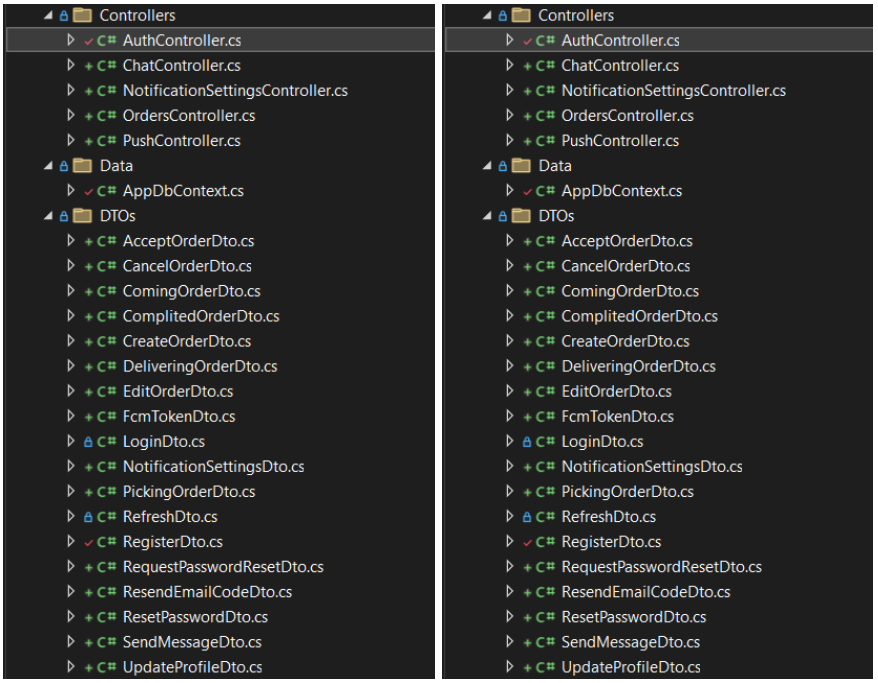


Figure 4.7 Structure of back-end components - Mobile application (source: *Developed by the authors*, VisualStudio)

4.7 System integration of web application

Registration in web application

The purpose of the registration screen is, obviously, allowing new users to enter their core personal details before moving on to the further stages. This screen collects things such as name and surname, email, password and city of service (for drivers). It also performs client-side validation to catch obvious errors.

```
<section class="auth-overlay" id="auth-section">
  <main class="auth-container">
    <div class="auth-card">
      <h2 class="auth-card__title">Registration</h2>

      <form class="auth-form" action="javascript:void(0);">
        <input
          type="text"
          id="reg-name"
          name="name"
          class="auth-input"
          placeholder="Name"
          required
        />
```

Figure 4.8 Registration markup - Web application (source: *Developed by the authors*, VisualStudioCode)

- type attribute on input controls behaviour and validation;
- name attribute controls transmission to the server;
- placeholder is necessary for user guidance

On registration button click, the form activates on-click handler and its main purpose is collecting the data and checking if everything is correct. If something is wrong, then it shows an error massage, so that user can replace information in certain rows.

```
btn.onclick = async () => {
  const email = emailInput.value.trim();
  const password = passwordInput.value.trim();
  const name = document.getElementById("reg-name").value.trim();
  const surname = document.getElementById("reg-surname").value.trim();
  const role = "customer";
  const carType = null;
  const carColor = null;
  const carNumber = null;
  const phoneNumber = phoneInput.value.trim();
  const cityName = null;

  const isEmailValid = validateEmail();
  const isPhoneValid = validatePhone();
  const isPasswordValid = validatePassword();

  if (!isEmailValid || !isPhoneValid || !isPasswordValid) {
    if (!isEmailValid) {
      emailInput.classList.add('input-error');
      emailError.classList.add('show');
    }
    if (!isPhoneValid) {
      phoneInput.classList.add('input-error');
      phoneError.classList.add('show');
    }
    if (!isPasswordValid) {
      passwordInput.classList.add('input-error');
      passwordError.classList.add('show');
    }
    return;
  }
```

Figure 4.9 Registration button on-click handler - Web application (source: *Developed by the authors*, VisualStudioCode)

1. Cleans and gathers the input values. Trims them be removing extra spaces;
2. Runs local validation checks using validation functions;
3. If anything is invalid, shows an error message and stops execution;
4. If anything is alright, then there is the next step of registration logic described in the next piece of code

The code snippet continues the previous on-click handler. Here it actually makes sure all the fields are not empty and gathers all the information from the form onto sessionStorage.

```
    if (!email || !password || !name || !surname || !phoneNumber) {
      return alert("Please fill all fields");
    }

    sessionStorage.setItem("tempRegData", JSON.stringify({
      email, password, name, surname, role, carType, carColor, carNumber, phoneNumber, cityName
    }));

    const res = await AuthService.register({email, password, name, surname, role, carType, carColor, carNumber, phoneNumber, city

    if (!res.success) {
      alert(res.error);
      return;
    }


    await Router.navigate("email-verification");
  };
}
};
```

Figure 4.10 Registration button on-click handler - Web application (source: *Developed by the authors*, VisualStudioCode)

1. Checks all the fields finally and returns an alert if everything is missing;
2. Saves drafts to the sessionStorage as a temporary copy before sending to the server;
3. Sends registration request to the back-end;
4. Handles server response;
5. Navigates to email verification as a successful path

Definition of authentication service is described in the next part that handles two most important features. These are the login and registration. It uses API helper in order to talk with the main C# [6] server that serves needs of both applications.

```
import { API } from "../api.js";
import { TokenService } from "./token.service.js";

export const AuthService = {
  async login(email, password) {
    try {
      const res = await API.post("Auth/login", { email, password });

      TokenService.save(res.accessToken, res.refreshToken);

      return { success: true };
    } catch (err) {
      return { success: false, error: err.message };
    }
  },

  async register(data) {
    try{
      await API.post("Auth/register", data);
      return { success: true };
    }catch(err){
      return { success: false, error: err.message };
    }

  },
```

Figure 4.11 Authentication logic. 1st part - Web application (source: *Developed by the authors*, VisualStudioCode)

1. Imports API helper and exports the authentication service in reusage purposes;
2. Applies login method if user's action is exactly log in. it sends a post request with email and password;
3. Applies register method if user is creating an account. It also sends a post request but with the bunch of necessary registration details;

The last registration snippet permits outgoing requests to the server by adding authentication headers automatically, parsing responses and managing errors. This is an API wrapper that connects the front-end with back-end.

```
import { TokenService } from "./auth/token.service.js";

const BASE_URL = `http://${window.__CONFIG__.SERVER_IP}:${window.__CONFIG__.SERVER_PORT}/api/`;



export const API = {
  async post(path, body) {
    const token = TokenService.getAccess();
    const headers = { "Content-Type": "application/json" };
    if (token) headers["Authorization"] = `Bearer ${token}`;

    const res = await fetch(BASE_URL + path, {
      method: "POST",
      headers: headers,
      body: JSON.stringify(body),
    });

    let data = null;
    const text = await res.text();
    if (text) {
    try {
      data = JSON.parse(text);
    } catch (e) {
      console.warn("Response is not JSON:", text);
      data = null;
    }
  }

  if (!res.ok) {
    throw new Error(data?.error || `Server error: ${res.status}`);
  }

  return data;
},
```

Figure 4.12 Authentication logic. 2nd part - Web application (source: *Developed by the authors*, VisualStudioCode)

1. Imports TokenService to retrieve the current JWT access token from storage;
2. Constructs basic URL;
3. Exports API object with the main POST method that takes two arguments: endpoint path and body;
4. Automatically injects JWT Bearer token by getting current access from TokenService;
5. Gets JSON content type for the server;
6. Sends the server request owing to fetch method;
7. Handles and parses the response;
8. Throws and handles errors;
9. Returns parsed data on success;

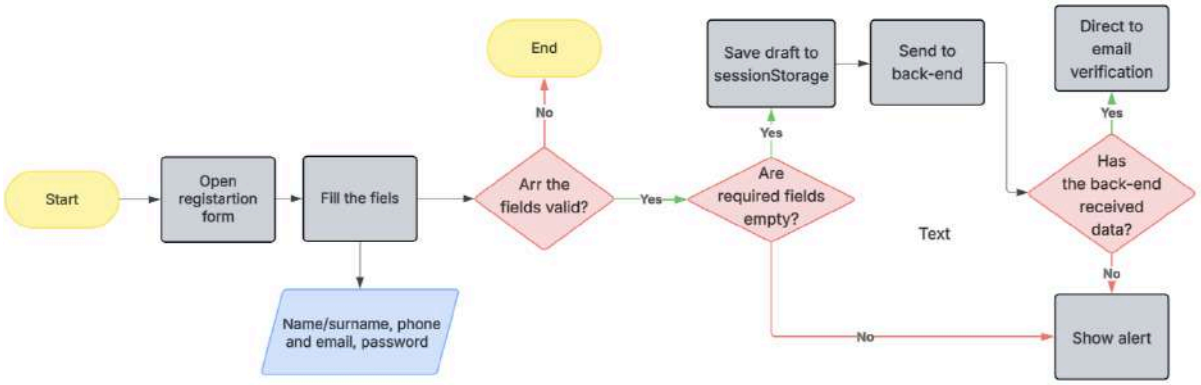


Figure 4.13 Registration flowchart - Web application (source: *Developed by the authors*, Lucidchart)

Email verification in web application

The compulsory registration step for both customer and driver. Email verification form is similar to the previous form due to inputs, attributes and other properties. Here, a user has to enter 6-digit email approval code they receive. Notice, that on this stage emails cannot be changed.

```
<section class="auth-overlay" id="auth-section">
  <main class="auth-container">
    <div class="auth-card">
      <h2 class="auth-card__title">Email Verification</h2>
      <p class="auth-card__subtitle">
        We've sent a verification code to your email. Please enter it below to complete registra
      </p>

      <form class="auth-form" action="javascript:void(0);">
        <div class="input-box">
          <label>Email</label>
          <input
            type="email"
            id="verify-email"
            class="auth-input"
            placeholder="Your email"
            required
            readonly
          />
        </div>

        <div class="input-box">
          <label>Verification code</label>
          <input
            type="text"
            id="verification-code"
            class="auth-input"
            placeholder="Enter 6-digit code"
            required
            maxlength="6"
          />
        </div>

        <button type="button" class="btn--auth-submit" id="verify-email-btn">
          Verify Email
        </button>
```

Figure 4.14 Email verification markup - Web application (source: *Developed by the authors*, VisualStudioCode)

The JavaScript implementation represents the on-click event handler for the Verify button. It responsible for collecting the code, retrieving email from temporary storage and sending the code.

```
verifyBtn.onclick = async () => {
  const code = document.getElementById("verification-code")?.value.trim();

  if (!code) {
    return alert("Please enter verification code");
  }

  if (!tempRegData.email) {
    return alert("Email not found. Please register again.");
  }

  const verifyRes = await AuthService.verifyEmail(tempRegData.email, code);
  if (!verifyRes.success) return alert(verifyRes.error);

    sessionStorage.removeItem("tempRegData");
    alert("Email verified successfully! Please login.");
    await Router.navigate("login");

};
```

Figure 4.15 Email verification logic - Web application (source: *Developed by the authors*, VisualStudioCode)

1. Reads and trims the value from the code input;
2. Checks if the code is not empty;
3. Retrieve email from temporary storage by getting and parsing the draft registration data;
4. Sends verification code to the back-end and handles its response;
5. Shows a success message;

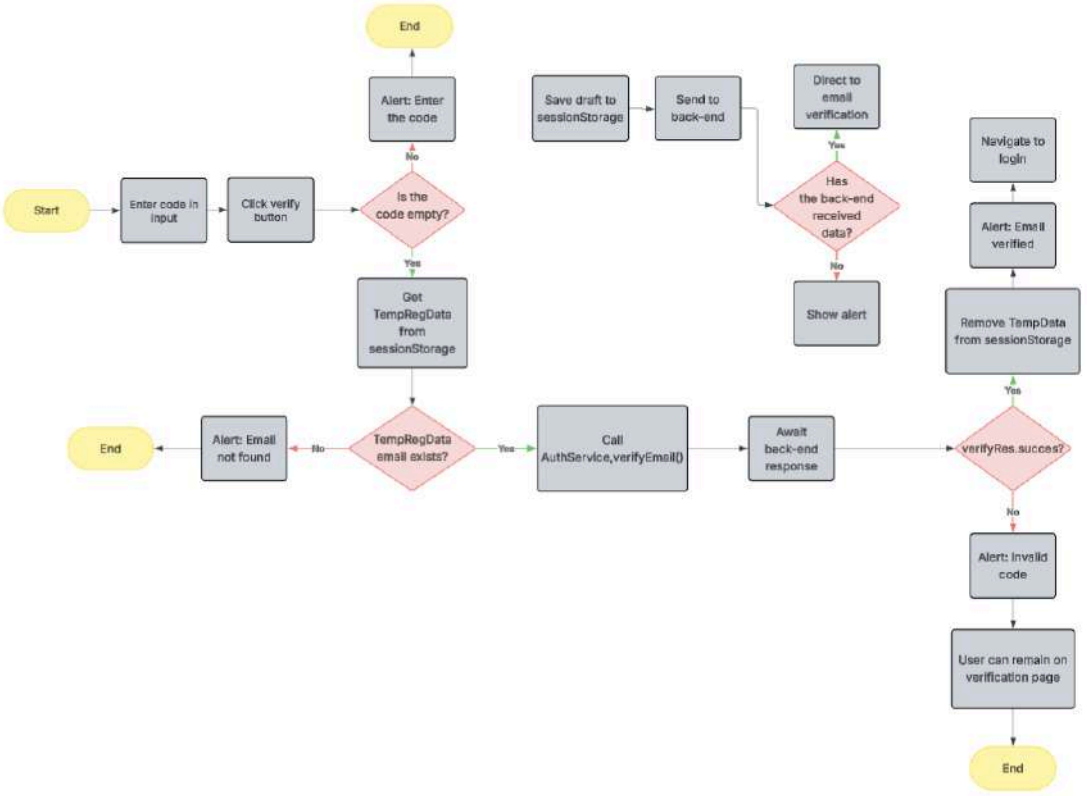


Figure 4.16 Email verification - Web application (source: *Developed by the authors*, Lucidchart)

Password recovery in web application

If it occurs that a user forgets their password, certain functionalities that are implemented in application for this purpose take place. The goal of request password reset function is to check if a user exists, generate, save and send verification code.

```
<form class="auth-form" id="forgot-password-form">
  <div class="input-box">
    <label style="font-family: inherit;">Code from email</label>
    <input
      type="text"
      id="recovery-code"
      placeholder="Enter code from email"
      required
      style="font-family: inherit;"
    />
  </div>

  <div class="input-box">
    <label style="font-family: inherit;">New password</label>
    <div class="password-wrapper">
      <input
        type="password"
        id="new-password"
        class="auth-input"
        placeholder="Enter new password"
        required
        style="font-family: 'DM Sans', sans-serif;"
      />
```

Figure 4.17 Password recovery markup - Web application (source: *Developed by the authors*, VisualStudioCode)

Password recovery core logic contains a couple of separate on-click handlers for buttons in password reset flow. Together they implement recovery code request and submit the new password.

```
sendCodeBtn.onclick = async () =>{
  const email = document.getElementById("recovery-email").value.trim();
  if (!email) {
    return alert("Please enter email");
  }
  const resp = await AuthService.requestPasswordReset({email});
  if (!resp.success) return alert(resp.error);
}

saveBtn.onclick = async () => {
  const code = document.getElementById("recovery-code")?.value.trim();
  const newPassword = newPasswordInput?.value.trim();
  const email = document.getElementById("recovery-email")?.value.trim();
  if (!code || !newPassword ) {
    return alert("Please fill all fields");
  }

  const res = await AuthService.resetPassword(email, code, newPassword);
  if (!res.success) return alert(res.error);

  alert("Password successfully changed!");
  await Router.navigate("login");
};
```

Figure 4.18 Password recovery logic - Web application (source: *Developed by the authors*, VisualStudioCode)

1. The first handler runs when the user requests a new password. It collects email, provides a quick empty check and alert if the email was not found. After that, it calls AuthService.requestPasswordReset(email) and sends a request. Back-end generates a code and sends it via email. Finally, it handles the response;
2. The second handler runs when the user enters the code and a new password. It collects data from inputs and checks if the required fields are filled. Later on it calls AuthService.requestPasswordReset(email, code, newPassword) and sends a request as well. The response here also remains handled;

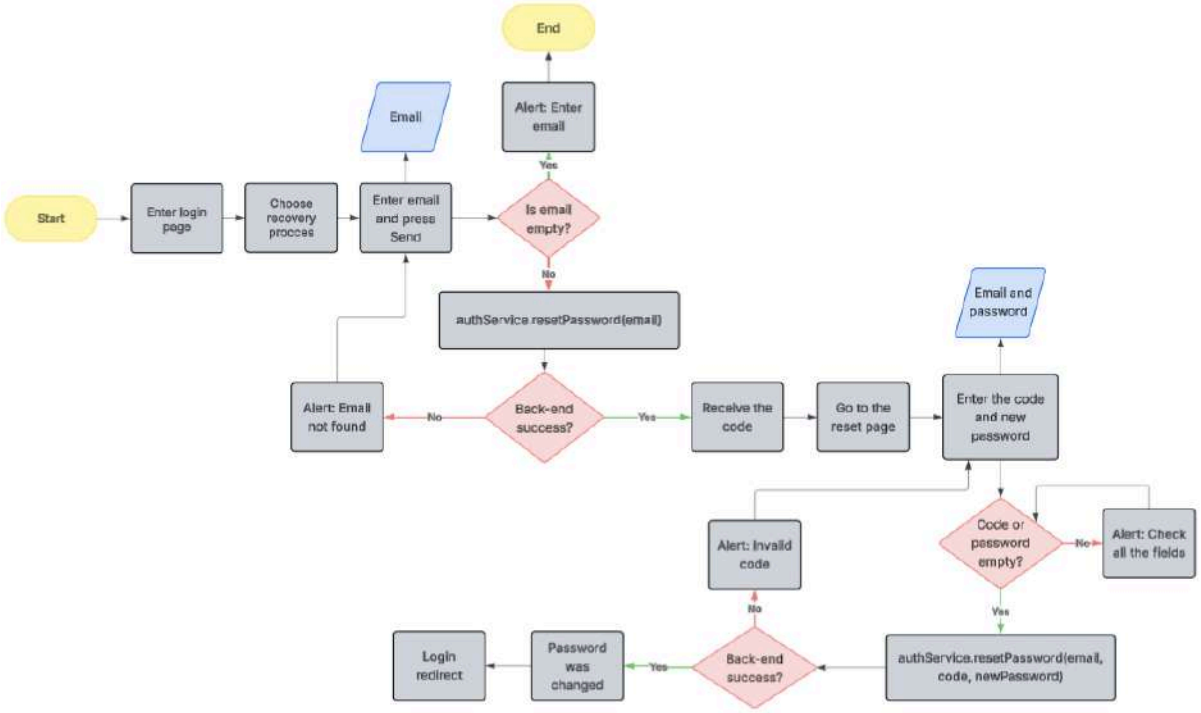


Figure 4.19 Password recovery flowchart - Web application (source: *Developed by the authors*, Lucidchart)

New order in web application

Order placement is the most essential function of the whole platform. It serves the main customer need of placing an order that includes plenty of user-friendly preferences.

```
<div class="order-form-content">
  <div class="address-group">
    <div class="input-wrapper">
      <img src="assets/icons/ic_location.svg" />
      <input type="text" id="from-location" placeholder="From:" />
    </div>
    <div class="input-wrapper">
      <img src="assets/icons/ic_location.svg" />
      <input type="text" id="to-location" placeholder="To:" />
    </div>
  </div>

  <div class="datetime-row">
    <div class="dt-input" onclick="document.getElementById('order-date').showPicker()">
      <label>Date</label>
      <input type="date" id="order-date" />
    </div>
    <div class="dt-input" onclick="document.getElementById('order-time').showPicker()">
      <label>Time</label>
      <input type="time" id="order-time" />
    </div>
  </div>
```

Figure 4.20 New order markup - Web application (source: *Developed by the authors*, VisualStudioCode)

On the attached Figure 4.20 there is the order creation form that includes multiple tags inside. General structure is shown on the example code of pick-up/drop-off locations choice and date/time interactive inputs. The first one is connected to Google Maps API, when the second one improves user experience by providing an intuitive option of not entering date and time of order manually.

```
export function initGoogleMap() {
  if (map) return;

  geocoder = new google.maps.Geocoder();

  map = new google.maps.Map(document.getElementById("map-canvas"), {
    center: { lat: 52.2297, lng: 21.0122 }, // Warsaw
    zoom: 12,
    disableDefaultUI: true,
  });

  map.addListener("click", (e) => {
    const latLng = e.latLng;

    if (marker) marker.setMap(null);

    marker = new google.maps.Marker({
      position: latLng,
      map,
    });

    reverseGeocode(latLng);
  });
}
```

Figure 4.21 Interactive map logic - Web application (source: *Developed by the authors*, VisualStudioCode)

1. This instant early exits if the interactive map already exists that prevents system overload;
2. Created Geocoder instance that is stored in global geocoder variable used for later reverse geocoding;
3. Created and configures the map;
4. Adds click listener to the map. Remove an older marker if it already exists and creates a new one on the clicked position;
5. Warsaw city center is set a default location according to coordinates;

Chat in web application

Communication is another important point of each double-role system, especially a moving or logistics one. It is significant due to the opportunity to stay updated of every order manipulation or unplanned circumstances.

```
<div class="chat-conversation-view" id="chat-conversation-view" style="display: none;">
  <div class="chat-messages" id="chat-messages">
    <div class="message driver">
      <div class="message-bubble">
      </div>
      <span class="message-time"></span>
    </div>
  </div>

  <div class="chat-input-area">
    <div class="chat-input-wrapper">
      <input type="text" id="chat-input" placeholder="Type a message..." autocomplete="off" /
      <button id="send-msg-btn" class="send-btn">
        <img src="assets/icons/ic_send.svg" alt="Send" />
      </button>
    </div>
  </div>
</div>
```

Figure 4.21 Chat markup - Web application (source: *Developed by the authors*, VisualStudioCode)

The following method (Figure 4.21) broadcasts new chat messages in real-time to both participants and conditionally sends push notifications.

```
var participants = new List<int> { chat.User1Id };
if (chat.User2Id.HasValue) participants.Add(chat.User2Id.Value);

foreach (var participantId in participants)
{
    await hub.Clients.Group(participantId.ToString())
    .SendAsync("ReceiveMessage",
        message.Id,           // int
        message.ChatId,       // int
        message.SenderId,     // int
        message.Text,         // string
        message.SentAt.ToString("yyyy-MM-ddTHH:mm:ssZ")  // string
    );
    if (participantId != senderId)
    {
        var isUserInChat = _presence.IsUserInChat(participantId, chat.Id);

        if (!isUserInChat)
        {
            var tokens = await _db.UserTokens
                .Where(t => t.UserId == participantId)
                .Select(t => t.FcmToken)
                .ToListAsync();

            foreach (var token in tokens)
            {
                var settings = await _db.UserNotificationSettings
                .FirstOrDefaultAsync(x => x.userId == participantId);
                if (settings?.chatEnabled == true)
                    await _pushService.SendPushAsync(
                    token,
                    "New message",
                    message.Text,
                    new Dictionary<string, string>
                    {
                        ["type"] = "chat",
                        ["orderId"] = chat.OrderId.ToString(),
                        ["chatId"] = chat.Id.ToString()
                    }

                );
            }
        }
```

Figure 4.22 Chat logic - Server (source: *Developed by the authors*, VisualStudio)

1. Builds a list of participants starting with the first user that is the customer who created an order and then adds the driver;
2. Broadcasts each message to both sides using SignalR. Uses group-based broadcasting and send ReceiveMessage event to both participants;
3. Skips push for the sender themselves in order to avoid duplicates;
4. Checks if the recipient currently viewing the chat;
5. Fetches all FCM tokens for the recipient;
6. Checks user notification settings and sends push to each token. Loads user notification preferences and sends only if chat notifications are enabled;

Successful SignalR connection ensures real-time messages exchange smoothly. It is important to describe its integration into the JavaScript code.

```
const SERVER_IP = window.__CONFIG__.SERVER_IP;
const SERVER_PORT = window.__CONFIG__.SERVER_PORT;

const connection = new signalR.HubConnectionBuilder()
  .withUrl(`http://${SERVER_IP}:${SERVER_PORT}/chatHub`, {
    accessTokenFactory: () => TokenService.getAccess()
  })
  .withAutomaticReconnect()
  .build();

connection.onclose(err => console.error("SignalR closed", err));
connection.onreconnecting(err => console.warn("SignalR reconnecting", err));
connection.onreconnected(() => console.log("SignalR reconnected"));

connection.on("ReceiveMessage", (id, chatId, senderId, text, sentAt) => {
  const msg = { id, chatId, senderId, text, sentAt };
  console.log("Incoming message:", msg);
 if (chatId == activeChatId) {
addMessageToDOM(msg);
 }
});

try {
  await connection.start();
  console.log("SignalR connected");
} catch (err) {
  console.error("SignalR connection failed:", err);
}
```

Figure 4.23 SignalR connection - Web application (source: *Developed by the authors*, VisualStudio)

1. Dynamically defines server URL using global config;
2. Builds SignalR connection. Connects to the chatHub endpoint on the back-end. Automatically attaches JWT Bearer token from TokenService, allowing back-end to authenticate the user. Lastly, enables automatic reconnect;
3. Logs connection events;
4. Listen for incoming messages;
5. Initiates WebSocket connection;

4.8 System integration of mobile application

Login and registration in mobile application

```
Button(
    onClick = {
        nameError = false
        surnameError = false
        emailError = false
        passwordError = false
        cityError = false

        var valid = true
        val emailRegex = Regex( pattern = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")

        if (name.isBlank()) {
            nameError = true; valid = false
        }
        if (surname.isBlank()) {
            surnameError = true; valid = false
        }
        if (email.isBlank() || !emailRegex.matches( input = email.trim())) {
            emailError = true; valid = false
        }
        if (password.length < 8) {
            passwordError = true; valid = false
        }
        if (regViewModel.cityName.isBlank() && regViewModel.role == "driver") {
            cityError = true; valid = false
        }

        if (valid) {
            regViewModel.name = name.trim()
            regViewModel.surname = surname.trim()
            regViewModel.email = email.trim()
            regViewModel.password = password.trim()
            navController.navigate( route = "phoneInfoReg")
        }
    },
```

Figure 4.24 The first stage of account registration (source: *Developed by the authors*, AndroidStudio)

Initially all error flags of the registration form are set to false (nameError, surnameError, emailError, passwordError, cityError). Sequential checks ensure the proper record of the data from registration form input, validating each of the points just while an input is being filed. The checks are eligible with certain criteria:

1. Name and surname must not be empty;
2. Email must not be empty and must match email regex;
3. Password length must be more than 8 symbols;
4. For driver role the city of service must not be empty;

Email regex used (raw from code): ^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$. It allows letters, numbers, +, _,., in local part, requires @ + domain +.+ TLD of at least two letters. Example that fails - user@.com and that passes - user@gmail.com.

If any check fails valid type becomes false, then the screen remains open and error messages appear under the invalid fields. At the same time, if all checks pass, then the values are trimmed and safe.

Figure 4.25 The second stage of account registration (source: *Developed by the authors*, AndroidStudio)

The next screenshot (Figure 4.25) describes the collection of user phone number and completing the basic information that has already been successfully validated. This is the last stage before initiation of the actual registration call to back-end.

A field is considered invalid if it is empty after trimming or does not match the phoneRegex. If the field is invalid, then a user keeps staying on the screen and the error. Now we can deeply observe the role-branching that is the main decision of registration flow. It is represented on the rest of the attached figure. According to the lines of the code, the division of the roles after phone number verification is following:

1. Customer: Immediate back-end registration - Successful operation - Email confirmation screen;
2. Driver: No registration call yet in order to collect more information on the next stage - redirection to vehicle information screen;

When the registration role is a customer (regViewModel.role == “customer”), then:

- The coroutine scope is launched;
- The authentication is called;

When the case is successful, the user is navigated to the Email Confirmation screen, passing the email in the route, the stack is cleared back to Welcome Screen and the user cannot go back after successful registration.

When the case is error, then Android Toast notification is shown and the user remains on the phone input screen.

The driver flow is pretty simple as well - they are redirected on the next page in order to fill in vehicle information. The driver registration will be sent only after collecting their car details.

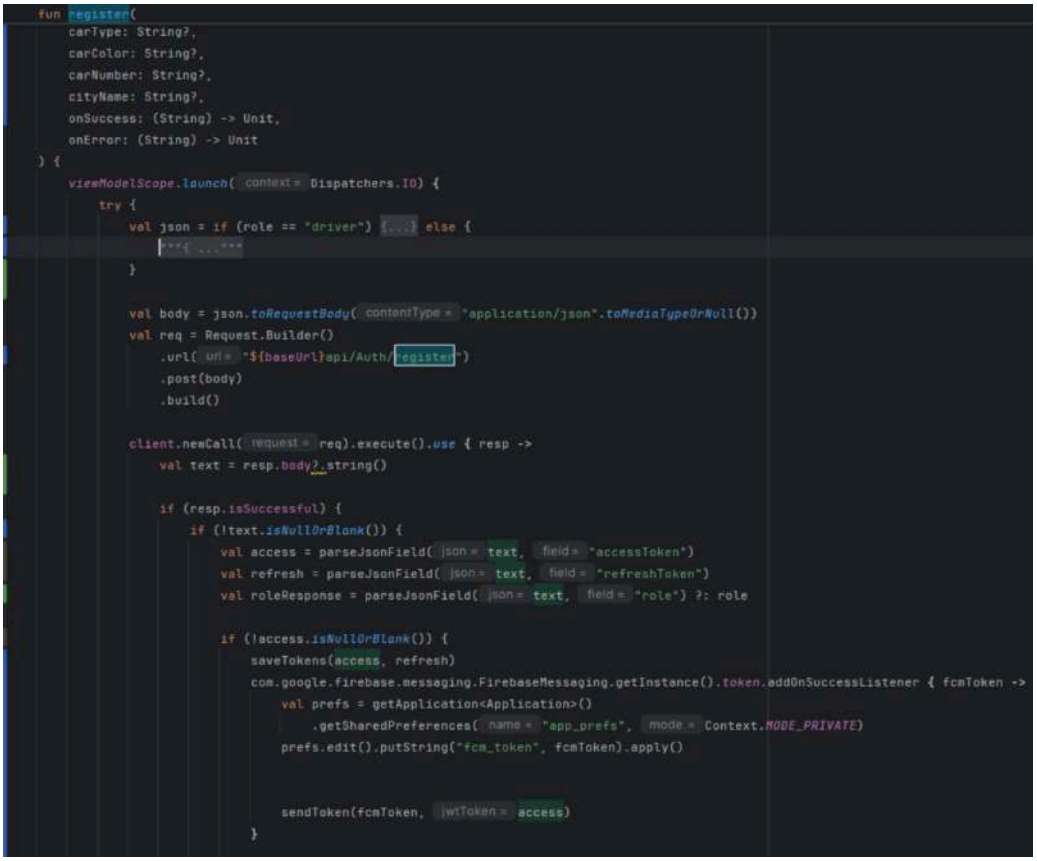


Figure 4.26 HTTP POST request request (source: *Developed by the authors*, AndroidStudio)

This picture shows back-end logic of the final registration step. Its purpose is performing the actual HTTP Post request to create the user account. The function is called:

- For customers - after phone verification number on the previous screen;
- For drivers - only after all additional driver-specific data has been collected;

The register function logic explained step-by-step:

1. Launches coroutine on ViewModelScope with Dispatchers.IO (network-safe context) [7];
2. Builds JSON payload conditionally: If the role is driver - includes CarType, carColor, CarNumber and cityName. If the role is customer - these fields are omitted*;*
3. Creates HTTP [17] request using POST method*;*
4. Handles server response. If the response is successful - accessToken, refreshToken are generated;

```
[HttpPost("register")]
0 references
public async Task<IActionResult> Register([FromBody] RegisterDto dto)
{
    try
    {
        var user = await _auth.RegisterAsync(dto.Email, dto.Password, dto.Name, dto.Surname, 
        var ip = HttpContext.Connection.RemoteIpAddress?.ToString();
        var ua = Request.Headers["User-Agent"].ToString();
        var (access, refresh) = await _auth.LoginAsync(dto.Email, dto.Password, ip, ua);

        await _auth.SendEmailConfirmationCodeAsync(dto.Email);

        return Ok(new
        {
            accessToken = access,
            refreshToken = refresh,
            id = user.Id,
            email = user.Email,
            name = user.Name,
            surname = user.Surname,
            role = user.Role,
            carType = user.CarType,
            carColor = user.CarColor,
            carNumber = user.CarNumber,
            phoneNumber = user.PhoneNumber,
            cityName = user.CityName,
        });

    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { error = ex.Message });
    }
}
```

Figure 4.27 HTTP REGISTER request - Mobile application (source: *Developed by the authors*, AndroidStudio)

The client sends a JSON object with the necessary fields filled in and this is where the main business logic activates:

1. It receives DTO from the request body;
2. Calls internal authentication service;
3. Captures client metadata for security and logging - IP addres and User Agent;
4. Automatically logs the newly created user in. It returns access and refresh tokens, vomiting separate login because user becomes already authenticated;
5. Triggers email confirmation. Sends a 6-digit confirmation code to the user's email;
6. Returns successful registration response;
7. Error handling is realised by utilizing Catch block that returns 400 request with a JSON;

The last registration step in the system is creation of a new user record in the database after validating their email. The password can also be hashed and persist the user.

```
public async Task<User> RegisterAsync(string email, string password, string? Name = null, strin
{
    email = email.ToLowerInvariant().Trim();
    if (await _db.Users.AnyAsync(u => u.Email == email))
        throw new InvalidOperationException("Email already in use");


    var hashed = BCrypt.Net.BCrypt.HashPassword(password);
    var user = new User { Email = email, PasswordHash = hashed, Name = Name, Surname = Surname,
    _db.Users.Add(user);
    await _db.SaveChangesAsync();
    return user;
}
```

Figure 4.28 RegisterAsync function (source: *Developed by the authors*, AndroidStudio)

1. This function normalizes email by making it case-sensitive and removing extra spaces;
2. Check for existing email. This prevents duplicate registration;
3. Hashes the password using BCrypt - the password is not stored as a plain text;
4. Creates a new user entity;
5. Saves result to the database. It inserts the row into the users table;
6. Returns created user object;

The main goal of API endpoint is managing new user registration, creating account in the database and automatically log the user in, sending confirmation code using email and, finally, returning authentication tokens (Figure 4.29);

	Id	Email	PasswordHash	Name	Surname	PhoneNumber
	Fil...	Filter	Filter	Filter	Filter	Filter
1	1	d	$2a$11$B9K0lwEkE8/…	Denis	Petuhov	12321321
2	2	adzhula@edu.cdv.pl	$2a$11$UU81SWczfbjvpatUhadoXu4L2Clb/…	Curikiii	Fufik	21321321

PhoneNumber	Role	CarType	CarColor	CarNumber	CreatedAt	FailedLoginAttempts
Filter	Filter	Filter	Filter	Filter	Filter	Filter
12321321	driver	Large Van	Green	sadsd	2026-02-11 02:05:19.4773722	0
21321321	customer	NULL	NULL	NULL	2026-02-11 02:06:29.966597	0
112	customer	NULL	NULL	NULL	2026-02-12 11:37:52.1524517	0
+03003020202	customer	NULL	NULL	NULL	2026-02-12 11:38:43.5210484	0

LockoutUntil	EmailConfirmed	EmailConfirmationCode	PasswordResetCode	CityName
Filter	Filter	Filter	Filter	Filter
NULL	1	NULL	NULL	Poznań
NULL	1	NULL	NULL	NULL
NULL	0	NULL	NULL	NULL
NULL	0	NULL	NULL	NULL

Figure 4.29 Account row in the database (source: *Developed by the authors*, Android studio)

Driver city of service picker in mobile application

City picker, as a small utility function, outlines an important step in driver registration. This feature is rather unique, being available for a single role. It creates and configures a launch of Google Places autocomplete in full-screen mode, specifically tuned for city selection.

```
fun createCityPickerIntent(activity: ComponentActivity) =
    Autocomplete.IntentBuilder(
        mode = AutocompleteActivityMode.FULLSCREEN,
        placeFields = listOf(
            Place.Field.ID,
            Place.Field.NAME,
            Place.Field.ADDRESS_COMPONENTS
        )
    )
        .setTypesFilter(listOf("locality"))
        .build( context = activity)
```

Figure 4.30 HTTP REGISTER request (source: *Developed by the authors*, AndroidStudio)

1. Initially this function creates the autocomplete intent builder, using full screen mode. It gives a cleaner and more useful UI;
2. Nextly it specifies which place fields have to returned and this data includes a unique Google Places id, city name along with city address;
3. Restricts results to localities only by telling Google Places to return cities only. It excludes streets, address etc. After that it uses the activity as context in order to build the Intent;

Email confirmation in mobile application

```
public async Task SendEmailConfirmationCodeAsync(string email)
{
    if (string.IsNullOrWhiteSpace(email))
        return;

    if (!new EmailAddressAttribute().IsValid(email))
        return;

    var user = await _db.Users.FirstOrDefaultAsync(x => x.Email == email);
    if (user == null)
        return;

    if (user.EmailConfirmed)
        return;

    user.EmailConfirmationCode = GenerateCode();

    await _db.SaveChangesAsync();

    await _emailService.SendAsync(
        email,
        "Email verification",
        $"Your code: {user.EmailConfirmationCode}"
    );
}
```

Figure 4.31 Email confirmation code function - Mobile application (source: *Developed by the authors*, Android studio)

It is called right after successful registration (as seen in the POST controller mode: await)

1. This method check early exit and skips only if input is valid;
2. Validates email format;
3. Find the user in the database. It looks user up by exact email and exits if a user was not found;
4. Skips if email is already confirmed. It prevents spamming confirmation emails;
5. Generates ans stores new confirmation code. The code is directly saved to the Users table;
6. Sends the email itself, using inject email service;

Currently we are moving to the next instruction that sends verification emails. It constructs text messages and reliably delivers it.

```
public async Task SendAsync(string to, string subject, string bod
{
    var message = new MimeMessage();
    message.From.Add(new MailboxAddress(
        _configuration["Email:FromName"],
        _configuration["Email:FromAddress"]
    ));
    message.To.Add(MailboxAddress.Parse(to));
    message.Subject = subject;

    message.Body = new TextPart("plain")
    {
        Text = body
    };

    using var client = new SmtpClient();

     await client.ConnectAsync(
        _configuration["Email:SmtpHost"],
        int.Parse(_configuration["Email:SmtpPort"]),
        SecureSocketOptions.StartTls
    );

    await client.AuthenticateAsync(
        _configuration["Email:Username"],
        _configuration["Email:Password"]
    );

    await client.SendAsync(message);
    await client.DisconnectAsync(true);
}
```

Figure 4.32 Send email function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The function accepts certain parameters like recipient email, subject line and content of the email;
2. Creates a MimeMessage object form MailKit library;
3. Sets a sender due to name and address read in the configuration, a recipient a subject and a body as plain tex;
4. Creates SMTP client and connects. It reads SMTP server host from configuration and uses STARTTLS that is an upgraded plain connection to TLS;
5. Authenticates by using the given name and password;
6. Sends the message;
7. Disconnects;

The other picture (Figure 4.32) represents the back-end verification endpoint logic and this is the final step that closes the email confirmation loop. The purpose of this is to confirm user ownership of the email address by entering the verification code that has previously been sent via email.

```
public async Task VerifyEmailAsync(string email, string code)
{
    if (string.IsNullOrWhiteSpace(email) || string.IsNullOrWhiteSpace(code))
        throw new InvalidOperationException("Email and code are required");

    var user = await _db.Users.FirstOrDefaultAsync(x => x.Email == email);
    if (user == null)
        throw new InvalidOperationException("User not found");

    if (user.EmailConfirmed)
        throw new InvalidOperationException("Email already confirmed");

    if (user.EmailConfirmationCode != code)
        throw new InvalidOperationException("Invalid code");

    user.EmailConfirmed = true;
    user.EmailConfirmationCode = null;

    await _db.SaveChangesAsync();
}
```

Figure 4.33 Email verification function - Mobile application (source: *Developed by the authors*, AndroidStudio)

The other picture (Figure 4.33) represents the back-end verification endpoint logic and this is the final step that closes the email confirmation loop.

1. The signature of this method involves two input parameters that are strings of user email and verification code;
2. After that it provides basic input validation showing a user the both fields are mandatory and throws an exception that returns an error request;
3. Later on it finds user email by looking up on the exact address. If there is no match, then the error is thrown;
4. It is now being checked whether the given email has already been confirmed. This step prevents multiple verification requests;
5. Now the function validates the code and if there is not match - it also throws an error message;
6. Email is marked as confirmed and the code is cleared. The changes are immediately saved to the database;

Password recovery in mobile application

```
public async Task RequestPasswordResetAsync(string email)
{
    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email)
    if (user == null)
        throw new InvalidOperationException("User not found");

    if (string.IsNullOrWhiteSpace(email))
        return;

    if (!new EmailAddressAttribute().IsValid(email))
        return;

    user.PasswordResetCode = GenerateCode();

    await _db.SaveChangesAsync();

    await _emailService.SendAsync(
        email,
        "Password reset",
        $"Your password reset code: {user.PasswordResetCode}"
    );
}
```

Figure 4.34 Password recovery request function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The signature function takes email as a string parameter. Succes is implied if no exceptions are thrown;
2. After that it looks up a user by their email. In this case, if a user does not exist, the exception is thrown;
3. Early exit if email is empty or invalid. The sending here is skipped. It uses built-in .NET EmailAddressAtttribute for basic format validation;
4. Generates and resets the code. It stored in the Users table as a PasswordResetCode column;
5. Sends reset email, using _emailService method with the subject and body;

The next function complements the previous one and finishes password reset cycle. It is called when the user submits their new password along with the reset code they received via email.

```
public async Task ResetPasswordAsync(string email, string code, string newPassword)
{
    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email);
    if (user == null)
        throw new InvalidOperationException("User not found");


    if (user.PasswordResetCode != code)
        throw new InvalidOperationException("Invalid code");

    user.PasswordHash = BCrypt.Net.BCrypt.HashPassword(newPassword);
    user.PasswordResetCode = null;

    await _db.SaveChangesAsync();
}
```

Figure 4.35 Reset password function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The parameters of this function are strings of email, verification code and new password;
2. Initially, it also finds a user by their email and throws an exception if there is an account mismatch;
3. Verifies the reset code by direct string comparison;
4. Hashes and updates user password using BCrypt and generating a new hash;
5. Clears the reset code;
6. Persists changes by saving the new password and cleared reset code;

New order in mobile application

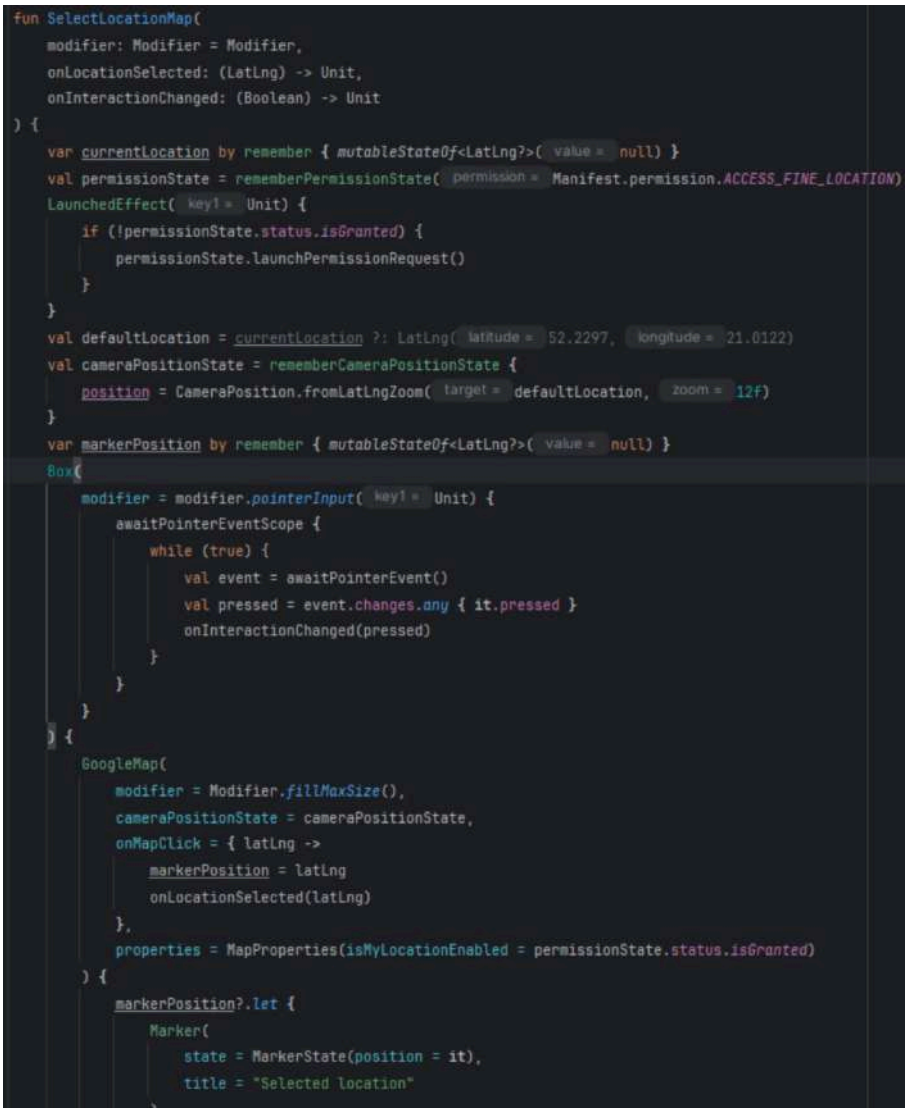


Figure 4.36 Map location selection function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The signature of this function takes three main parameters that are modifier, selected location (a callback invoked when the user has finalized a location selection) and interaction status (a callback that reports if a user is interacting with the map);
2. The function handles permission by requesting. If it already granted, then enables current location layer (a blue dot in the map) and if not granted, then triggers system permission dialogue;
3. According to the coordinates, default camera is set on Warsaw;
4. Map interaction overlay is represented by a Box + pointerInput. A transparent box covers all the map and uses pointerInput in order to detect any pressed events. Whenever at least one pointer is pressed it is called with a true boolean. When all the pointers are lifted - the line is called with a false boolean;
5. The core map of the project is the Google one that is launched on. Its parameters are a map click that instantly moves marker to that position, location enability that shows a blue dot on my location;
6. Marker appears only after the first tap and has a fixed title of Selected location. It is not draggable in this implementation;

The next function is one of the core utilities used during order creation and order matching. It calculated the driving distance between pick-up and drop-off locations from the map (Figure 4.37) Here is the function explained:

Figure 4.37 Distance calculation function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. As parameters the function takes a couple of stings that are points A and B. The string here is the format expected by Google Directions API (latitude and longitude);
2. It runs by implementing IO (Kotlin Coroutines) context and builds Google Maps directions API URL using both of the points on the map;

3. Executes HTTP GET request using OkHttp and reads response body (*lines 9-14 of the Figure 4.17)*;
4. Parses JSON and exits if no routes found;
5. Extracts distance taking first route, first leg, distance and value fields;
6. Coverts metres to kilometres;
7. Handles error by catching the exceptions and printing stack trace for debugging;

Our next function explains the core calculation price logic used in the application when creating a new order or showing an estimated price to the customer. Its purpose is calculating the final price of the delivery based on:

- Distance obtained earlier;
- Selected vehicle type;
- Selected cargo options;

```
fun calculatePrice(
    distanceKm: Double,
    vehicleType: String,
    standard: Boolean,
    valuable: Boolean,
    fragile: Boolean,
    heavy: Boolean
): Double {
    val rate = vehicleRates[vehicleType] ?: throw IllegalArgumentException( s = "Unknown vehicle type")
    var price = distanceKm * rate

    if (standard) price += standardFee
    if (valuable) price += valuableFee
    if (fragile) price += fragileFee
    if (heavy) price += heavyFee

    price = maxOf( a = minPrice,  b = price)
    return kotlin.math.ceil( x = price)
}
```

Figure 4.38 Price calculation function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The signature of this function includes the already mentioned parameters. The potential cargo types available for selection on extra fees are standard, valuable, fragile and heavy. The return value of the signature is a Double that is a final. calculated price, runded up to the nearest whole number;
2. This function gets the basic rate per kilometer by using a snippet with available vehicle prices per kilometer that are sedan, van, large van and cargo. The price is basically a result of multiplying the whole distance on vehicle rate per kilometer;
3. The fixed fees are added conditionally and only if a cargo category was chosen;
4. Lastly, the minimum price is rounded up the nearest integer and applied. This an external constant that assures no order is cheaper then the platform minimum;

This snippet (Figure 4.39) describes a private C# method that implements the core business logic in delivery price calculation.

```
private double CalculatePrice(double distance, string vehicleType, bool standard, bool valuabl
{
    if (!VehicleRates.ContainsKey(vehicleType))
        throw new ArgumentException("Unknown vehicle type");

    double price = distance * VehicleRates[vehicleType];

    if (standard) price += StandardFee;
    if (valuable) price += ValuableFee;
    if (fragile) price += FragileFee;
    if (heavy) price += HeavyFee;

    price =  Math.Max(MinPrice, price);

    return Math.Ceiling(price);
}
```

Figure 4.39 Price calculation function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The signature of this function computes the total order price by applying a distanced-based rate specific to the chosen vehicle type, fixed additional fees for selected cargo options, a minimum price threshold and ceiling rounding;
2. Then validation of vehicle type takes place. If there is no match in vehicle types, an exception is thrown;
3. The base price is then calculated by multiplying the final distance on vehicle type fee per kilometer;
4. Conditionally some fixed charges depending on a chosen cargo type (Standard, Valuable, Fragile or Heavy) may be applied;
5. We apply the minimum price floor. This ensure that short-distance order remain profitable;
6. Lastly, everything is rounded up to an integer;

	OrderId	PointA	PointB
	Filter	Filter	Filter
1	1	Stefana Żeromskiego 6B, 60-544 …	Wiadukt Kolejowy Hetmańska, Hetmańsk…
2	2	al. Jana Pawła II 43, 00-828 …	al. Niepodległości 223, 02-087 …
3	3	Okopowa 2b, 01-063 Warszawa, Poland	Franciszka Klimczaka 1, 02-797 …

Distance	VehicleType	DateTime	
Filter	Filter	Filter	
5.21	Large Van	2026-02-13 18:00:00	
2.986	Large Van	2026-02-19 12:00:00	
14.099	Large Van	2026-02-26 12:00:00	
20.427	Luton Van	2026-02-14 12:00:00	

Standard	Valuable	Fragile	Heavy	*CustomerId*	*DriverId*	Price
Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	1	2	1	82.0
1	1	1	1	2	*NULL*	70.0
1	1	1	1	2	*NULL*	126.0
1	1	1	1	2	*NULL*	178.0

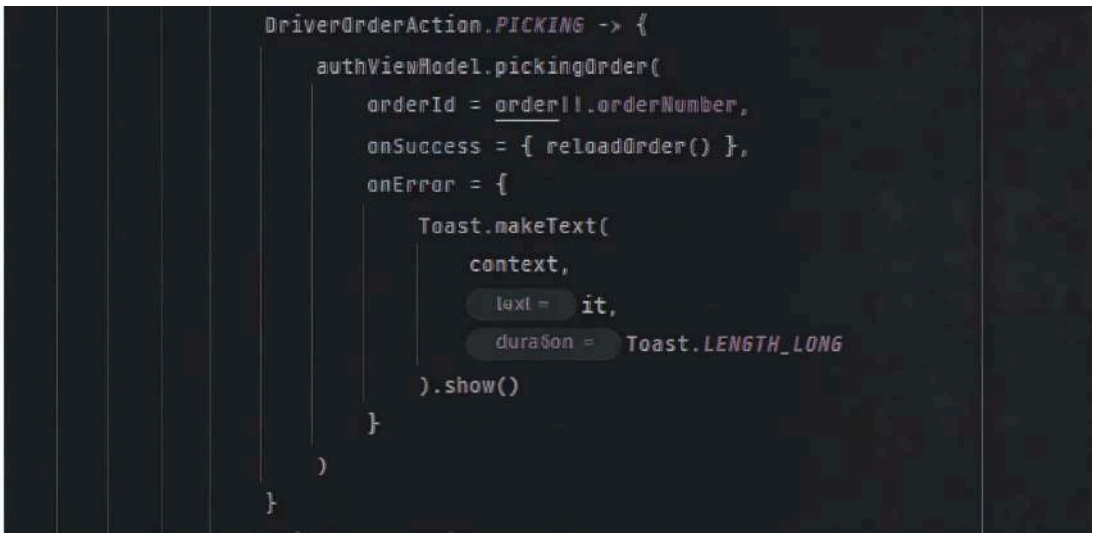
Price	Status	ClientReminderSent	DriverReminderSent	NewOrderNotificationSent
Filter	Filter	Filter	Filter	Filter
82.0	1	0	0	1
70.0	0	0	0	1
126.0	0	0	0	1
178.0	0	0	0	1

Figure 4.40 Created order in the database (source: *Developed by the authors*, SQLite)

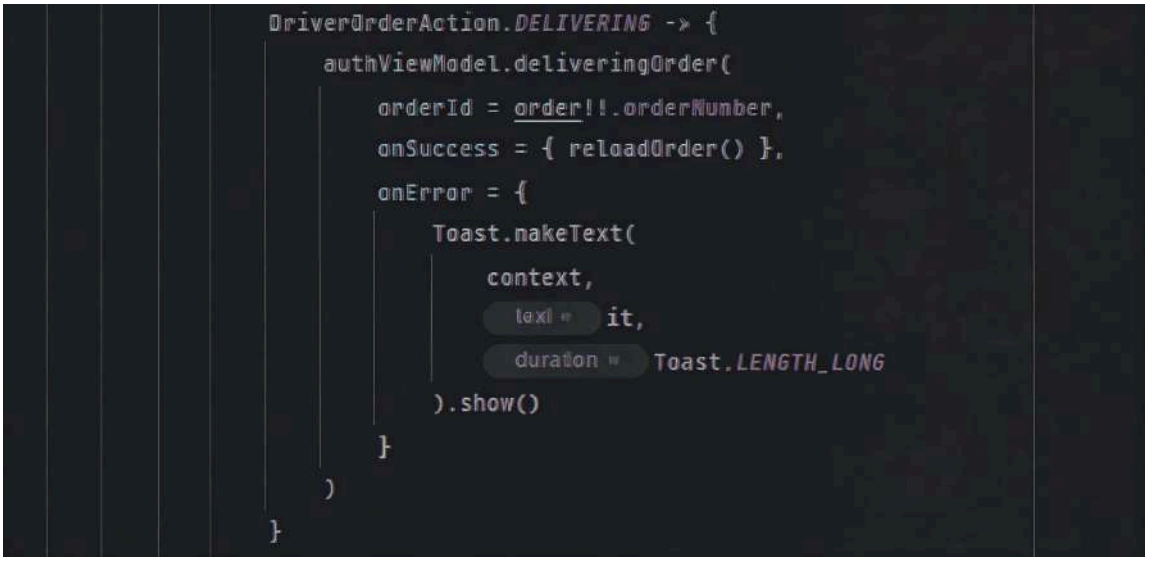
Order status management in mobile application

While releasing delivery process as a whole, a driver is responsible for updating order status through their interface by utilizing the below described functions. The purpose of this is to allow drivers to change the status of an active order.

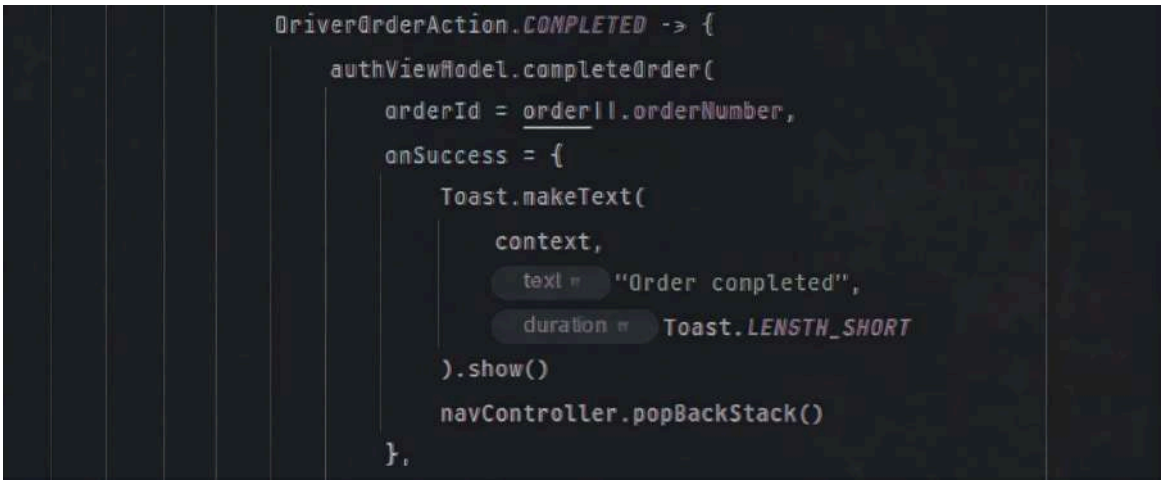
```
driverAction?.let { action ->
    Button(
        onClick = {
            when (action.action) {
                DriverOrderAction.CONING -> {
                    authViewModel.coningOrder(
                        orderId = order!!.orderNumber,
                        onSuccess = { reloadOrder() },
                        onError = {
                            Toast.makeText(
                                context,
                                text = it,
                                duration = Toast.LENGTH_LONG
                            ).show()
                        }
                    )
                }
```



```
DriverOrderAction.PICKING -> {
    authViewModel.pickingOrder(
        orderId = order!!.orderNumber,
        onSuccess = { reloadOrder() },
        onError = {
            Toast.makeText(
                context,
                text = it,
                duration = Toast.LENGTH_LONG
            ).show()
        }
    )
}
```



```
DriverOrderAction.DELIVERING -> {
    authViewModel.deliveringOrder(
        orderId = order!!.orderNumber,
        onSuccess = { reloadOrder() },
        onError = {
            Toast.makeText(
                context,
                text = it,
                duration = Toast.LENGTH_LONG
            ).show()
        }
    )
}
```



```
DriverOrderAction.COMPLETED -> {
    authViewModel.completeOrder(
        orderId = order!!.orderNumber,
        onSuccess = {
            Toast.makeText(
                context,
                text = "Order completed",
                duration = Toast.LENSTH_SHORT
            ).show()
            navController.popBackStack()
        },
```

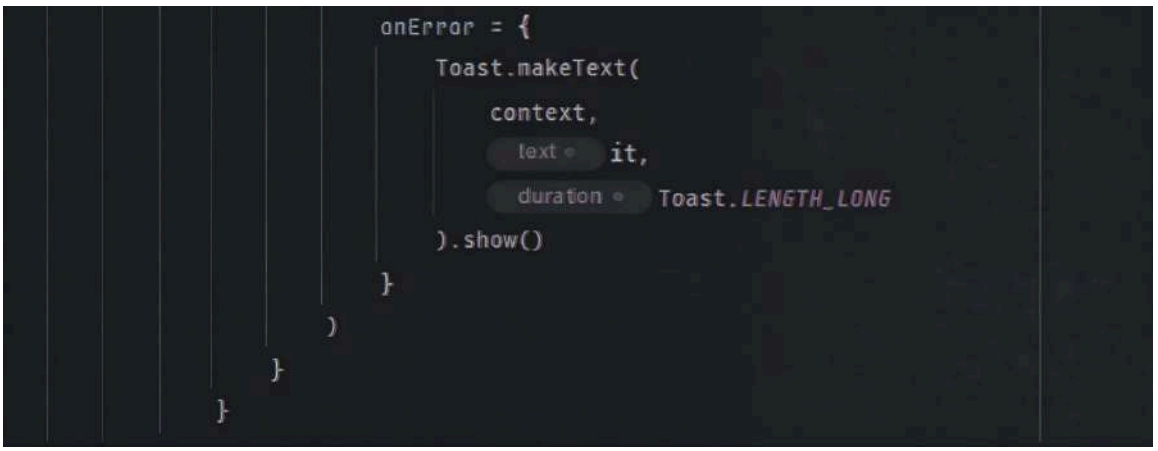
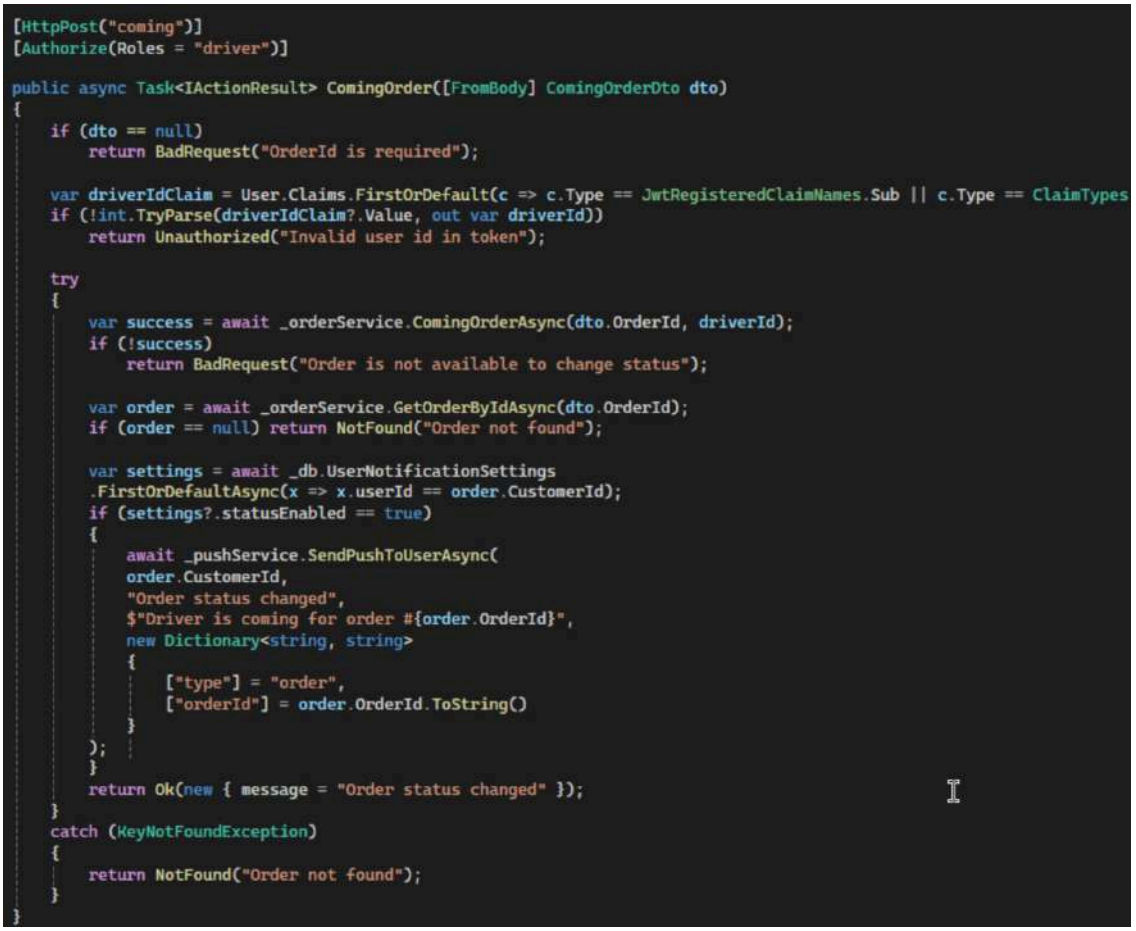


Figure 4.41 Order status change function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. The buttons onClick uses a when expression that branches based on the current order status;
2. Status Coming means a driver heading to the pick-up point. It calls comingOrder function that sends the request to the back-end. Successful request reloads order data and updates UI status to Picking. Error request shows a Toast message with server message;
3. Status Picking signalizes a driver has arrived at the pick-up location. It calls pickingOrder function that, in successful case, changes order status to Delivering, reloads UI and shows delivery route. In error case it shows the same Toast with an error message;
4. Status Delivering shows that a driver is delivering and has arrived at the drop-off. It similarly calls the deliveringOrder function and sends a back-end request for ride completion. In successful case, changes order status to Delivering, reloads UI and shows delivery route. In error case it shows the same Toast with an error message;

Table 4.2 Driver status progression (source: *Developed by the authors*)

Current status	**Method**	**New status**
COMING	comingOrder()	PICKING
PICKING	pickingOrder()	DELIVERING
DELIVERING	deliveringOrder()	COMPLETED



```
[HttpPost("coming")]
[Authorize(Roles = "driver")]

public async Task<IActionResult> ComingOrder([FromBody] ComingOrderDto dto)
{
    if (dto == null)
        return BadRequest("OrderId is required");

    var driverIdClaim = User.Claims.FirstOrDefault(c => c.Type == JwtRegisteredClaimNames.Sub || c.Type == ClaimTypes
    if (!int.TryParse(driverIdClaim?.Value, out var driverId))
        return Unauthorized("Invalid user id in token");

    try
    {
        var success = await _orderService.ComingOrderAsync(dto.OrderId, driverId);
        if (!success)
            return BadRequest("Order is not available to change status");

        var order = await _orderService.GetOrderByIdAsync(dto.OrderId);
        if (order == null) return NotFound("Order not found");

        var settings = await _db.UserNotificationSettings
        .FirstOrDefaultAsync(x => x.userId == order.CustomerId);
        if (settings?.statusEnabled == true)
        {
            await _pushService.SendPushToUserAsync(
            order.CustomerId,
            "Order status changed",
            $"Driver is coming for order #{order.OrderId}",
            new Dictionary<string, string>
            {
                ["type"] = "order",
                ["orderId"] = order.OrderId.ToString()
            }
        );
        }
        return Ok(new { message = "Order status changed" });
    }
    catch (KeyNotFoundException)
    {
        return NotFound("Order not found");
    }
}
```

Figure 4.42 Order status change function - Mobile application (source: *Developed by the authors*, AndroidStudio)

1. Creates POST on HTTP, leaving driver as the only authorised user for this action and forming a request body;
2. Validates DTO (Data Transfer Object) input - it must be present;
3. Extracts driver id from JWT token claims. Firstly, it gets the authenticated driver id and then ensure that only the logged-in driver can collect orders;
4. Called business logic service delegating the actual status change to _orderService and returning the false boolean if orders does not exist, is already in later status, driver is not assigned to this order or it is cancelled/completed;
5. Fetches updated order for notification;
6. Checks if customer wants to push notifications by looking up customer's notification settings;
7. Returns a successful response;
8. Handles exceptions by catching specific errors;

Table 4.2 Order status transitions (source: *Developed by the authors*)

Current status	**Driver action**	**Resulting status**	**Notified**
ACCEPTED	COME	COMING	Customer
COMING	PICKK	PICKING	Customer

PICKING	DELIVER	DELIVERING	Customer
DELIVERING	COMPLETE	COMPLETED	Customer

Orders availability in mobile application

This code snippet is responsible for loading the list of available orders for drivers that they can see and accept. This is typically the main driver screen when they are online and willing to pick an order.

```
LaunchedEffect( key1 = Unit) {
    authViewModel.getAvailableOrders(
        onSuccess = { list ->
            orders = list
            isLoading = false
        },
        onError = { err ->
            error = err
            isLoading = false
            Toast.makeText(context,  text = "Error: $err",  duration = Toast.LENGTH_LONG)
        }
    )
}
```

Figure 4.43 Order availability in mobile application (source: *Developed by the authors*, AndroidStudio)

Triggers automatically on screen entry. It calls the getAvailableOrders view model to fetch available orders - the function inside is a callback method of the view model. It makes a network request to the back-end in order to get the list of new orders. If it is successfully handled - UI recomposes automatically with the list of orders. If not, then the errors are saved to the error state that shows a specific message.

This controller serves as the back-end endpoint for drivers to fetche the list of currently available orders.

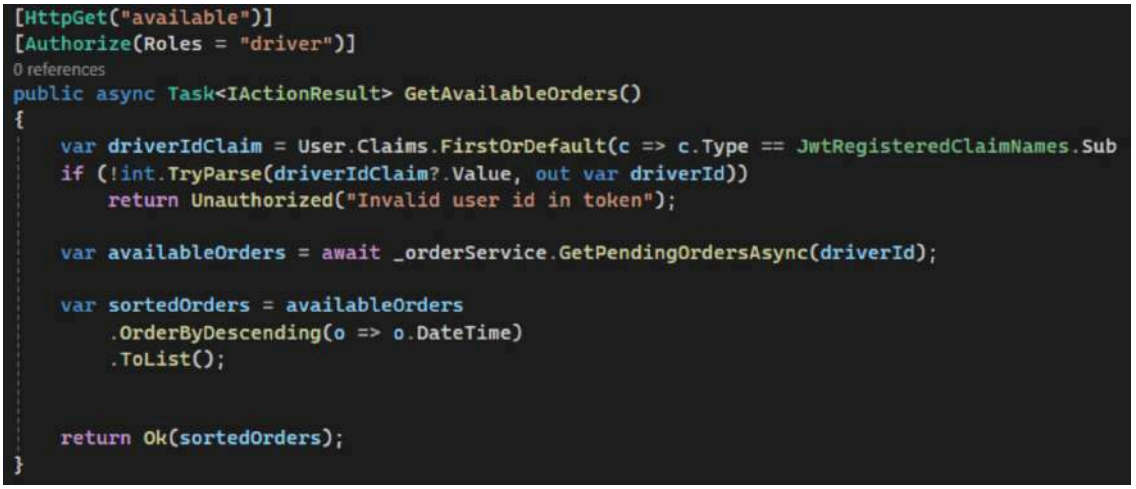


Figure 4.44 Orders availability function of web application (source: *Developed by the authors*, Android studio)

1. Deploys the GET path of HTTP method. Only drivers can call this endpoint;
2. Extract driver id by reading this from standard JWT token claims. Assure the token is valid and contains an integer in id. If parsing fails, it returns an unauthorised message;
3. Fetches pending orders for this driver by delegating filtering logic to _orderService;
4. Sorts the results by the newest by DateTime timestamp;
5. Finally, it returns the list of orders;

Orders acceptance in mobile application

This function allows authenticated drivers to accept pending orders from the list of available orders. It is called when the driver presses the Accept order button on an order card.

```
[HttpPost("accept")]
[Authorize(Roles = "driver")]
0 references
public async Task<IActionResult> AcceptOrder([FromBody] AcceptOrderDto dto)
{
    if (dto == null)
        return BadRequest("OrderId is required");

    var driverIdClaim = User.Claims.FirstOrDefault(c => c.Type == JwtRegisteredClaimNames.Sub || c.Type == ClaimTypes
    if (!int.TryParse(driverIdClaim?.Value, out var driverId))
        return Unauthorized("Invalid user id in token");



    try
    {
        var success = await _orderService.AcceptOrderAsync(dto.OrderId, driverId);
        if (!success)
            return BadRequest("Order is not available to accept");

        var order = await _orderService.GetOrderByIdAsync(dto.OrderId);
        if (order == null) return NotFound("Order not found");

        var settings = await _db.UserNotificationSettings
        .FirstOrDefaultAsync(x => x.userId == order.CustomerId);
        if (settings?.statusEnabled == true)
        {
            await _pushService.SendPushToUserAsync(
            order.CustomerId,
            "Order accepted",
            $"Your order #{order.OrderId} was accepted by driver",
            new Dictionary<string, string>
            {
                ["type"] = "order",
                ["orderId"] = order.OrderId.ToString()
            }
        );
        }
        return Ok(new { message = "Order accepted"});
    }
    catch (KeyNotFoundException)
    {
        return NotFound("Order not found");
    }
}
```

Figure 4.45 Accept order function of web application (source: *Developed by the authors*, Android studio)

1. The POST request of HTTP method that is allowed only for verified drivers. AcceptOrderDto here contains the id of the order a driver wants to accept;
2. Validates DTO input ensuring the request body contains data;
3. Extracts driver id from JWT token and return an error message whether a driver is not authorised;
4. Attempts to accept the order by calling _orderService where validation and update happen. Returns false if an order is not available to accept;
5. Fetches the updated order;
6. Sends push notifications to a customer if enabled;
7. Returns successful response;
8. Catches errors if the order was not found;

Table 4.3 Conditions of not accepting an order (source: *Developed by the authors*, Android studio)

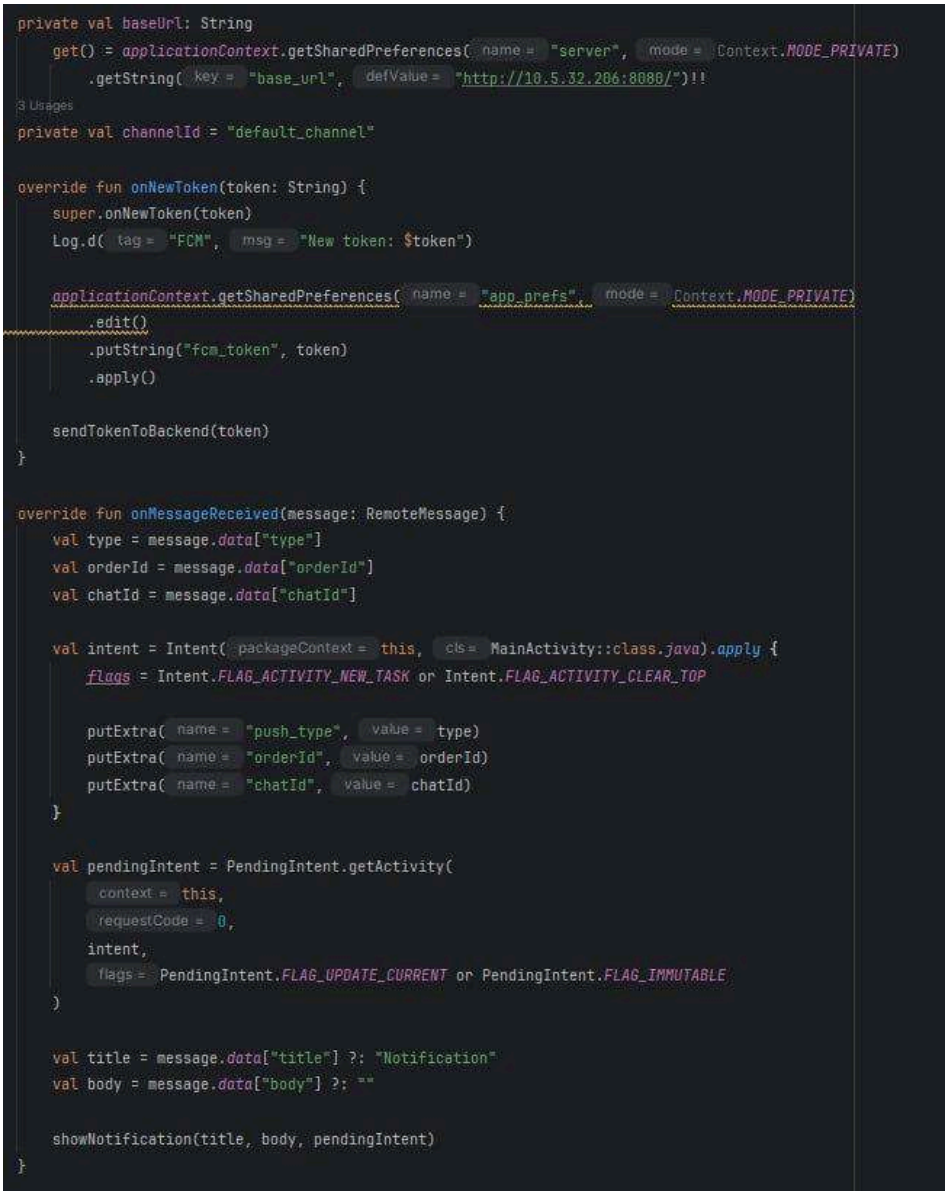
Problem	**Solution**
Does not exist	Reconsider user details
Is already accepted	Browse other orders
Status is not pending	Refresh the page for new orders
Driver is not eligible	Change a city or vehicle

Notifications in mobile application

This aspect of the application is responsible for keeping real-time awareness without constant app checking and enables offline communications, providing all up-to-date information.

This snippet of code is a part of Firebase Cloud Messaging service class that handles two main lifecycles:

- When a new FCM registration token is generated or refrefreshed;
- When a push notification message is received from the server;

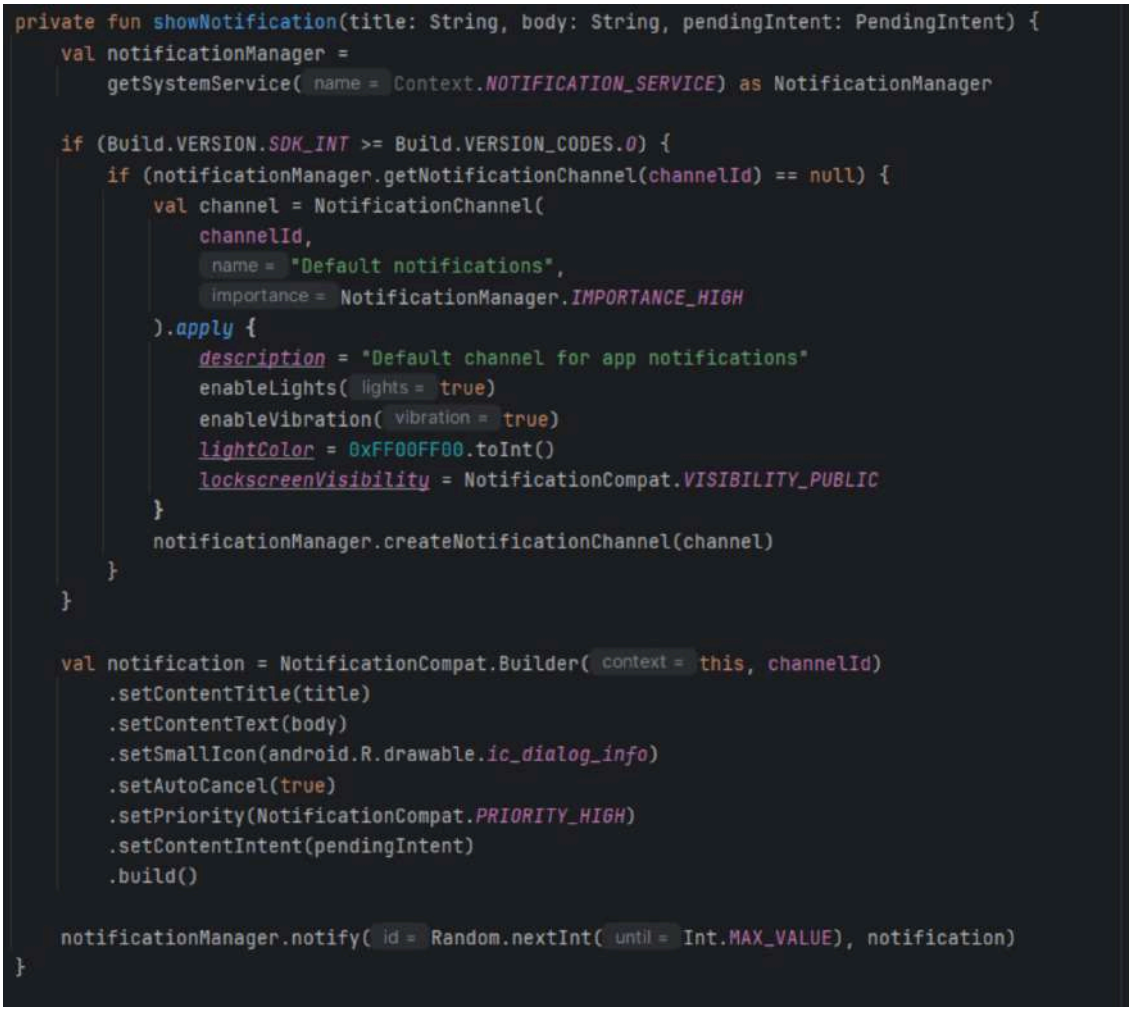


```
private val baseUrl: String
    get() = applicationContext.getSharedPreferences( name = "server", mode = Context.MODE_PRIVATE)
        .getString( key = "base_url", defValue = "http://10.5.32.206:8080/")!!
3 Usages
private val channelId = "default_channel"

override fun onNewToken(token: String) {
    super.onNewToken(token)
    Log.d( tag = "FCM", msg = "New token: $token")

    applicationContext.getSharedPreferences( name = "app_prefs", mode = Context.MODE_PRIVATE)
        .edit()
        .putString("fcm_token", token)
        .apply()

    sendTokenToBackend(token)
}

override fun onMessageReceived(message: RemoteMessage) {
    val type = message.data["type"]
    val orderId = message.data["orderId"]
    val chatId = message.data["chatId"]

    val intent = Intent( packageContext = this, cls = MainActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP

        putExtra( name = "push_type", value = type)
        putExtra( name = "orderId", value = orderId)
        putExtra( name = "chatId", value = chatId)
    }

    val pendingIntent = PendingIntent.getActivity(
        context = this,
        requestCode = 0,
        intent,
        flags = PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
    )

    val title = message.data["title"] ?: "Notification"
    val body = message.data["body"] ?: ""

    showNotification(title, body, pendingIntent)
}
```

Figure 4.46 New message token function in mobile application (source: *Developed by the authors*, Android studio)

1. onNewToken function is called automatically by Firebase whenever the app is first installed, the FCM token is refreshed by Google and when the token is changed for any reason. It logs the new token for debugging, saves it persistently using a private mode for secure storage;
2. onMessageReceived function is called every time the app receives a data message from FCM. It extracts custom data payload - all of them come from the data dictionary sent by the server. After that it creates a deep-link intent with targets, flags and extras. It also creates an immutable PendingIntent that is used as a tap action for the notification. Finally, it shows system notification;

The following part of the back-end serves a private helper function for building and displaying system notifications when a push message is received via FCM. It creates a notification channel, builds a high-priority notification with title, body, icon, sound and a tap action. Then posts to the system.



```
private fun showNotification(title: String, body: String, pendingIntent: PendingIntent) {
    val notificationManager =
        getSystemService( name = Context.NOTIFICATION_SERVICE) as NotificationManager

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        if (notificationManager.getNotificationChannel(channelId) == null) {
            val channel = NotificationChannel(
                channelId,
                name = "Default notifications",
                importance = NotificationManager.IMPORTANCE_HIGH
            ).apply {
                description = "Default channel for app notifications"
                enableLights( lights = true)
                enableVibration( vibration = true)
                lightColor = 0xFF00FF00.toInt()
                lockscreenVisibility = NotificationCompat.VISIBILITY_PUBLIC
            }
            notificationManager.createNotificationChannel(channel)
        }
    }

    val notification = NotificationCompat.Builder( context = this, channelId)
        .setContentTitle(title)
        .setContentText(body)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setAutoCancel(true)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .build()

    notificationManager.notify( id = Random.nextInt( until = Int.MAX_VALUE), notification)
}
```

Figure 4.47 Show notification function in mobile application (source: *Developed by the authors*, Android studio)

1. The signature parameters such as title, body and pendingIntent (for the tap action);
2. Creates a notification channel if Android is 8+, names it as default notifications, places a high importance, colors it light green and places a small icon. Created only once;
3. The notification itself is built by utilising NotificationCompat.Builder for backward compatibility. Nextly, the auto cancel by tap is set, the head-up banner is shown;
4. Posts notification and generates a random id;

Table 4.4 Main types of push notifications (source: *Developed by the authors*, Android studio)

Event	**Body example**	**Deep-link extras**
New order	New order #12345 is available	type=order, orderId=12345
Order accepted	Your order #12345 was accepted by the driver	type=order, orderId=12345

Reminder	Your order #12345 will start in 15 minutes	type=order, orderId=12345
Order status change	Driver is coming for order #12345	type=order, orderId=12345

The other piece of code contains several closely related pieces that together implement the client-side mechanism for registering the device's FCM token with the back-end server every time it is changed or refreshed.

```
1 Usage
private fun sendTokenToBackend(token: String) {
    CoroutineScope( context = Dispatchers.IO).launch {
        try {
            val jwtToken = TokenStorage.getAccessToken(applicationContext) ?: return@laun
            val json = Json.encodeToString( value = FcmTokenDto(token))
            val body = json.toRequestBody( contentType = "application/json".toMediaType())

            val request = Request.Builder()
                .url( url = "${baseUrl}api/push/token")
                .addHeader( name = "Authorization", value = "Bearer $jwtToken")
                .post(body)
                .build()

            OkHttpClient().newCall(request).execute().close()
            Log.d( tag = "FCM", msg = "Token sent to backend")
        } catch (e: Exception) {
            Log.e( tag = "FCM", msg = "Failed to send token", tr = e)
        }
    }
}

2 Usages
@Serializable
data class FcmTokenDto(val token: String)

1 Usage
object TokenStorage {
    1 Usage
    fun getAccessToken(context: Context): String? {
        val prefs = context.getSharedPreferences( name = "auth", mode = Context.MODE_PRIVATE
        return prefs.getString( key = "access_token", defValue = null)
    }
}
```

Figure 4.48 Show notification function in mobile application (source: *Developed by the authors*, Android studio)

The main function sendTokenToBackend() sends freshly received/updates FCM registration token to the back-end, so that server knows which device belongs to the currently logged-in user. (Figure 4.x)

1. Runs in IO dispatcher and uses JWT token for authentication that ensures only logged-in users can register their device token. Describes the endpoint and JSON body, logs success or failure;
2. A single token string is used for DTO;
3. The token source reads JWT access token from SharedPreferences;

Overall flow of FCM registration token

- Firebase refreshes FCM token: FirebaseMessagingService.onNewToken(token);
- onNewToken saves the token to SharedPreferences and then calls sendTokenToBackend(token);
- SendTokenToBackend gets the current JWT and skips if there is no JWT. Builds a POST request with Bearer token, sends a token object to api/push/token, where back-end saves it in UserTokens table;
- Back-end can now send pushes to this device via SendPushToUserAsync(userId);

	Id	UserId	FcmToken
	Fil...	Filter	Filter
1	1	1	dcziWPidTmG_pthDflAKmB:APA91bEoPqlVW…
2	2	2	dcziWPidTmG_pthDflAKmB:APA91bEoPqlVW…
3	3	22	cgFANHWw_5aFIA1G35qsqh:APA91bGeIKfcX…
4	4	23	cgFANHWw_5aFIA1G35qsqh:APA91bGeIKfcX…
5	5	24	cgFANHWw_5aFIA1G35qsqh:APA91bGeIKfcX…

Figure 4.49 User tokens in the database (source: *Developed by the authors*, SQLite)

	Id	Token	Expires	IsRevoked	
	Filter	Filter	Filter	Filter	F
1	1	I9QSI8kJ7zmhKIRo1wsIc3iAo6ckUdTjNgr3…	2026-02-18 02:05:20.0331154	0	2
2	2	zlePLJMGYMdi+Lz6A4AeiKu6vvi1MJqAathS…	2026-02-18 02:05:25.205977	1	2
3	3	nVed2YFlNMxquO2Ki/J3srXItzqT/…	2026-02-18 02:06:30.1128123	0	2
4	4	VvoWG934U9ZtrAHJrVslNQ5pleew0CmT12/…	2026-02-18 02:11:18.1735186	0	2
5	5	URQwbRzw7D20zjLuVDeWmchcYxH4oPzd6FdX…	2026-02-18 03:20:16.688691	0	2

	CreatedAt	Ip	UserAgent	*UserId*
	Filter	Filter	Filter	Filter
0	2026-02-11 02:05:20.0330581	127.0.0.1	okhttp/5.2.1	1
1	2026-02-11 02:05:25.2059768	127.0.0.1	okhttp/5.2.1	1
0	2026-02-11 02:06:30.1128121	127.0.0.1	okhttp/5.2.1	2
0	2026-02-11 02:11:18.1735176	127.0.0.1	okhttp/5.2.1	2
0	2026-02-11 03:20:16.6886902	127.0.0.1	okhttp/5.2.1	2

Figure 4.50 Continuation of user tokens in the database (source: *Developed by the authors*, SQLite)

This part of the code is in charge for scanning for newly created orders every 60 seconds that have not yet had notifications sent to the drivers and then push them. (Figure 4.51)

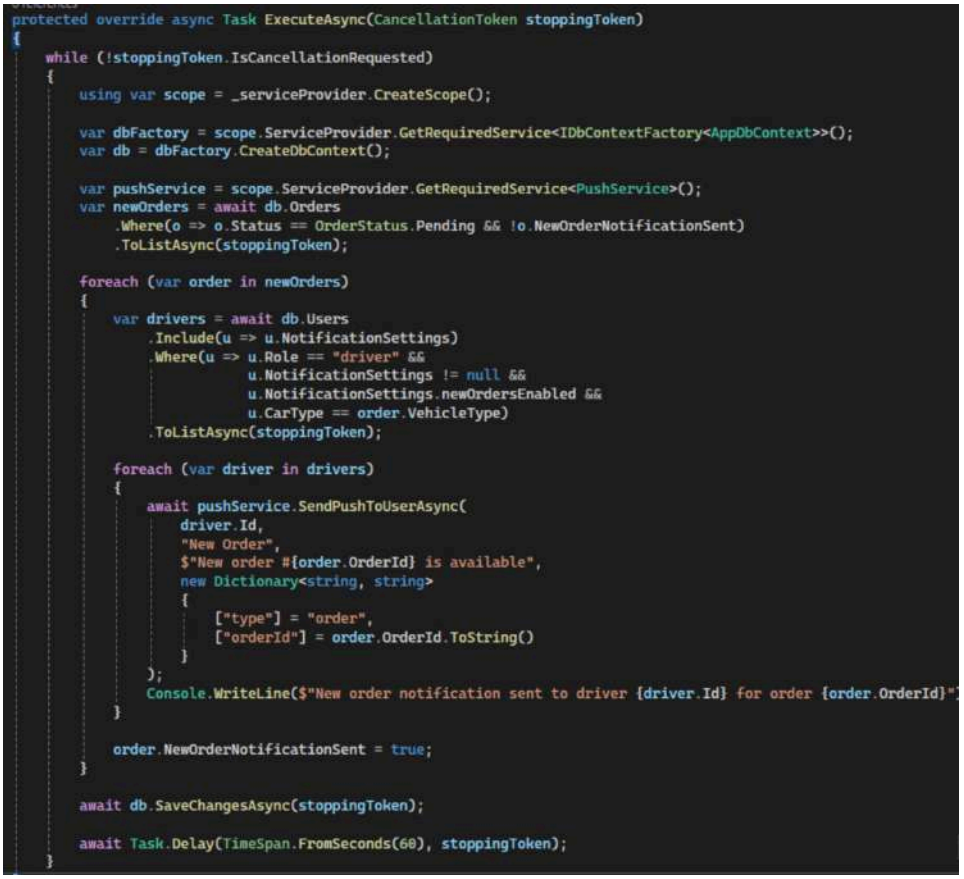
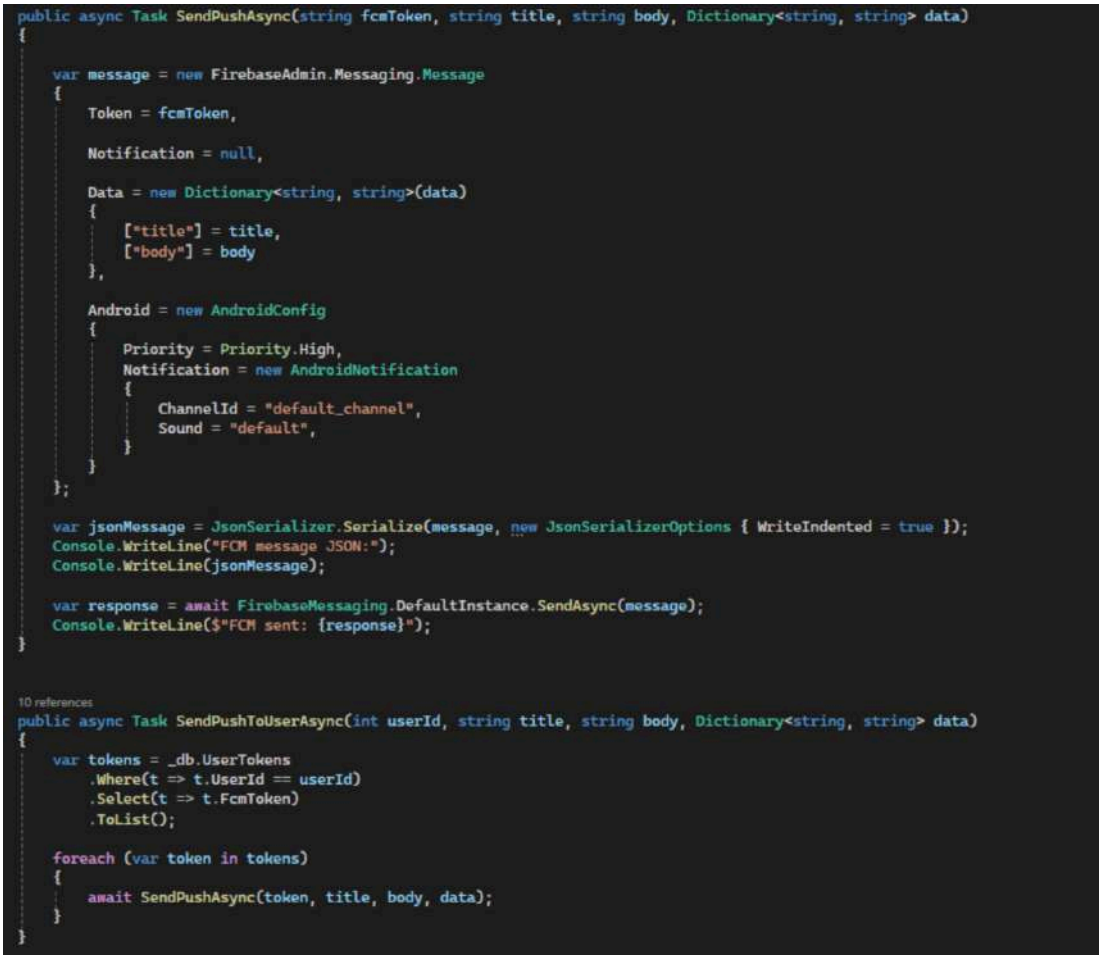


Figure 4.51 Execute notifications function (source: *Developed by the authors*, Android studio)

1. Creates fresh dependency scope by implementing a new DbContext instance per cycle and refreshing the PushService instance;
2. Finds all pending orders without notifications sent;
3. Finds eligible drivers for each pending order by matching vehicle type;
4. Sends push notifications containing title and body to each eligible driver;
5. Logs the action in the console;
6. Mark the order as notification sent;
7. Waits 60 seconds before the next cycle;

This function periodically checks active orders that are about to start soon and sends reminder push notifications to both the customer and driver a configurable number of minutes before the scheduled start time. (Figure 4.52)



```
public async Task SendPushAsync(string fcmToken, string title, string body, Dictionary<string, string> data)
{

    var message = new FirebaseAdmin.Messaging.Message
    {
        Token = fcmToken,

        Notification = null,

        Data = new Dictionary<string, string>(data)
        {
            ["title"] = title,
            ["body"] = body
        },

        Android = new AndroidConfig
        {
            Priority = Priority.High,
            Notification = new AndroidNotification
            {
                ChannelId = "default_channel",
                Sound = "default",
            }
        }
    };

    var jsonMessage = JsonSerializer.Serialize(message, new JsonSerializerOptions { WriteIndented = true });
    Console.WriteLine("FCM message JSON:");
    Console.WriteLine(jsonMessage);

    var response = await FirebaseMessaging.DefaultInstance.SendAsync(message);
    Console.WriteLine($"FCM sent: {response}");
}


10 references
public async Task SendPushToUserAsync(int userId, string title, string body, Dictionary<string, string> data)
{
    var tokens = _db.UserTokens
        .Where(t => t.UserId == userId)
        .Select(t => t.FcmToken)
        .ToList();

    foreach (var token in tokens)
    {
        await SendPushAsync(token, title, body, data);
    }
}
```

Figure 4.52 Reminding notifications function (source: *Developed by the authors*, Android studio)

1. Creates fresh dependencies;
2. Gets current UTC time;
3. Loads all relevant orders with includes;
4. Customer reminder logic is the following: a customer exits, has upcomingEnabled to true in their notifications along with start order time. The system sends once, flips flag and never sends again;
5. Driver reminder logic is pretty identical but uses driver's NotificationSettings and DriverRemindSent flag;

The purpose of this method is pushing a single notification to one specific device identified by its registration token.

	userId	chatEnabled	newOrdersEnabled	statusEnabled	upcomingEnabled	upcomingMinutes
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	1	1	1	15
2	2	1	1	1	1	15
3	21	1	1	1	1	15
4	27	1	1	1	1	15
5	30	1	1	1	1	15
6	32	1	1	1	1	15
7	34	1	1	1	1	15

Figure 4.53 User notification settings in the database (source: *Developed by the authors*, SQLite)

Table 4.5 Send notifications function parameters (source: *Developed by the authors*)

Parameter	**Type**	**Description**
fcmToken	string	FCM registration token
title	string	Notification title
body	string	Notification text
data	Dictionary<string, string>	Custom key-value payload

Chat in mobile application

```
LaunchedEffect( key1 = Unit) {
    authViewModel.getChats(
        onSuccess = { list ->
            chats = list
            isLoading = false
        },
        onError = { err ->
            error = err
            isLoading = false
            Toast.makeText(context, text = "Error: $err", duration = Toast.LENGTH_LONG).
        }
    )
}
```

Figure 4.53 Active chats trigger function in mobile application (source: *Developed by the authors*, AndroidStudio)

The LaunchedEffect() is triggered automatically on screen entry. key1 = Unit executes exactly once when the composable enters composition. It calls the view model method to fetch. That makes a network request to retrieve all active conversations. It also handles success operations by showing the chat list and hiding the loading bar. If an error occurs, then it is saved to the state, loading is also stopped and the appropriate message is shown.

The second snippet consists of two separate LaunchedEffect(). Both are placed inside a single chat screen and together they handle the initial loading and setup of a single active chat tied to a particular order. (Figure 4.54)

Figure 4.54 Separated launch effects in mobile application (source: *Developed by the authors*, Android studio)

- The first LaunchedEffect() immediately asks the view model to load or create the chat associated with this order. Its key point is key1=orderId meaning that the effect re-runs only if orderId changes. If the order is the same, it runs only once. authViewModel.loadChat(orderId, onError), is the main call that fetches existing chats. If the backend returns an error, the Toast is shown;
- The second LaunchedEffect() loads chat history and enters chat. Once the currentChatId becomes available, it performs only two actions: loads the full message history for the chat and marks the chat as entered. Its key point is key1=currentChatId that runs every time currentChadId changes. authViewModel.loadChatHistory(chatId) fetches the full list of messages;

Table 4.6 Responsibilities of each launched effect (source: *Developed by the authors*)

LaunchedEffect key	When it runs	Main actions	Side effects
orderId	Screen opens.orderId changes	Load or create a chat for this order	Get currentChatId from back-end
currentOrderId	currentChatId appears	Load messages and enter mark	Message list and reset unread

The next piece of code represents a pair of compose effects that together handle the real-time connection lifecycle for the chat feature using a WebSocket. They are placed inside the single chat screen. (Figure 4.55)

```
LaunchedEffect( key1 = Unit) {
    authViewModel.connectToChatHub()
}

DisposableEffect( key1 = Unit) {
    onDispose {
        authViewModel.leaveChat()
    }
}
```

Figure 4.55 Separated compose effects in mobile application (source: *Developed by the authors*, Android studio)

- The purpose of the first effect is to establish a real-time connection as soon as the chat screen is displayed. It runs only once when the screen first appears and opens a WebSocket connection. It subscribes for the current chat events, sends authentication token to the hub, registers current user as online in the chat and sets up callbacks in ViewModel for incoming messages;
- The second effect is in charge of clean disconnection from the chat hub when the chat screen is not displayed anymore. DisposableEffect() provides a cleanup mechanism, while onDispose runs automatically when the user leaves the screen. Eventually, it unsubscribes from chat-specific events, closes WebSocket, sends a stop message to the server and resets view model state;

DisposableEffect() was used here in order to assure a proper cleanup, prevent memory leaks and stop unnecessary server loads.

Description of Send button logic is considered the other important element of each chat. It is responsible for sending a new text message from the user.

Figure 4.56 Send button logic in mobile application (source: *Developed by the authors*, Android studio)

1. Checks if a message is not empty;
2. Launches specific background;
3. Calls the view model to send the message. Sends the message via WebSocket;
4. Handles possible errors. If back-end fails, it shows a short Toast warning;
5. Clears the input field;

Here is a server-side method that runs every time a new chat successfully saves to the database. This method is responsible for two main things that are real-time broadcasting and offline push notifications.

```
var participants = new List<int> { chat.User1Id };
if (chat.User2Id.HasValue) participants.Add(chat.User2Id.Value);

foreach (var participantId in participants)
{
    await hub.Clients.Group(participantId.ToString())
    .SendAsync("ReceiveMessage",
        message.Id,           // int
        message.ChatId,       // int
        message.SenderId,     // int
        message.Text,         // string
        message.SentAt.ToString("yyyy-MM-ddTHH:mm:ssZ")  // string
    );
    if (participantId != senderId)
    {
        var isUserInChat = _presence.IsUserInChat(participantId, chat.Id);

        if (!isUserInChat)
        {
            var tokens = await _db.UserTokens
                .Where(t => t.UserId == participantId)
                .Select(t => t.FcmToken)
                .ToListAsync();

            foreach (var token in tokens)
            {
                var settings = await _db.UserNotificationSettings
                .FirstOrDefaultAsync(x => x.userId == participantId);
                if (settings?.chatEnabled == true)
                    await _pushService.SendPushAsync(
                    token,
                    "New message",
                    message.Text,
                    new Dictionary<string, string>
                    {
                        ["type"] = "chat",
                        ["orderId"] = chat.OrderId.ToString(),
                        ["chatId"] = chat.Id.ToString()
                    }

                );
            }
        }
```

Figure 4.56 Push notifications function in web application (source: *Developed by the authors*, Android studio)

1. Collects chat participants IDs;
2. Broadcasts to each of the sides via SignalR. Each user is subscribed to a SignalR group named after their userId. Online users get instant update;
3. Skips pushes for the sender;
4. Checks if the recipient is actively viewing the chat;
5. Fetches all the FCM token for the recipient;
6. Loads notification settings and sends pushes. Checks if user notifications are active and creates a new message with a title and body;

	Id	User1Id	User2Id	OrderId
	Fil...	Filter	Filter	Filter
1	1	2	1	1
2	2	2	NULL	2
3	3	2	NULL	3
4	4	2	NULL	4
5	5	2	29	5
6	6	2	NULL	6
7	7	2	1	7
8	8	2	NULL	8

Figure 4.57 Chats according to the user IDs in the database (source: *Developed by the authors*, SQLite)

	Id	*ChatId*	SenderId	Text	SentAt
	Fil...	Filter	Filter	Filter	Filter
1	1	14	21	21321312312	2026-02-13 12:49:32.117553
2	2	14	21	32131232	2026-02-13 12:49:34.8381887
3	3	14	21	213123	2026-02-13 12:49:39.2748359
4	4	14	21	312313	2026-02-13 12:53:20.895066

Figure 4.58 Chat messages in the database (source: *Developed by the authors*, SQLite)

4.6 Database modelling

In the final system design stage, the eventual version of database was visually represented on the following diagram. It illustrated primary data types and keys along with representation of relationship among main data entities.

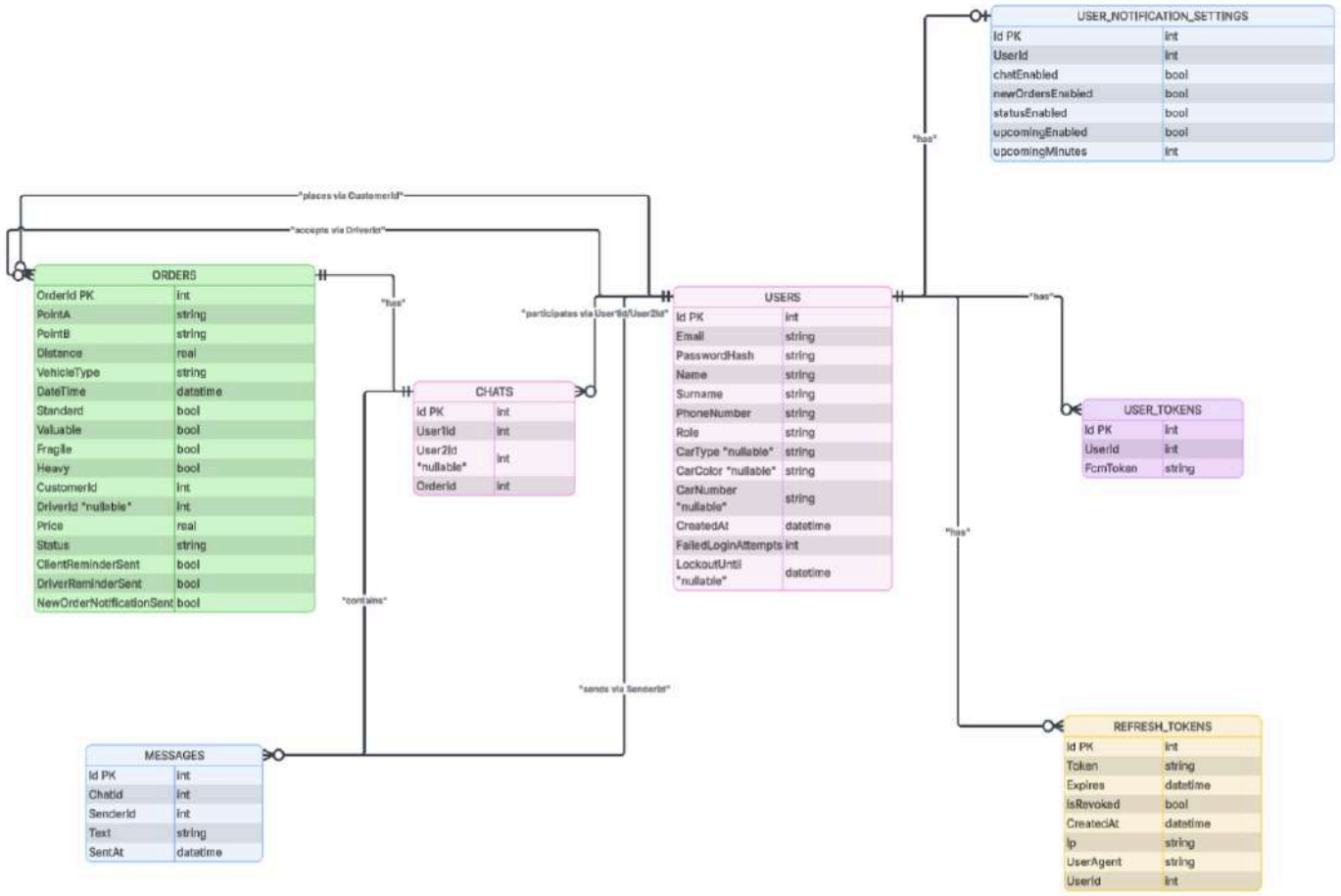


Figure 4.59 Database diagram (source: *Developed by the authors*, Lucidchart)

4.7 Database architecture

As described in the below figure, the database is presented in the form of tables divided by the main functionalities of the platform. Each group of tables is provided with automatically generated schemas.

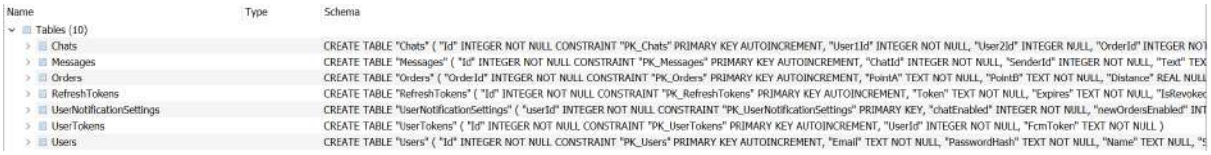


Figure 4.60 Database structure (source: *Developed by the authors*, SQLite)

Chats folder stores conversation links to specific order and its two participants. Created automatically when an order is accepted.

Messages folder stores the whole message history for each chat. Used for loading past messages and real-time display.

Orders folder tracks full order lifecycle from creation, acceptance to status updates and completion.

RefreshTokens folder allows clients to get new access tokens without re-login each time that enables more secure sessions.

UserTokens folder enables sending push notifications to specific devices even when the application is closed.

UserNotificationSettings folder controls whether a user receives FCM pushes for chat messages, new orders, status changes or upcoming reminders.

Users folder is in charge of authentication, user profiles and authorisation due to role-based access.

4.8 Server architecture

The back-end of ‘Gazelka’ application is implemented using a RESTful API with endpoints organized into logical groups for authentication, chat management, notification settings, order lifecycle and push token handling. To facilitate development, testing and documentation, Swagger [31] was integrated. It provided an interactive UI for exploring endpoints, sending test requests and generating client code. Swagger automatically scans the controlers to produce comprehensive API specification, ensuring consistency and ease if maintenance. All endpoints are prefixed with /api/ and protected routes require JWT Bearer authentication for security.

The server is divided into 4 main categories, each serving a specific purpose and below they will be described one by one.

Auth controller table

This group contains endpoints for user authentication, registration, account management, email verification and password recovery. All the routes are prefixed with /api/Auth. Most endpoints are protected (require a JWT Bearer token), except registration, login and password reset.

Table 4.7 Authentication server requests (source: *Developed by the authors*, Swagger UI)

Method	**Path**	**Request body**
POST	/api/Auth/register	email, password, name. surname, role, carType, carColor, carNumber, phoneNumber, cityName
POST	/api/Auth/login	email, password
POST	/api/Auth/refresh	refreshToken
POST	/api/Auth/logout	refreshToken
GET	/api/Auth/userInfo	No parameters
POST	/api/Auth/updateProfile	email, name, surname, phoneNumber, carType, carColor, carNumber, cityName
DELETE	/api/Auth/deleteAccount	No parameters
POST	/api/Auth/verifyEmail	email, code
POST	/api/Auth/resendEmailCode	email
POST	/api/Auth/requestPasswordReset	email
POST	/api/Auth/resetPassword	email, code, newPassword
GET	/api/Auth/userInfo/{userId}	userId

Chat controller table

This table manages order-specific conversations between customers and drivers. It includes listing chats, loading message history, sending messages that triggers real-time SignalR broadcast and retrieving chats by orderId. All endpoints require authentication.

Table 4.8 Chat server requests (source: *Developed by the authors*, Swagger UI)

Method	**Path**	**Request body**
GET	/api/chat/list	No parameters
GET	/api/chat/history/{chatId}	chatId
POST	/api/chat/send	orderId, text
GET	/api/chat/show/{orderId}	orderId

Notifications controller table

Such a small group handles user preferences for push notifications (chat messages, new orders, status updates and reminders). It allows retrieving and updating settings via GET and POST requests. Both endpoints are identically protected, requiring JWT authentication.

Table 4.9 Notifications server requests (source: *Developed by the authors*, Swagger UI)

Method	**Path**	**Request body**
GET	/api/notifications/settings	No parameters
POST	/api/notifications/settings	chatEnabled, newOrdersEnabled, statusEnabled, upcomingEnabled, upcomingMinutes

Orders controller table (includes Push)

This is the largest and most central group of the server that covers the full order lifecycle and its details retrieval. The push endpoint registers FCM tokens for notifications. All routes require authentication.

Table 4.10 Orders server requests (source: *Developed by the authors*, Swagger UI)

Method	**Path**	**Request body**
POST	/api/orders/create	pointA, pointB, vehicleType, dateTime, standard, valuable, fragile, heavy
GET	/api/orders/customerScheduled	pointA, pointB, vehicleType, dateTime, standard, valuable, fragile, heavy
GET	/api/orders/customerHistory	No parameters
GET	/api/orders/driverScheduled	No parameters
GET	/api/orders/driverHistory	No parameters
GET	/api/orders/available	No parameters
POST	/api/orders/accept	orderId
POST	/api/orders/edit	orderId, pointA, pointB, vehicleTYpe, dateTime, standard, valuable, fragile, heavy
POST	/api/orders/cancel	orderId
GET	/api/orders/{orderId}	orderId
POST	/api/orders/coming	orderId
POST	/api/orders/picking	orderId
POST	/api/orders/delivering	orderId
POST	/api/orders/completed	orderId
POST	/api/push/token	token

Data Transfer Object schemas

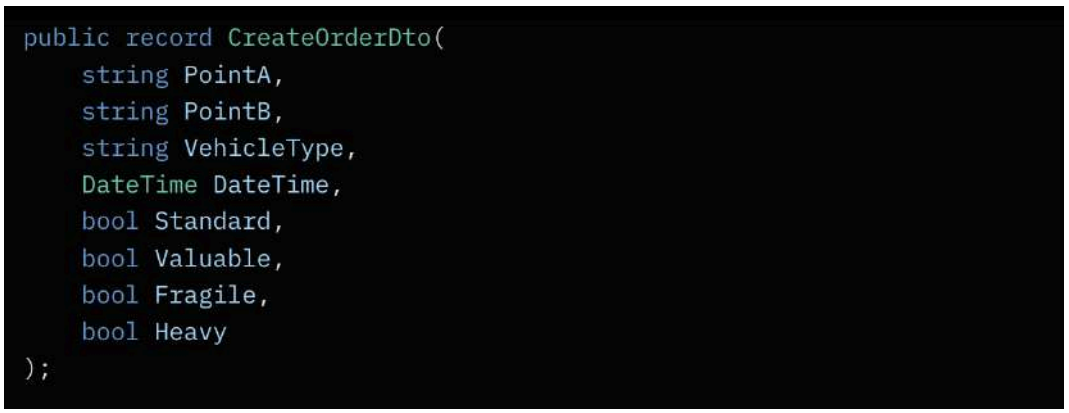
These schemas are used as lightweight, purpose-specific classes to transfer data between the client and the server. DTOs act as the explicit contract defining exactly which fields are sent or received in each API request ans response. It prevents exposure of internal domain models and improves security

Table 4.11 DTO schemas (source: *Developed by the authors*, Swagger UI)

Schema	**Object parameters**
AcceptOrderDTO	orderId
CancelOrderDTO	orderId
ComingOrderDTO	orderId
ComplitedOrderDTO	orderId
CreateOrderDTO	pointA, pointB, vehicleType, dateTime, standard, valuable, fragile, heavy
DeliveringOrderDTO	orderId
EditOrderDTO	orderId, pointA, pointB, vehicleType, dateTime, standard, valuable, fragile, heavy
FcmTokenDTO	token
LoginDTO	email, password
NotificationSettingsDTO	chatEnabled, newOrdersEnabled, statusEnabled, upcomingEnabled, upcomingMinutes
PickingOrderDTO	orderId
RefreshDTO	refreshToken
RegisterDTO	email, password, name, surname, role, carType, carColor, carNumber, phoneNumber, cityName
RequestPasswordResetDTO	email
ResendEmailCodeDTO	email
ResetPasssordDTO	email, code, newPassword
SendMessageDTO	orderId, text
updateProfileDTO	email, name, surname, phoneNumber, carType, carColor, carNumber, cityName
verifyEmailDTO	email

Data Transfer schemas implementation

Examples of Dtos in usage will be held on CreateOrderDTO and PickingOrderDto schemas that represent basic C# record types. They helped to define the exact shape of data sent to the API endpoints.



```
public record CreateOrderDto(
    string PointA,
    string PointB,
    string VehicleType,
    DateTime DateTime,
    bool Standard,
    bool Valuable,
    bool Fragile,
    bool Heavy
);
```

Figure 4.61 Create order DTO schema implementation (source: *Developed by the authors*, VisualStudioCode)

This DTO (Figure 4.61) is the input model for the POST /api/orders/create endpoint. It represents the data a customer submits when creating a new order

```
public record PickingOrderDto(int OrderId);
```

Figure 4.62 Picking order DTO schema implementation (source: *Developed by the authors*, VisualStudioCode)

This other DTO (Figure 4.62) represents the input of the POST /api/orders/picking endpoint. It tells the backend that the drivers has started the picking process phase of an order. orderId here is the identifier of the order being updated.

The DTO implementation looks so basic because its endpoint does not need more data and the driver comes from the authenticated user (JWT) and status change is just a transition.

Service registration and authentication

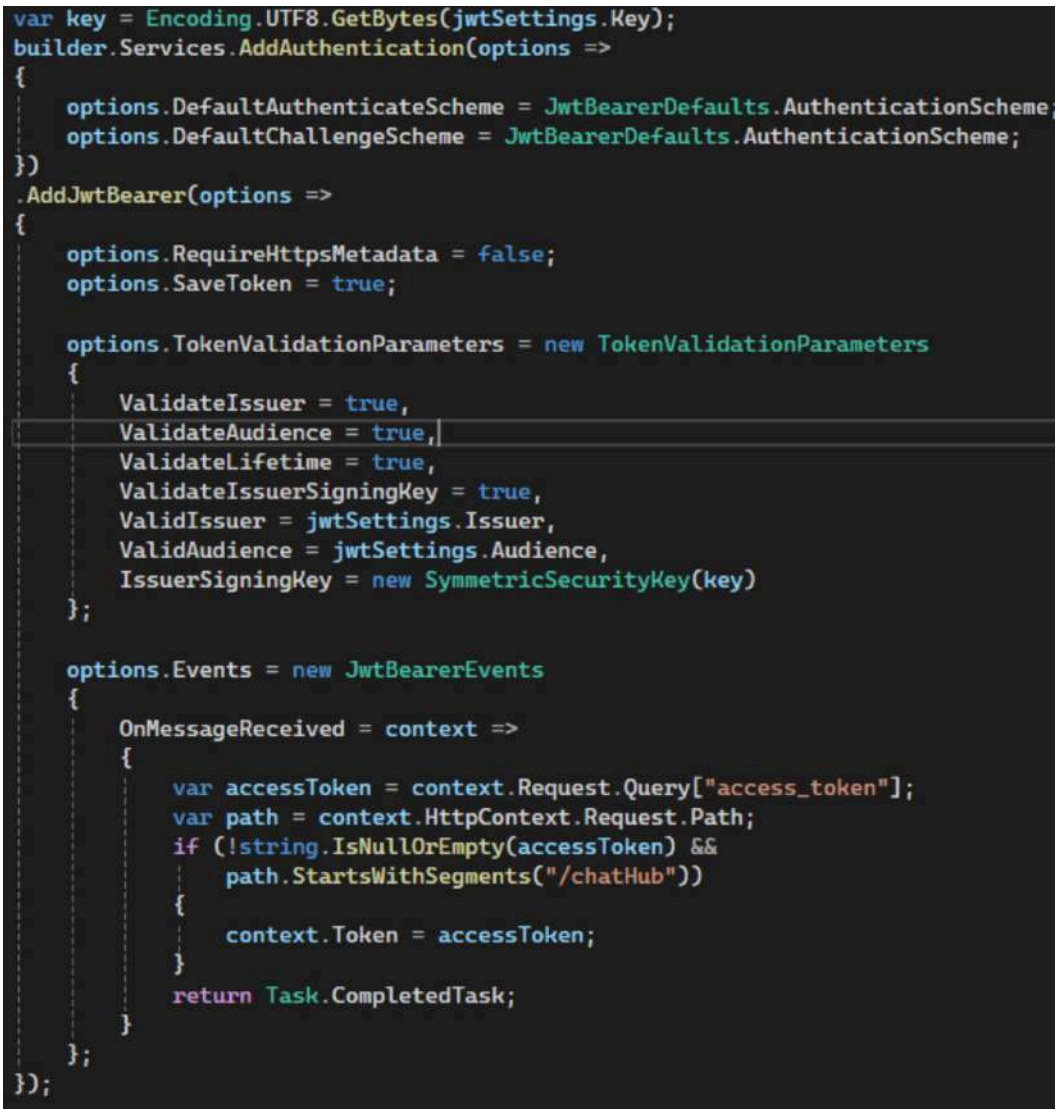
The entry point of the ASP .NET Core application is configured in Program.cs. This file is responsible for service registration, database connection, authentication setup and application launch. The configuration is shown in several consecutive code snippets below.

```
var builder = WebApplication.CreateBuilder(args);

// config
builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("Jwt"));
var jwtSettings = builder.Configuration.GetSection("Jwt").Get<JwtSettings>()!;

// Db
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection"))

// DbContextFactory для Hosted Service
builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection"))
    ServiceLifetime.Scoped);

// services
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<IPushTokenService, PushTokenService>();
builder.Services.AddScoped<PushService>();
builder.Services.AddSingleton<ChatPresenceService>();

// Hosted service
builder.Services.AddHostedService<OrderReminderService>();
builder.Services.AddHostedService<NewOrderNotificationService>();

// SignalR
builder.Services.AddSignalR();
```

Figure 4.63 Service registration (source: *Developed by the authors*, VisualStudioCode)

This section registers the database context (SQLite), business services (authentication, order) and hosted services for reminders and notifications along with SignalR for chat.

1. Initializes WebApplicationBuilder;
2. Loads JWT settings from configuration;
3. Registers the Entity framework Core AppDbContext with SQLite as the database provider;
4. Adds both regular scoped services and a factory for hosted service. All core business services are registered as scoped (iAuthService, iOrderService etc.)
5. Additionally, a couple of background services are added: OrderReminderService for upcoming order reminders and NewOrderNotificationService for alerting drivers about new available orders;
6. SignalR is enabled for real-time communication.

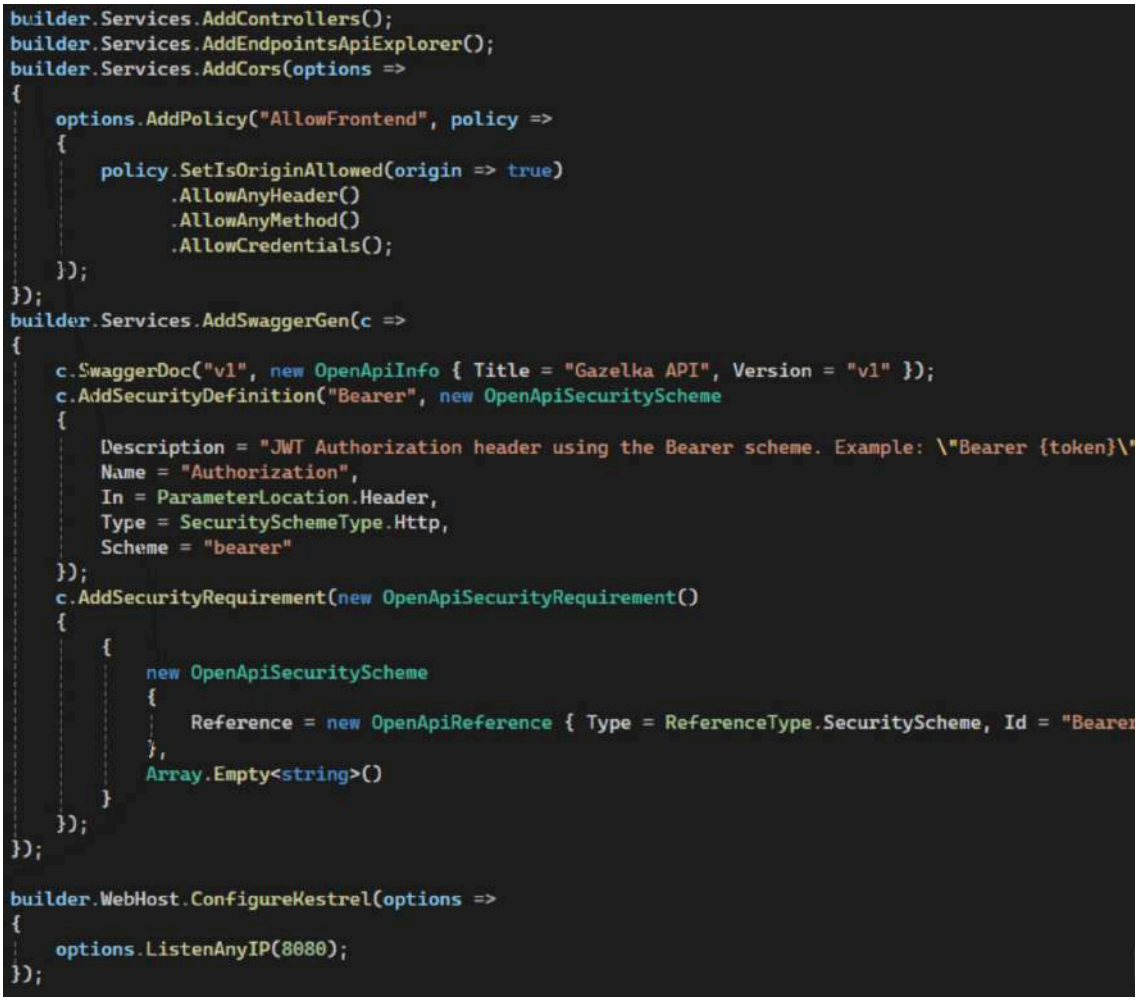


```
var key = Encoding.UTF8.GetBytes(jwtSettings.Key);
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.RequireHttpsMetadata = false;
    options.SaveToken = true;

    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings.Issuer,
        ValidAudience = jwtSettings.Audience,
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };

    options.Events = new JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            var accessToken = context.Request.Query["access_token"];
            var path = context.HttpContext.Request.Path;
            if (!string.IsNullOrEmpty(accessToken) &&
                path.StartsWithSegments("/chatHub"))
            {
                context.Token = accessToken;
            }
            return Task.CompletedTask;
        }
    };
});
```

Figure 4.64 JWT Bearer authentication (source: *Developed by the authors*, VisualStudioCode)

The second block (Figure 4.64) configures JWT Bearer authentication as the generic scheme. Token validation parameters are set to check issuer, audience, lifetime and signing key.

A custom onMessageReceived event handler is added to extract the JWT token from the query string (access_token) when connecting to the SignalR hub (/chatHub) because WebSocket connections do not support standard Authorisation headers. This ensures secure real-time chat while maintaining token-based protection for all endpoints.



```
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.SetIsOriginAllowed(origin => true)
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "Gazelka API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Description = "JWT Authorization header using the Bearer scheme. Example: \"Bearer {token}\"
        Name = "Authorization",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.Http,
        Scheme = "bearer"
    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement()
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer
            },
            Array.Empty<string>()
        }
    });
});

builder.WebHost.ConfigureKestrel(options =>
{
    options.ListenAnyIP(8080);
});
```

Figure 4.65 API controllers (source: *Developed by the authors*, VisualStudioCode)

The third piece of code enables MVC controllers and OpenAPI data generation.

1. Swagger, in this context, is configured to document the API under the title of ‘Gazelka API’ and include Bearer token authentication support in the UI;
2. Permissive CORS policy named AllowFrontend is defined to allow requests from any origin that is essential for both web and mobile clients during development;
3. Kestrel is configured to listen on all IP addresses on port 8080, making the API accessible from local network devices.

```
var mdns = new MulticastService();
var sd = new ServiceDiscovery(mdns);

mdns.Start();

var service = new ServiceProfile("Gazelka API", "_http._tcp", 8080);

sd.Advertise(service);

AppDomain.CurrentDomain.ProcessExit += (s, e) => mdns.Stop();


var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseCors("AllowFrontend");

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.MapHub<ChatHub>("/chatHub");

// ensure db
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}

// Firebase
FirebaseApp.Create(new AppOptions()
{
    Credential = GoogleCredential.FromFile("gazelka-5cd2d-firebase-adminsdk-fbsvc-e6eb7385c2.json"
});

app.Run();
```

Figure 4.66 Application launch (source: *Developed by the authors*, VisualStudioCode)

This final block build the application pipeline and applies middleware in the correct order:

Swagger UI - CORS - authentication - authorisation - controller mapping - SignalR Hub mapping (/chatHub)

1. Database migrations are automatically applied on startup using a scoped AppBdContext. Firebase admin SDK is inialised with a service account JSON file to enable FCM push notifications;
2. mDNS (mulitcast DNS) service discovery is set up to advertise the API as '*http.*tcp' on port 8080 that allows devices on the local network to auto-discover the server without manual API entry. The application is finally started with app.Run().

App database context

AppDbContext (Figure 4.67) is the entity framework core DbContext that is the main bridge between C# server and SQLite database. Its core functionalities are mapping C# classes to database tables, defining DbSet properties for each table and configuring indexes with unique constraints.

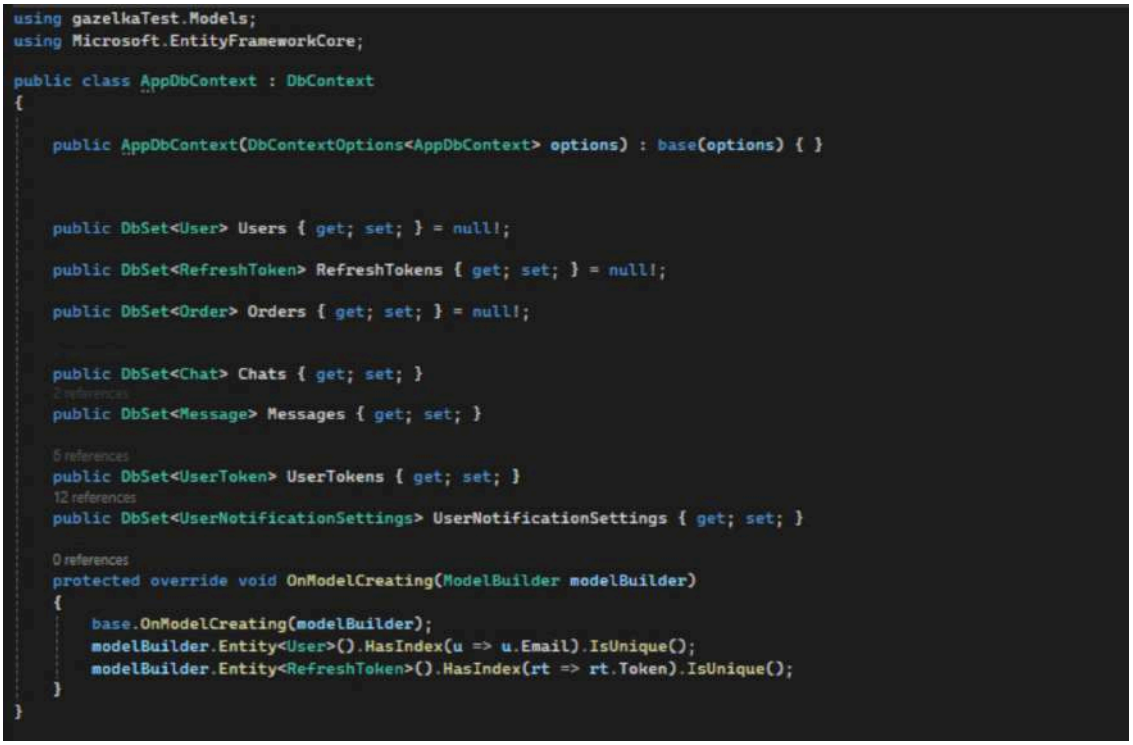


Figure 4.67 Application database context (source: *Developed by the authors*, VisualStudioCode)

1. Receives DbContextOptions configured in Program.cs with SQLite connection string;
2. Each of the DbSet<T> properties represents a table:

    - Users table is for main users table (customers + drivers);
    - RefreshTokens is for JWT token refresh;
    - Orders is for all moving and delivering orders;
    - Chats is for one cat per order;
    - Messages is for chat messages;
    - UserTokens id for FCM push tokens per user/device;
    - UserNotificationSettings is for per-user push preferences

3. Configurates a model according to a table. HasIndex(...).IsUnique() creates unique indexes on User.Email that prevents duplicate email during registration and RefreshToken.Token that ensures no duplicate refresh tokens.

Database entities and domain models

The 'Gazelka' back-end also uses entity framework core with a code-first approach, where C# classes directly define the database schema. The AppDbContext (Figure 4.67) class maps these entities to SQLite tables and domain models that represent critical data structures.

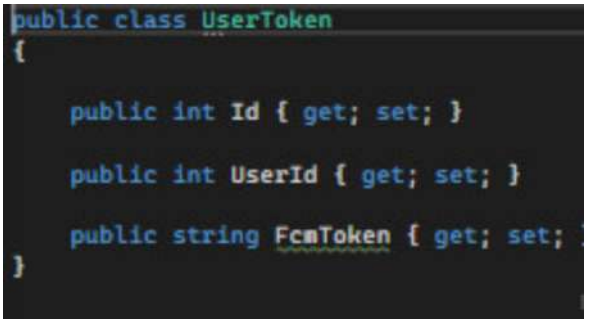


Figure 4.68 UserToken entity (source: *Developed by the authors*, VisualStudioCode)

The UserToken class represents a row in the UserToken table, which stores Firebase Cloud Messaging (FCM) registration tokens for each user's device. This entity enables multidevice support and is queried before sending FCM messages and updated via the /api/push.token endpoint. Its key fields are:

- id for a primary key;
- UserId as a foreign key to Users;
- FcmToken as the actual long FCM token string.

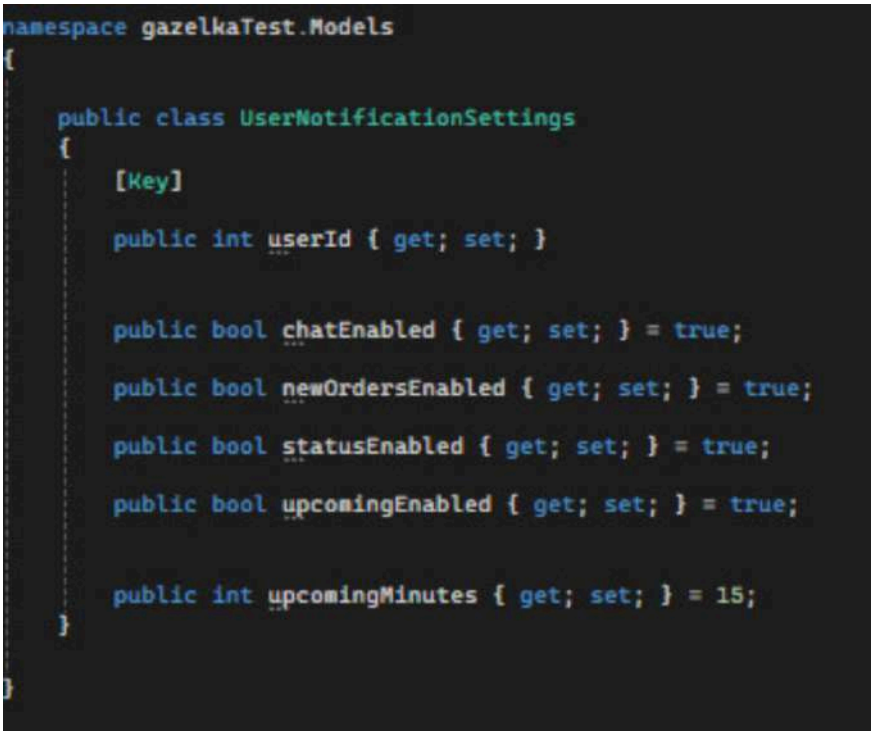
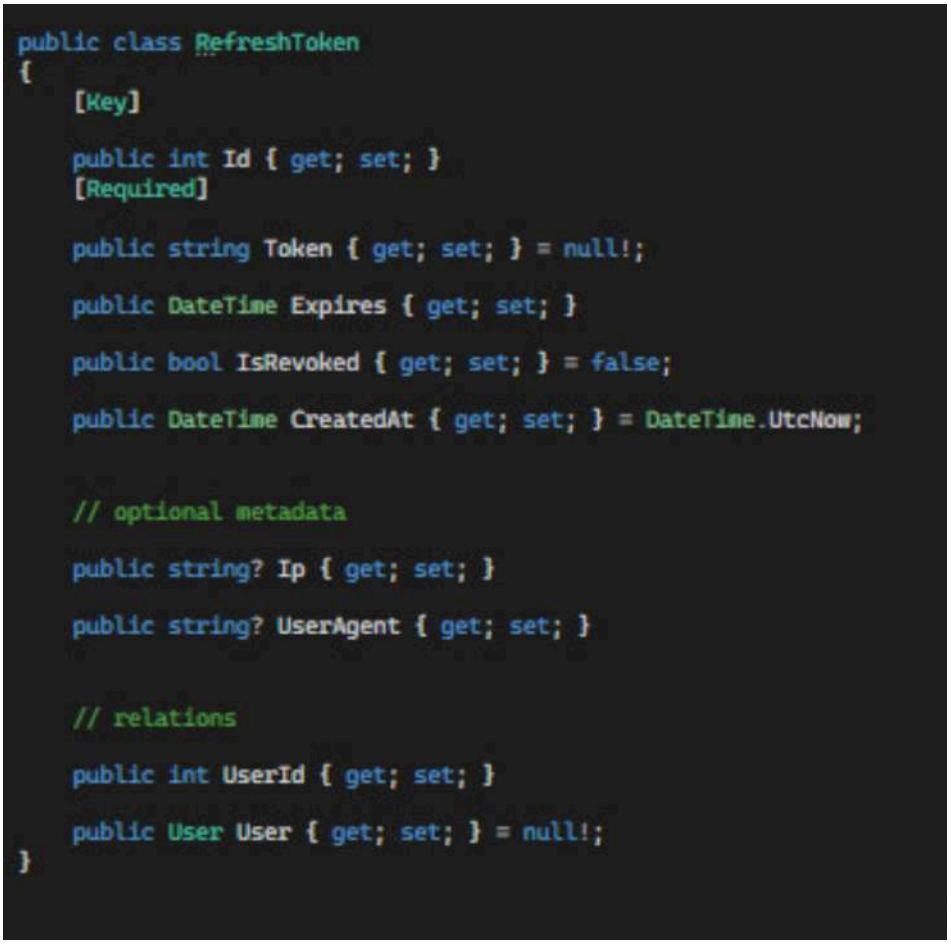


Figure 4.69 UserNotificationSettings entity (source: *Developed by the authors*, VisualStudioCode)

The UserNotificationSettings class (Figure 4.69) defines per-user preferences for push notifications and maps to a table of the same name. It contains:

- Id as a composite key;
- boolean flags (on chatEnabled, newOrdersEnabled, etc.) that control whether a user receives notifications;
- upComingMinutes integer specifies how many minutes before an order start time a reminder should be sent (default is 15 minutes)

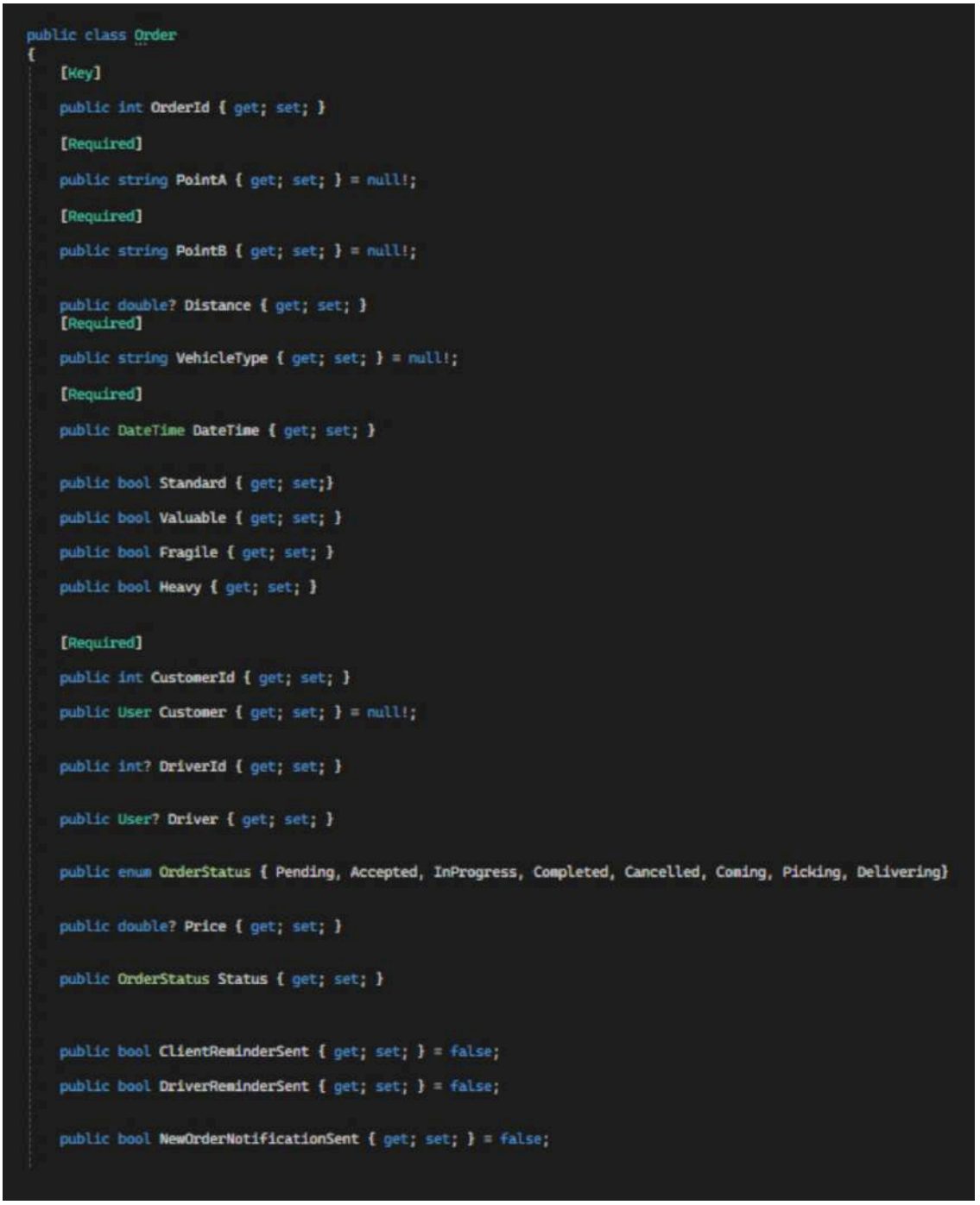


```
public class RefreshToken
{
    [Key]

    public int Id { get; set; }
    [Required]

    public string Token { get; set; } = null!;

    public DateTime Expires { get; set; }

    public bool IsRevoked { get; set; } = false;

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;


    // optional metadata

    public string? Ip { get; set; }

    public string? UserAgent { get; set; }


    // relations

    public int UserId { get; set; }

    public User User { get; set; } = null!;
}
```

Figure 4.70 RefreshToken entity (source: *Developed by the authors*, VisualStudioCode)

The RefreshToken entity models tokens for JWT-based sessions. Tokens are stored in the RefreshTokens table and used for issuing new access tokens without re-login. Its fields includes:

1. individual Id;
2. Token as a unique secure string;
3. Expires for expiration lifetime;
4. IsRevoked as a boolean flag;
5. CreatedAt with a date of creation;
6. optional IP address;
7. UserAgent for security audit
8. UserId + User navigation property (relation to Users)



```
public class Order
{
    [Key]
    public int OrderId { get; set; }
    [Required]
    public string PointA { get; set; } = null!;
    [Required]
    public string PointB { get; set; } = null!;

    public double? Distance { get; set; }
    [Required]
    public string VehicleType { get; set; } = null!;
    [Required]
    public DateTime DateTime { get; set; }

    public bool Standard { get; set;}
    public bool Valuable { get; set; }
    public bool Fragile { get; set; }
    public bool Heavy { get; set; }

    [Required]
    public int CustomerId { get; set; }
    public User Customer { get; set; } = null!;

    public int? DriverId { get; set; }

    public User? Driver { get; set; }

    public enum OrderStatus { Pending, Accepted, InProgress, Completed, Cancelled, Coming, Picking, Delivering}

    public double? Price { get; set; }

    public OrderStatus Status { get; set; }


    public bool ClientReminderSent { get; set; } = false;
    public bool DriverReminderSent { get; set; } = false;

    public bool NewOrderNotificationSent { get; set; } = false;
```

Figure 4.71 Order entity (source: *Developed by the authors*, VisualStudioCode)

The Order class is the central domain model for delivery requests. It drives the entire order cycle from creation till complete and is provided with:

1. individual Id;
2. pointA/pointB as addreses;
3. Distance;
4. VehicleType;
5. DateTime as a scheduled time;
6. Cargo flags (standard, valuable, fragile, heavy);
7. CustomerId/Customer and optional DriverId.Driver as relations to Users;

8. Price;
9. Order status (pending, accepted, coming, picking, delivering, completed);
10. Boolean flags (ClientReminderSent, DriverReminderSent etc.) to prevent duplicate notifications.

API controllers and chat hub

The selected controllers described below handle push token registration (Figure 4.72), order creation (Figure 4.73) and real-time chat management (Figure 4.74). They show the core implementation of these components, including authorisation and business logic integration.

```
[ApiController]
[Route("api/push")]
1 reference
public class PushController : ControllerBase
{
    private readonly IPushTokenService _pushTokenService;

    0 references
    public PushController(IPushTokenService pushTokenService)
    {
        _pushTokenService = pushTokenService;
    }

    [HttpPost("token")]
    [Authorize]
    0 references
    public async Task<IActionResult> SaveToken([FromBody] FcmTokenDto dto)
    {
        if (string.IsNullOrEmpty(dto.Token))
            return BadRequest("Token is required");

        var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier);
        if (userIdClaim == null)
            return Unauthorized();

        if (!int.TryParse(userIdClaim.Value, out int userId))
            return BadRequest("Invalid user ID");

        await _pushTokenService.SaveTokenAsync(userId, dto.Token);

        return Ok(new { message = "FCM token saved" });
    }
}
```

Figure 4.72 Push controller (source: *Developed by the authors*, VisualStudioCode)

The controller is responsible for device token management in Firebase Cloud messaging.

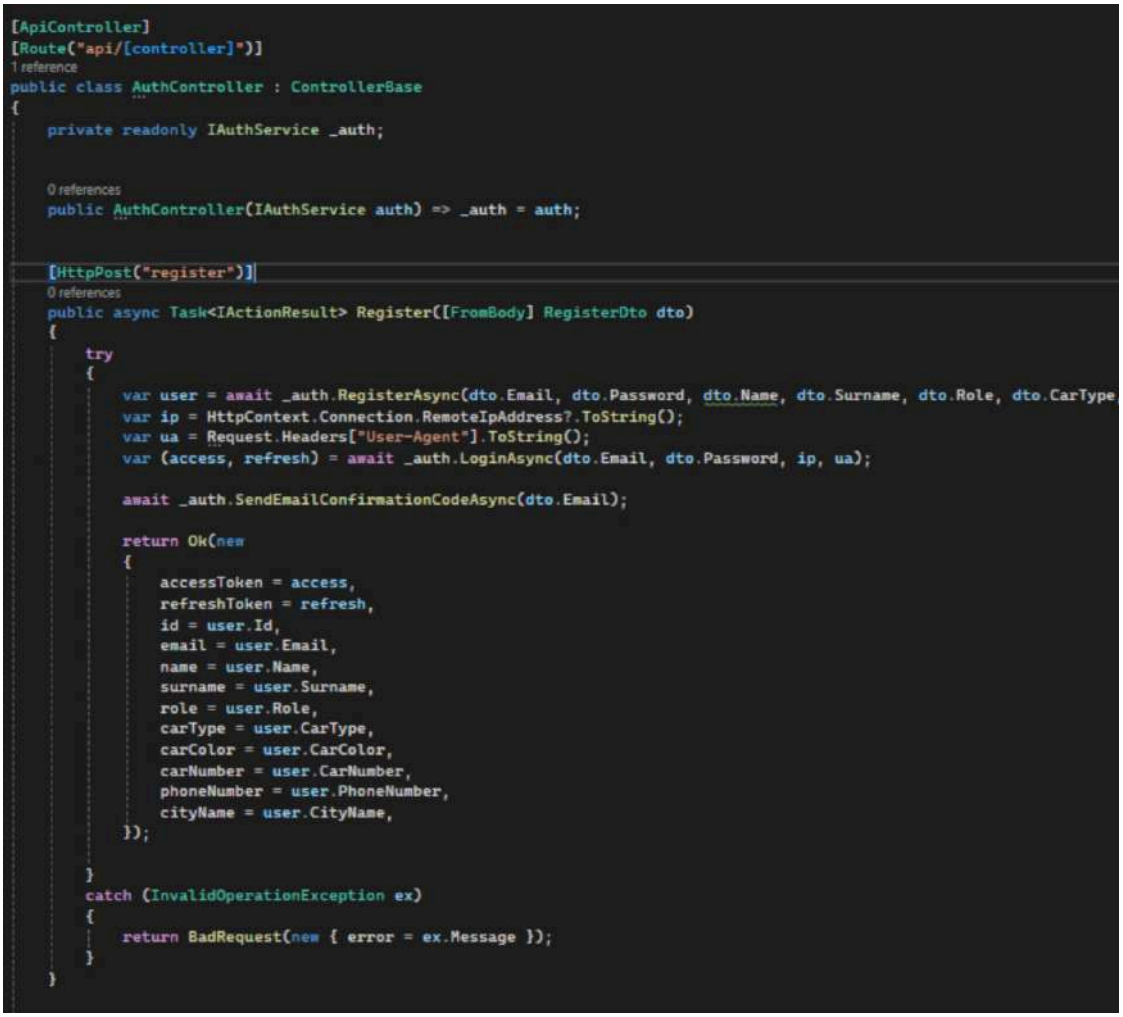
- It goes with [ApiController], [Route(“api/push”) and requires authorization;
- Constructor injects iPushTokenService;
- SaveToken action (POST /api/push/token) accepts a FcmTokenDto from the body, validates the token, extracts the authenticated userId from JWT claims and calls the service to persist the token;
- Returns a success message on completion or appropriate error responses.

```
[ApiController]
[Route("api/[controller]")]
1 reference
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;

    private readonly PushService _pushService;

    private readonly AppDbContext _db;

    0 references
    public OrdersController(IOrderService orderService, PushService pushService, AppDbContext db)
    {
        _orderService = orderService;
        _pushService = pushService;
        _db = db;
    }

    [HttpPost("create")]
    [Authorize(Roles = "customer")]
    0 references
    public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto dto)
    {
        var customerIdClaim = User.Claims.FirstOrDefault(c => c.Type == JwtRegisteredClaimNames.Sub || c.Type

        if (!int.TryParse(customerIdClaim?.Value, out var customerId))
            return Unauthorized("Invalid user id in token");

        var order = await _orderService.CreateOrderAsync(customerId, dto); ;



        return Ok();
    }

    [HttpGet("customerScheduled")]
    [Authorize(Roles = "customer")]
    0 references
    public async Task<IActionResult> GetCustomerActiveOrders()
    {
        var customerIdClaim = User.Claims.FirstOrDefault(c => c.Type == JwtRegisteredClaimNames.Sub || c.Type
        if (!int.TryParse(customerIdClaim?.Value, out var customerId))
            return Unauthorized("Invalid user id in token");

        var customerOrders = await _orderService.GetOrdersByCustomerAsync(customerId);
        var activeOrders = customerOrders
            .Where(o => o.Status == Order.OrderStatus.Pending || o.Status == Order.OrderStatus.Accepted || o.S
            .OrderBy(o => o.DateTime)
            .ToList();

        return Ok(activeOrders);
    }
```

Figure 4.73 Order controller (source: *Developed by the authors*, VisualStudioCode)

OrdersController manages order-related operations.

- Injects IOrderService, PushService and AppDbContext via constructor;
- The CreateOrder action (POST /api/orders/create) is restricted to the customer role. It extracts customerId from JWT claims, calls CreateOrderAsync to process the incoming CreateOrderDto and returns OK on success;
- The GetCustomerActiveOrders action (GET /api/orders/customerScheduled) fetches and returns the customer's active orders after validation its id.



```
[ApiController]
[Route("api/[controller]")]
1 reference
public class AuthController : ControllerBase
{
    private readonly IAuthService _auth;


    0 references
    public AuthController(IAuthService auth) => _auth = auth;


    [HttpPost("register")]
    0 references
    public async Task<IActionResult> Register([FromBody] RegisterDto dto)
    {
        try
        {
            var user = await _auth.RegisterAsync(dto.Email, dto.Password, dto.Name, dto.Surname, dto.Role, dto.CarType
            var ip = HttpContext.Connection.RemoteIpAddress?.ToString();
            var ua = Request.Headers["User-Agent"].ToString();
            var (access, refresh) = await _auth.LoginAsync(dto.Email, dto.Password, ip, ua);

            await _auth.SendEmailConfirmationCodeAsync(dto.Email);

            return Ok(new
            {
                accessToken = access,
                refreshToken = refresh,
                id = user.Id,
                email = user.Email,
                name = user.Name,
                surname = user.Surname,
                role = user.Role,
                carType = user.CarType,
                carColor = user.CarColor,
                carNumber = user.CarNumber,
                phoneNumber = user.PhoneNumber,
                cityName = user.CityName,
            });

        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { error = ex.Message });
        }
    }
```

Figure 4.74 Authentication controller (source: *Developed by the authors*, VisualStudioCode)

The AuthenticationController is the central REST API controller responsible for all user authentication-related operations. Its endpoints are [ApiController] and [Route(“api/[controller]”)] that accessible under /api/Auth/*. This controller requires no global [Authorize] attribute but protected actions use it selectively.

Dependencies injected through constructor:

1. IAuthService _auth is injected business service handling registration, login, email verification. password reset and token generation.

The main endpoint is POST /api/Auth/register ([HttpGet(“register”)]:

1. Accepts a RegisterDto from request body;
2. Extracts client IP and User Agent for security logging;
3. Calls _auth.RegisterAsync(...) in order to create and save the user to Users table;
4. On succes calls _auth.LoginAsync(...) to generate access and refresh tokens;
5. Sends email confirmation code;
6. Returns 200 OK with a DTO containing accessToken, refreshToken, userId, email, name, surname, role, car details, phone and city name;

7. Catches InvalidOperationException and returns 400 BadRequest with error message

The chat controller (Figure 4.75) is a protected REST API controller responsible for managing order related conversations between customers and drivers. It includes [ApiController], [Route("api/chat")] and [Authorize] that together require a valid JWT token and assure only authenticated individuals can access chat.

```
[ApiController]
[Route("api/chat")]
[Authorize]
public class ChatController : ControllerBase
{
    private readonly AppDbContext _db;

    private readonly PushService _pushService;

    private readonly ChatPresenceService _presence;

    public ChatController(AppDbContext db, PushService pushService, ChatPresenceService presence
    {
        _db = db;
        _pushService = pushService;
        _presence = presence;
    }
    [HttpGet("list")]
    public async Task<IActionResult> GetChats()
    {
        int userId = int.Parse(
            User.FindFirst(ClaimTypes.NameIdentifier)!.Value
        );

        var chats = await _db.Chats
            .Where(c => (c.User1Id == userId || c.User2Id == userId) && c.User2Id != null)
            .Select(c => new {
                ChatId = c.Id,
                OtherUserId = c.User1Id == userId ? c.User2Id : c.User1Id,
                OtherUserName = _db.Users
                    .Where(u => u.Id == (c.User1Id == userId ? c.User2Id : c.User1Id))
                    .Select(u => u.Name + " " + u.Surname)
                    .FirstOrDefault(),
                OrderId = c.OrderId,
                LastMessage = c.Messages
                    .OrderByDescending(m => m.SentAt)
                    .Select(m => new {
                        m.Id,
                        m.Text,
                        m.SenderId,
                        m.SentAt
                    })
                    .FirstOrDefault()
            })
            .ToListAsync();

        return Ok(chats);
    }
```

Figure 4.75 Chat controller (source: *Developed by the authors*, VisualStudioCode)

Dependencies injected through constructor:

1. AppDbContext _db - for direct database access;
2. Pushservice _pushService for sending FCM push notifications when a new message is sent to an offline user;
3. ChatPresenceService _presence is a service that tracks which users are currently viewing which chats.

The main endpoint is GET /api/chat/list ([HttpGet("list")]:

4. Extracts the authenticated userId from JWT claims;
5. Queries the Chats table for all chats where the current user is either User1Id for customer and User2Id for driver;
6. For every chat builds a DTO with:

- chatId;
- OtherUsedId for conversation partner;
- OtherUserName that is fetched from Users table;
- OrderId
- LastMessage as the most recent message text and timestamp);

```
[Authorize]

public class ChatHub : Hub
{
    private readonly ChatPresenceService _presence;

    0 references
    public ChatHub(ChatPresenceService presence)
    {
        _presence = presence;
    }

    0 references
    public override Task OnConnectedAsync()
    {
        var userId = int.Parse(Context.UserIdentifier!);
        Groups.AddToGroupAsync(Context.ConnectionId, userId.ToString(
        return base.OnConnectedAsync();
    }

    0 references
    public Task EnterChat(int chatId)
    {
        var userId = int.Parse(Context.UserIdentifier!);
        _presence.EnterChat(userId, chatId);
        return Task.CompletedTask;
    }

    0 references
    public Task LeaveChat()
    {
        var userId = int.Parse(Context.UserIdentifier!);
        _presence.LeaveChat(userId);
        return Task.CompletedTask;
    }

    0 references
    public override Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = int.Parse(Context.UserIdentifier!);
        _presence.LeaveChat(userId);
        return base.OnDisconnectedAsync(exception);
    }
}
```

Figure 4.76 Chat hub (source: *Developed by the authors*, VisualStudioCode)

This hub enables real-time presence detection which is used to decode if to send FCM push notifications (are skipped if a user is online or reading the chat). The hub inherits from SignalR Hub and is mapped to /chatHub. It requires authorisation ([Authorize]). Injects ChatPresenceService to track which users are actively viewing which chats. Key methods include:

- OnConnectedAsync that adds the user to their personal group using userId from claims;
- EnterChat/LeaveChat that update presence when a user joins or leaves a specific chat;
- onDisconnectedAsync that cleans up presence on disconnect.

Service interfaces and implementations

The service layer contains the business logic of the application. Here, interfaces describe clear contrasts and implementations handle database operations, calculations and real-time features.

```
public interface IOrderService
{
    Task<Order> CreateOrderAsync(int customerId, CreateOrderDto dto);

    Task<List<Order>> GetOrdersByCustomerAsync(int customerId);

    Task<List<Order>> GetOrdersByDriverAsync(int driverId);
    Task<bool> AcceptOrderAsync(int orderId, int driverId);

    Task<bool> EditOrderAsync(int customerId, EditOrderDto dto);
    Task<bool> CancelOrderAsync(int userId, int orderId, bool isDriver);
    Task<List<Order>> GetPendingOrdersAsync(int driverId);

    Task<Order> GetOrderByIdAsync(int orderId);

    Task<bool> ComingOrderAsync(int orderId, int driverId);

    Task<bool> PickingOrderAsync(int orderId, int driverId);

    Task<bool> DeliveringOrderAsync(int orderId, int driverId);

    Task<bool> ComplitedOrderAsync(int orderId, int driverId);
}
```

Figure 4.77 IOOrderService interface (source: *Developed by the authors*, VisualStudioCode)

The interface defines the complete set operations for managing orders. It serves as the layer between order controller and service. It enables dependency injection, testability and loose coupling.

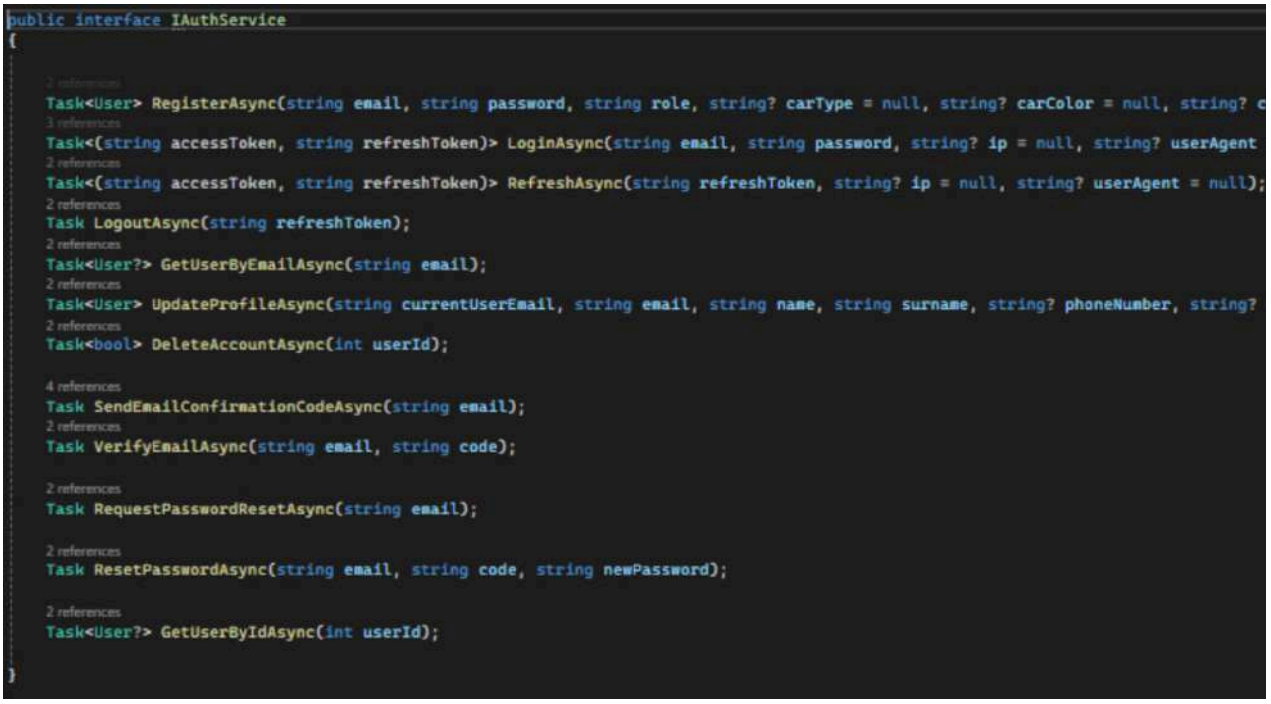
Key methods with their purposes:

1. Task<Order> CreateOrderAsync (int customerId, CreateOrderDto dto) creates a new order for the given customer using data from CreateOrderDto. Finally, it returns the created Order entity;
2. Task<List<Order>> GetOrdersByCustomerAsync (int customerId) retrieves all orders that belong to a specific customer;

3. Task<List<Order>> GetOrdersByDriverAsync (int driverId) retrieves all orders assigned to a specific driver;
4. Task<bool> AcceptOrderAsync (int OrderId, int driverId) allows a driver to accept a pending order. Returns true on success;
5. Task<bool> EditOrderAsync (int customerId, EditOrderDto dto) permits the customer to edit an order before before acceptance;
6. Task<bool> CancelOrderAsync (int userId, int orderId, bool isDriver) cancels an order. The isDriver flag determines permission logic;
7. Task<List<Order>> GetPendingOrderAsync (int driverId) returns a list of currently available orders for drovers to browse and accept;
8. Task<Order> GetOrderByIdAsync (int orderId) fetches a single order by id;

Status update methods that return Task<bool> are:

- ComingOrderAsync;
- PickingOrderAsync;
- DeliveringOrderAsync;
- CompleteOrderAsync.



```
public interface IAuthService
{

    Task<User> RegisterAsync(string email, string password, string role, string? carType = null, string? carColor = null, string? c
    Task<(string accessToken, string refreshToken)> LoginAsync(string email, string password, string? ip = null, string? userAgent
    Task<(string accessToken, string refreshToken)> RefreshAsync(string refreshToken, string? ip = null, string? userAgent = null);
    Task LogoutAsync(string refreshToken);
    Task<User?> GetUserByEmailAsync(string email);
    Task<User> UpdateProfileAsync(string currentUserEmail, string email, string name, string surname, string? phoneNumber, string?
    Task<bool> DeleteAccountAsync(int userId);

    Task SendEmailConfirmationCodeAsync(string email);
    Task VerifyEmailAsync(string email, string code);

    Task RequestPasswordResetAsync(string email);

    Task ResetPasswordAsync(string email, string code, string newPassword);

    Task<User?> GetUserByIdAsync(int userId);
}
```

Figure 4.78 IAuthService interface (source: *Developed by the authors*, VisualStudioCode)

The iAuthService interface defines the complete contract for user authentication and account management. It serves as a layer between authentication controller and service.

Key methods with their purposes:

1. Task<User> RegisterAsync (string email, string password, string role) registers a new user and returns the created User entity;
2. Task<(string accessToken, string refreshToken)> LoginAsync (string email, string password, string? ip = null, string? userAgent = null) authenticates the user, issues JWT access and refreshes tokens. After that, it logs IP and User Agent;
3. Task LogoutAsync(string refreshToken) revokes the specified refresh token that invalidates the session;
4. Task<User?> GetUserByEmailAsync(string email) retrieves a user by email;
5. Task UpdateProfileAsync(string currentUserEmail, string email, string name) updates user profile details;
6. Task<bool> DeleteAccountAsync(int userId) permanently deletes user account;

Email verification tasks are: Task SendEmailConfirmationCodeAsync(string email) for generating/sending verification code and Task VerifyEmailAsync(string email, string code) for verifying the code.

Password reset tasks are: Task RequestPasswordResetAsync (string email) to send reset code and Task ResetPasswordAsync(string email, string code, string newPassword) to complete the password.

```
namespace gazelkaTest.Services
{

    public class ChatPresenceService
    {
        // userId -> chatId
        private readonly ConcurrentDictionary<int, int> _activeChats = new();


        public void EnterChat(int userId, int chatId)
        {
            _activeChats[userId] = chatId;
        }


        public void LeaveChat(int userId)
        {
            _activeChats.TryRemove(userId, out _);
        }


        public bool IsUserInChat(int userId, int chatId)
        {
            return _activeChats.TryGetValue(userId, out var activeChatId)
                   && activeChatId == chatId;
        }
    }

}
```

Figure 4.79 ChatPresenceService interface (source: *Developed by the authors*, VisualStudioCode)

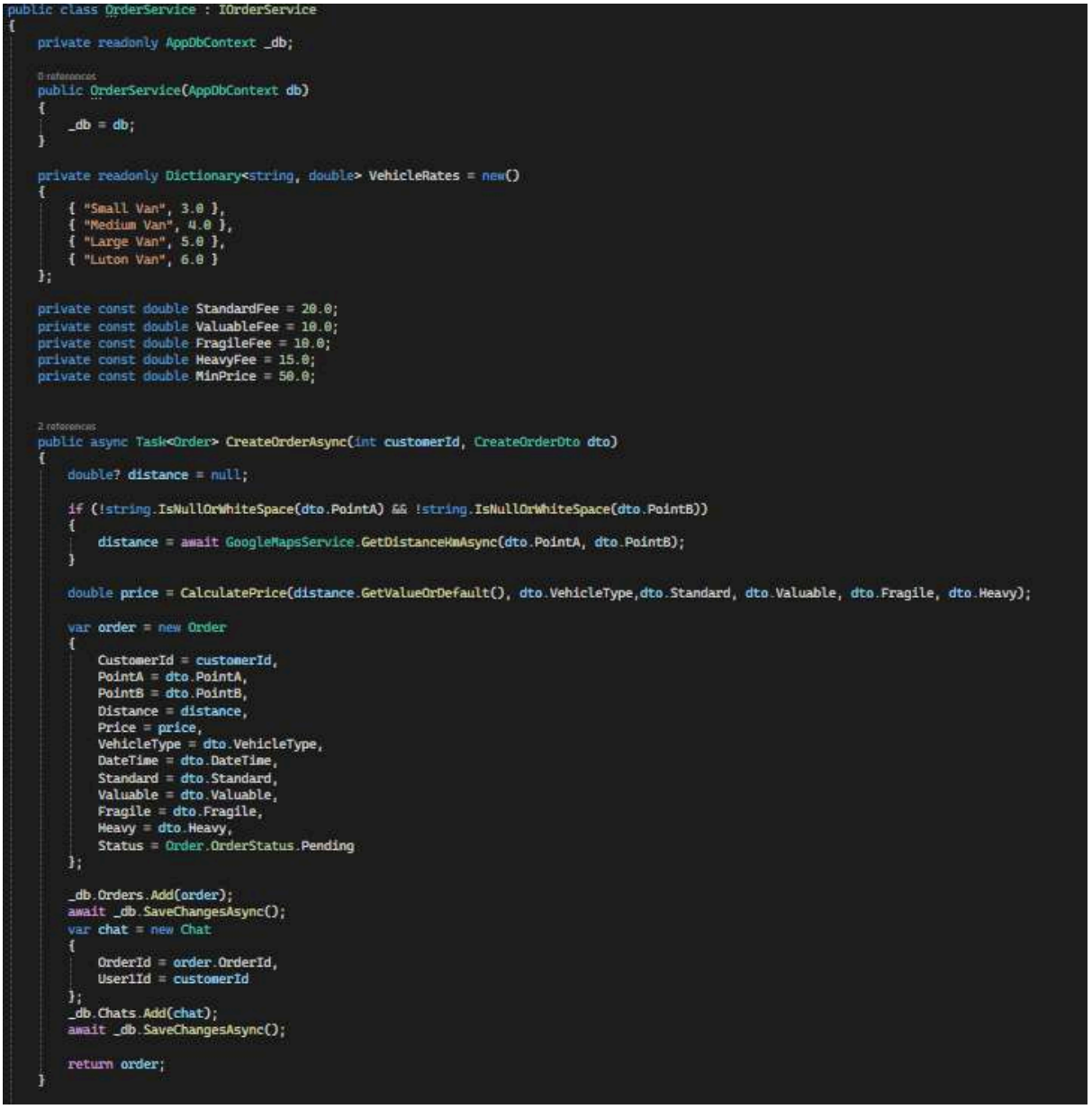
This ChatPresenceService is a service that tracks which users are viewing the conversation in real time. It is used by the SignalR ChatHub to determine user presence and enable the back-end to optimise push notification delivery.

Data structure of the interface:

1. A private readonly ConcurrentDictionary<int, int> _activeChats = new();
2. userId is a key and chatId is a value;
3. Uses ConcurrentDictionary for thread-safety;

Methods of the interface:

1. public void EnterChat(int userId, int chatId) that sets _activeChats[userId] = chatId and marks the user has entered the chat;
2. public void LeaveChat(int userId) that removes the entry for userId from the dictionary using TryRemove;
3. public bool IsUserInChat(int userId, int chatId) checks if the user is currently viewing the specified chat: uses TryGetValue(userId, out var activeChatId) and returns true only if the value matches chatId.



```
public class OrderService : IOrderService
{
    private readonly AppDbContext _db;

    0 references
    public OrderService(AppDbContext db)
    {
        _db = db;
    }

    private readonly Dictionary<string, double> VehicleRates = new()
    {
        { "Small Van", 3.0 },
        { "Medium Van", 4.0 },
        { "Large Van", 5.0 },
        { "Luton Van", 6.0 }
    };

    private const double StandardFee = 20.0;
    private const double ValuableFee = 10.0;
    private const double FragileFee = 10.0;
    private const double HeavyFee = 15.0;
    private const double MinPrice = 50.0;


    2 references
    public async Task<Order> CreateOrderAsync(int customerId, CreateOrderDto dto)
    {
        double? distance = null;

        if (!string.IsNullOrWhiteSpace(dto.PointA) && !string.IsNullOrWhiteSpace(dto.PointB))
        {
            distance = await GoogleMapsService.GetDistanceKmAsync(dto.PointA, dto.PointB);
        }

        double price = CalculatePrice(distance.GetValueOrDefault(), dto.VehicleType,dto.Standard, dto.Valuable, dto.Fragile, dto.Heavy);

        var order = new Order
        {
            CustomerId = customerId,
            PointA = dto.PointA,
            PointB = dto.PointB,
            Distance = distance,
            Price = price,
            VehicleType = dto.VehicleType,
            DateTime = dto.DateTime,
            Standard = dto.Standard,
            Valuable = dto.Valuable,
            Fragile = dto.Fragile,
            Heavy = dto.Heavy,
            Status = Order.OrderStatus.Pending
        };

        _db.Orders.Add(order);
        await _db.SaveChangesAsync();
        var chat = new Chat
        {
            OrderId = order.OrderId,
            UserId = customerId
        };
        _db.Chats.Add(chat);
        await _db.SaveChangesAsync();

        return order;
    }
```

Figure 4.80 OrderService interface (source: *Developed by the authors*, VisualStudioCode)

The last interface implements the main rules for creating, managing and processing orders in the application. It is registered and scoped in Program.cs and injected into controllers through dependency injection.

Key dependencies of the interface:

1. private readonly AppDbContext _db that as injected for database operations;
2. private readonly Dictionary<string, double> VehicleRates that is an in-memory static rates for different transport types.

Private constants for additional fees:

1. StandardFee = 20.0;
2. ValuableFee = 10.0;
3. HeavyFee = 5.0;
4. FragileFee = 10.0
5. MinPrice = 50.0

The main method CreateOrderAsync:

1. Accepts customerId and CreateOrderDto;
2. Calculates distance using GoogleMapsService.GetDistanceKmAsync (externalAPI call);
3. Computes total price through CalculatePrice;
4. Creates a new Order entity;
5. Automatically creates a linked Chat entity (OrderId + User1Id - customerId);
6. Persists both order and chat to the database (_db.Orders.Add, _db.Chats.Add, _db.SaveChangesAsync);
7. Eventually, returns the created Order object.

4.9 Testing methods at various stages

Functional testing of the project is presented in this chapter. It focuses on the main features of the applications that were realised during development process. Each table describes a key function, preconditions, the result observed and specific problems encountered along with their solutions. The problems are drawn from real implementation challenges visible in the code.

Table 4.12 Customer registration test - Web application (source: *Developed by the authors*)

Function name	Customer registration.
Input	Email, password, name, surname and phone.
Observed result	Account created, sessionStorage saved and redirect to validation
Occurred problems	Form data lost on refresh.
Applied solutions	Everything was saved to the sessionStorage before API call.

Table 4.13 Email verification test - Web application (source: *Developed by the authors*)

Function name	Verify email.
Input	Code from email and email from sessionStorage.
Observed result	Account verified, sessionStorage cleaned up and redirected to login.
Occurred problems	Email missing after refresh
Applied solutions	Retrieved everything from sessionStorage and added an alert if email is not found.

Table 4.14 Location selection test - Web application (source: *Developed by the authors*)

Function name	Select location.
Input	Click the map and place markers.
Observed result	Address fields are autofilled owing to reverse geocoding.
Occurred problems	API quota exceeded during repeated testing
Applied solutions	Cached latitude and longitude in sessionStorage fallback to text input.

Table 4.15 Chat test - Web application (source: *Developed by the authors*)

Function name	Send and receive messages.
Input	Type and send text.
Observed result	Real-time message broadcast.
Occurred problems	Messages miss if the connection falls.
Applied solutions	Enabled .withAutomaticReconnect() in HubConnectionBuilder to re-fetch history on reconnect.

Table 4.16 SignalR connection test - Web application (source: *Developed by the authors*)

Function name	Establish and maintain SignalR connection
Input	Page load or chat open
Observed result	Connection starts and logs success.
Occurred problems	Periodical connection falls.
Applied solutions	Added .withAutomaticReconnect() logged onclose, onreconnecting and onreconnected events.

Table 4.17 Reset code request test - Web application (source: *Developed by the authors*)

Function name	Request reset code via email.
Input	Enter email and click Send.
Observed result	Code sent to email.
Occurred problems	Code not received.
Applied solutions	Verified Gmail SMTP config and added /api/Auth/resendEmailCode endpoint

Table 4.18 FCM token registration test - Mobile application (source: *Developed by the authors*)

Function name	Register FCM token on the first launch.
Input	App opened and Firebase generates a token.
Observed result	The token sent to /api/push/token.
Occurred problems	Token not sent
Applied solutions	Added registration on app start with JWT from SharedPreferences POST with the token inside.

Table 4.19 Offline notifications test - Mobile application (source: *Developed by the authors*)

Function name	Push for a new message when the app is closed.
Input	Recipient is offline and a sender sends a message.
Observed result	Push received and a tap opens the chat.
Occurred problems	Push not arriving.
Applied solutions	Added deep-link data (type=chat, orderId, chatId) and checked chatEnabled before sending.

Table 4.20 Locations selection test - Mobile application (source: *Developed by the authors*)

Function name	Select location using GoogleMaps.
Input	Select location on the map
Observed result	Order is sent to /api/orders/create.
Occurred problems	Map quota issues.
Applied solutions	Cached recent locations and added a fallback to text input.

Table 4.21 Accept order test - Mobile application (source: *Developed by the authors*)

Function name	Accept available order.
Input	View list of orders, tap and accept.
Observed result	POST /api/orders/accept and update orders status.
Occurred problems	Race condition.
Applied solutions	Back-end transaction and order availability check before status update.

Table 4.22 Push notifications test - Mobile application (source: *Developed by the authors*)

Function name	Tap push notifications.
Input	Push with orderId and chatId.
Observed result	App opens to correct order and chat.
Occurred problems	Wrong screen opened.
Applied solutions	Added a deep-link handling in app (type=chat, orderId, chatId) parsing.

Table 4.23 Server IP test - Mobile application (source: *Developed by the authors*)

Function name	Enter server IP on first run
Input	Input IPV4 address.
Observed result	App connect to the back-end.
Occurred problems	Wrong IP and failed connection.
Applied solutions	Prompted user to enter host IPV4 on first launch.

5. Description of the project implementation

5.1 Web application installation guide

To run the web version of the project locally or on a development machine, the next steps have to be followed:

1. Pull the required Docker [8] images by opening terminal and running the prompt:

docker pull adzhula/gazelkaserver:latest
docker pull adzhula/gazelkaweb:latest

2. Find your computer's local IP address by opening command prompt and typing **ipconfig** in;
3. Look under the 'Wireless WLAN adapter' in order to find the IPv4 address, then copy it
4. Start the back-end server container according to the following instruction;

docker run -d -p 8080:8080 --name gazelka_server adzhula/gazelkaserver

5. Start the web front-end container by running the following command and replacing (WRITE HERE IPV4) with the IP that was copied in the second step;

docker run -e SERVER_IP=(WRITE HERE IPv4) -e SERVER_PORT=8080 -p 80:80 adzhula/gazelkaweb

6. Access the application. Open a browser and go to http://(YOUR IPV4 ADDRESS):80/. The front-end will automatically connect to the back-end at http://(YOUR IPV4 ADDRESS):8080/api/

5.2 Mobile application installation guide

To run the mobile version of the project locally or on a development machine, the next steps have to be followed:

1. Obtain the latest APK file;
2. Install the APK on your Android device by:

- Enabling 'Install unknown apps' in your device settings;
- Opening the APK file file and tapping 'Install';
- Confirming any permissions requested;

3. Configure the server address for the first launch by:

- Opening the app;
- Entering the server IPV4 address;
- Entering the same IPV4 address you found in the second step if the web application installation;

- The app will now connect to the back-end at http://(YOUR IPV4 ADDRESS):8080/api/

4. Use the application

5.3 Web application user guide

The initial entry point for all users is the Welcome screen (Figure 5.1). As shown on the web application onboarding screen, users are greeted with the brand's value proposition before pressing the 'Create order' button.

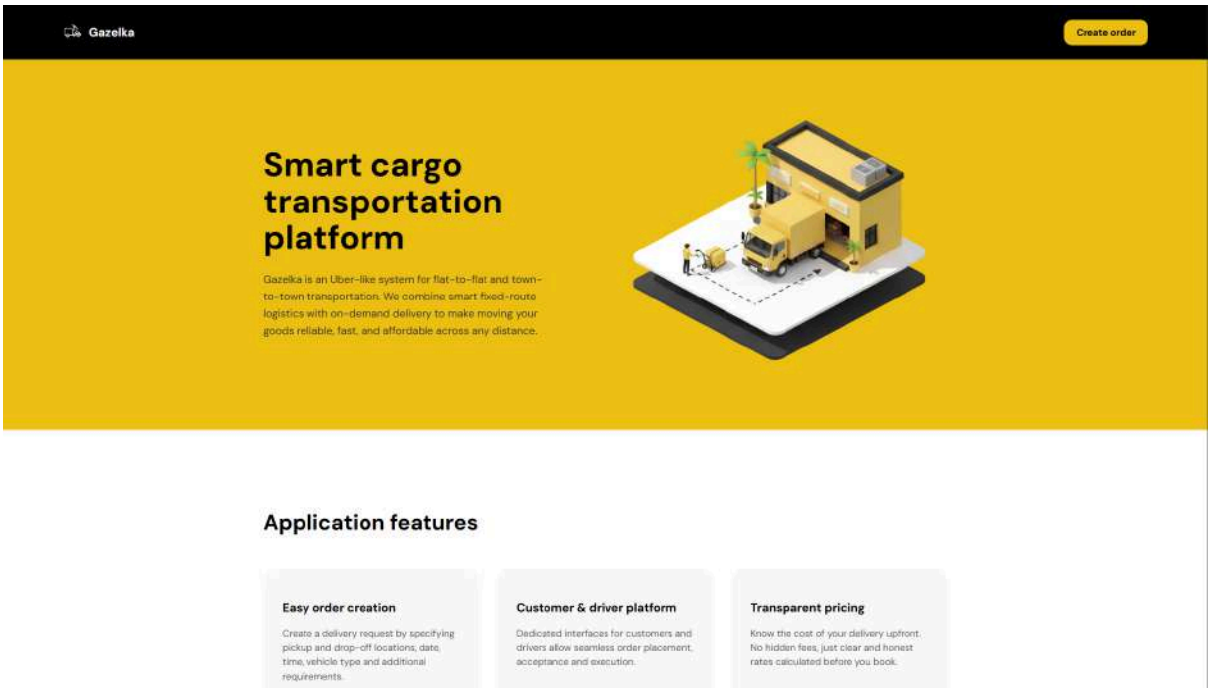


Figure 5.1 Welcome page of web application (source: *Developed by the authors*)

The transition from the landing page to the core services is handled through a unified authentication phase. For returning users, the Login interface (Figure 5.2) continues the brand's visual identity, featuring a dark modal and yellow background with truck graphics. The fields to be filled so as to login to the account are:

- Unique user email used during registration;
- Password consisting of 8 symbols;

This screen allows users to access their personalized dashboards by entering their registered credentials. In order to improve both security and convenience, the password field includes a visibility toggle, while clear links for password recovery and account registration provide easy navigation for these tasks.

Figure 5.2 Login page of web application (source: *Developed by the authors*)

If a user loses access, the Reset password flow provides a secure way to regain it. The process is designed to be straigtforward. The user enters their email to receive a verification code, which they then put alongside a new password. By using 'Send code' and 'Save changes' buttons, the password gets restored without multiple frictions.

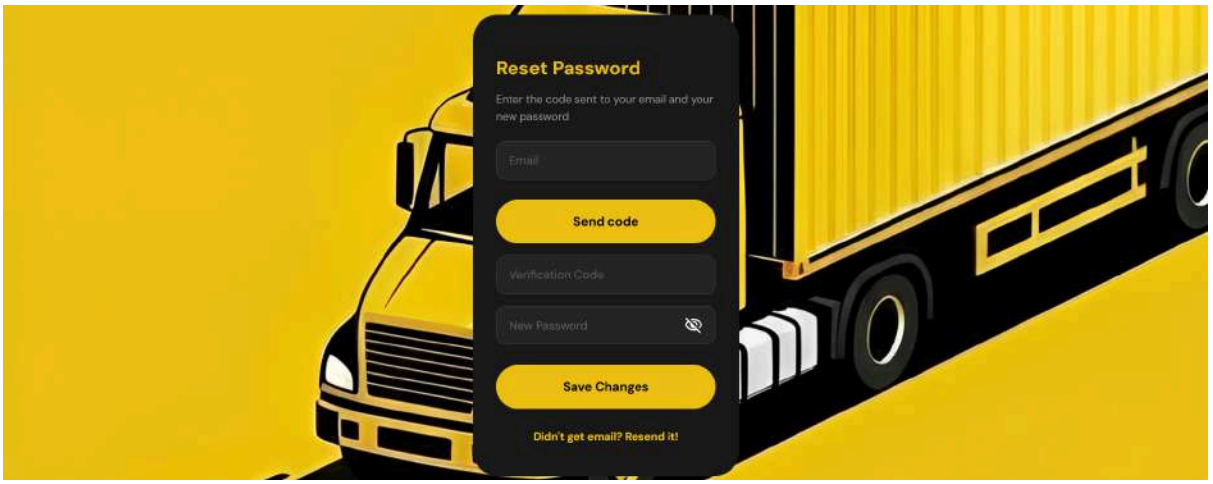


Figure 5.3 Reset password page of web application (source: *Developed by the authors*)

For those new to the platform, the Registration flow serves as the starting point for building their personal record. This interface collects essential information such as:

- Name and surname;
- Email and phone number;
- Password consisting of 8 symbols;

Once this data is entered, a ‘Next step’ button leads to the role selection phase. This allows the system to finalize the setup by tailoring the dashboard experience specifically either a customer or a user.

Figure 5.4 Registration page of web application (source: *Developed by the authors*)

Once a user logs in for the first time after registering, they arrive at the primary orders dashboard. In this initial phase, the interface displays empty state, showing that no scheduled orders have been found.

Figure 5.5 Orders page after the first login to web application (source: *Developed by the authors*)

To move from this blank screen to active use, the user must start a service request. This is done through the ‘New order’ function, easily found via the ‘+’ icon on mobile of the menu on the dashboard of Figure 5.6. The dashboard only populates with interactive order cards once the user defines their order details. This logical flow successfully guides the user from the initial setup to the scheduling transport services.

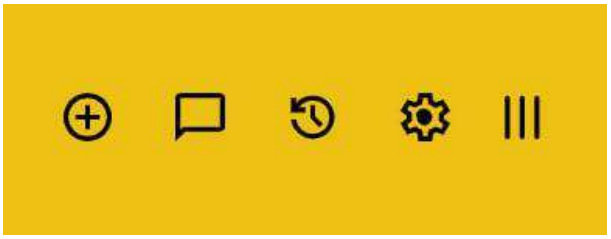


Figure 5.6 Main menu of web application (source: *Developed by the authors*)

In order to start using the service, the user navigates to the *'New Order'* section through the main navigation bar from Figure 5.6. THe order creation process is organized into logical form. First, the user enters the logistics details such as pick-up and drop-off locations, preferred date and time. Next, an illustrated menu allows them to choose a vehicle ranging from small vans to large trucks.

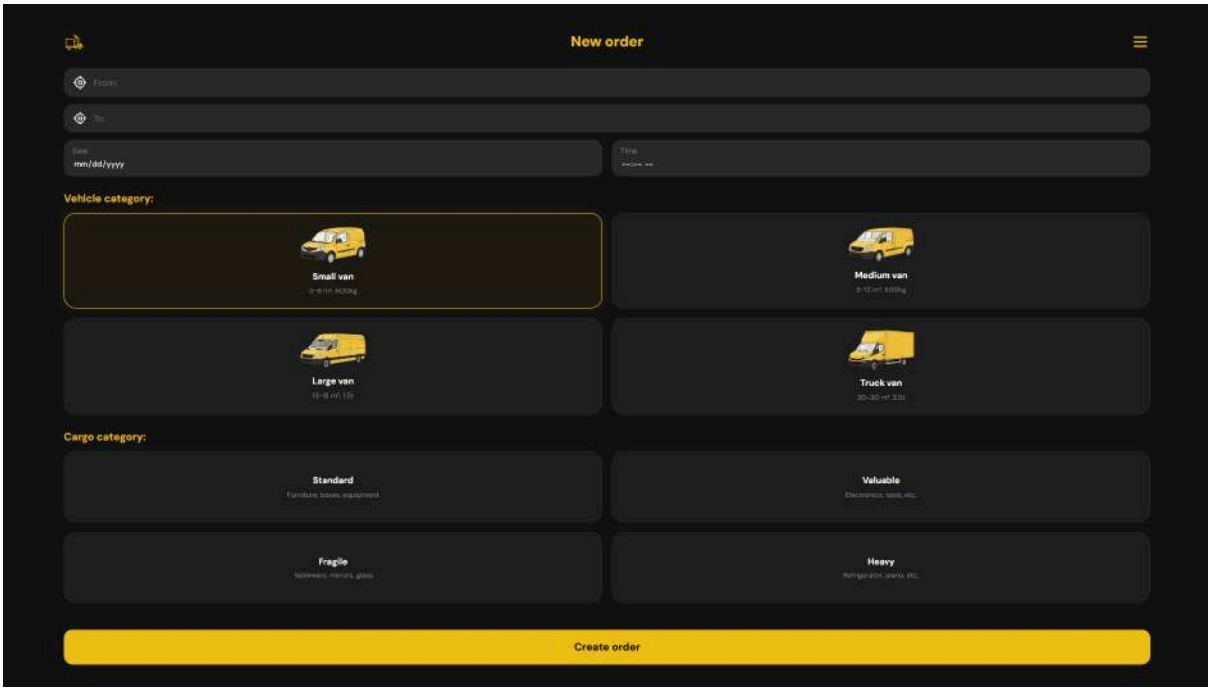


Figure 5.7 Create order page of web application (source: *Developed by the authors*)

The user then classifies their cargo as *Standard, Valuable, Fragile* or *Heavy* , so the driver knows exactly how to handle the goods. Finally, clicking the highcontrast *'Create order'* button sends the information to the backend and instantly generates the first active order card. At this point the empty state on the dashboard is replaced by an interactive view, allowing users to track their order status.

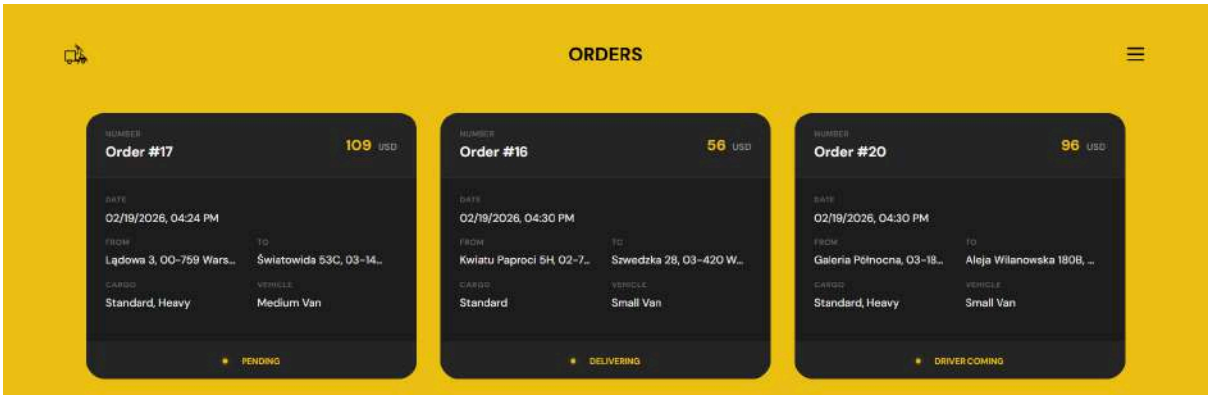


Figure 5.8 Orders page after creating some orders in web application (source: *Developed by the authors*)

Once an order is accepted, the system moves to a detailed monitoring view to keep everything organized. The Order Details screen for a specific request, such as Order 26, shows the current status. scheduled time and cargo type while also calculating the total trip distance (for example. 20.35km).

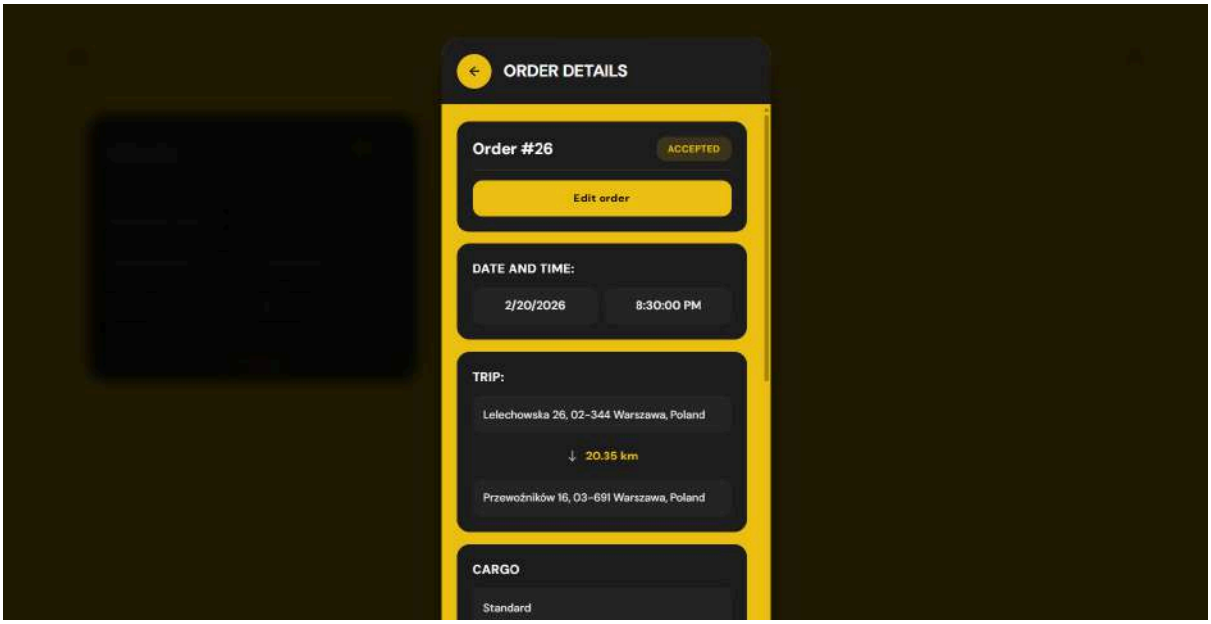


Figure 5.9 Order details modal of web application (source: *Developed by the authors*)

To ensure everyone is on the same page, the interface provides full transparency by listing the driver's name and contact information alongside the vehicle details. Finally, the total price is shown in a high-contrast green font making the cost clear and preventing any confusion before the trip begins.

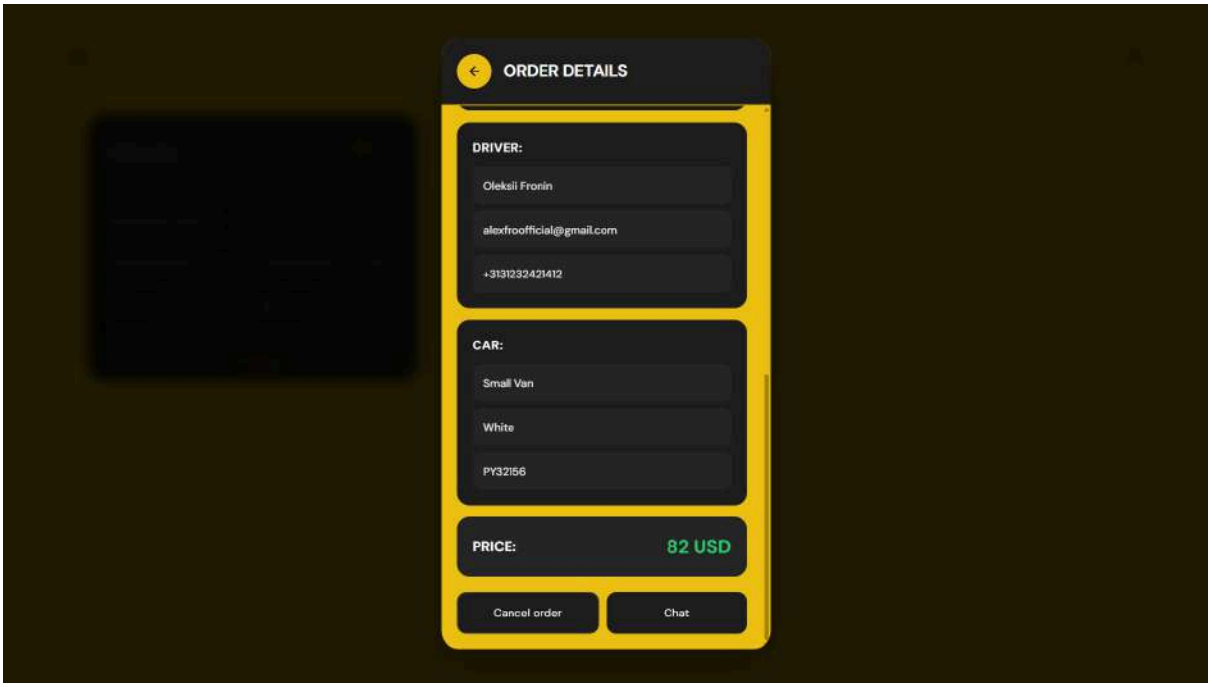


Figure 5.10 Order details modal of web application (source: *Developed by the authors)*

The platform also includes a built-in Chat system that allows customers and drivers to talk directly about specific orders. Users can access a *Chat List*, where active conversations are organized by their order number (for instance, Order 26) making it easy to find the right conversation quickly. The *Messaging Interface* itself is simple, representing a clear window where users can type and send messages. Each message is shown in a bubble with a timestamp, ensuring that the timing and flow of the conversation are easy to follow and professional.

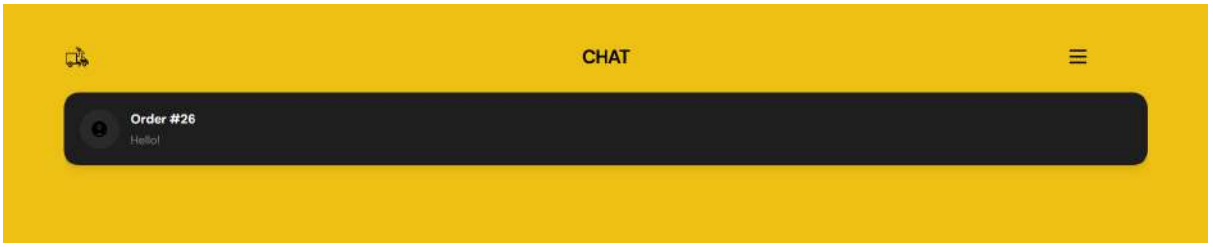


Figure 5.11 Chats home page of web application (source: *Developed by the authors*)

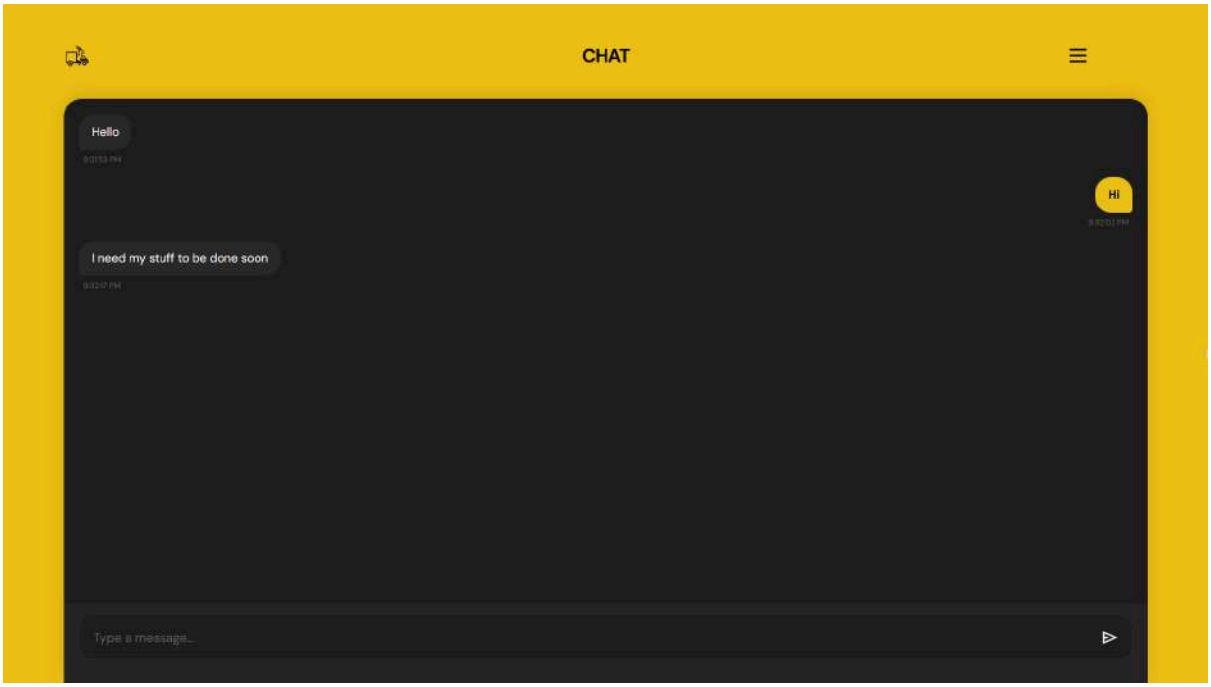


Figure 5.12 Active chat in web application (source: *Developed by the authors*)

The Settings section acts as the central hub for managing personal information and account security. Within the *Profile Overview*, users can view their current account details and access a brief *About Us* section that explains the mission of 'Gazelka'.

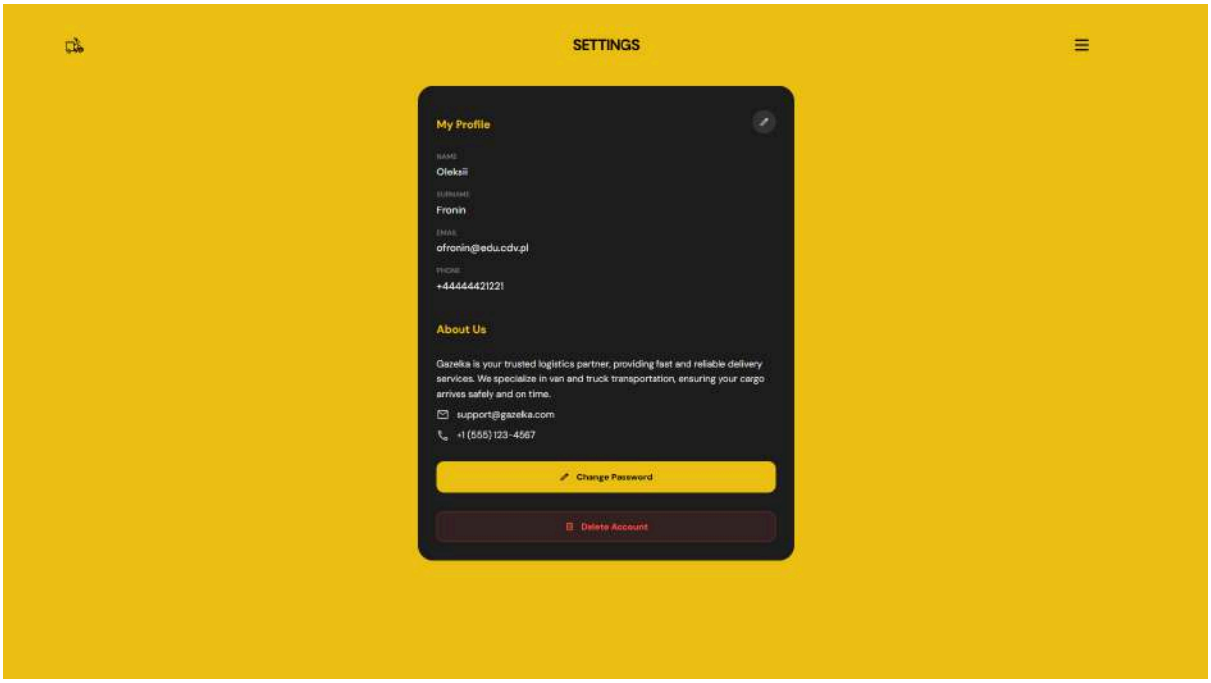


Figure 5.13 Settings page in web application (source: *Developed by the authors*)

To keep information accurate, the interface includes editing capabilities that allow users to update their personal information. Once the changes are made, 'Save Changes' button finalizes the process, assuring all account data remains current and secure.

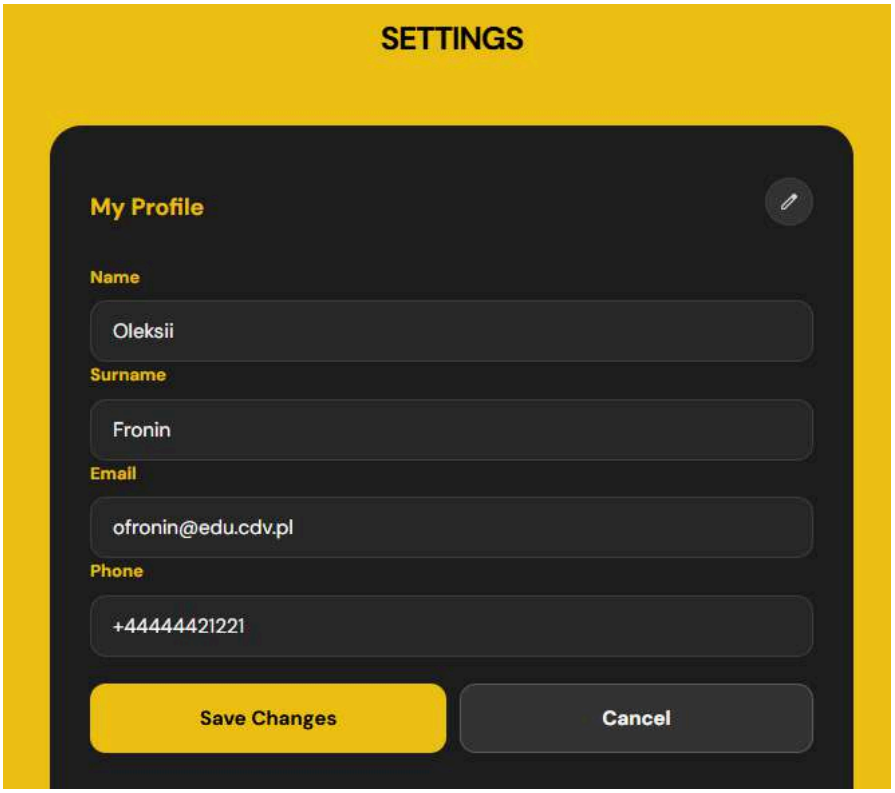


Figure 5.14 Edit personal information page in Settings of web application (source: *Developed by the authors*)

5.4 Mobile application user guide

Similarly to the web application, the Welcome Screen of the mobile application is bright yellow with a large '*Start'* button. Its main job is to smoothly guide customers from the front page to the heart of the system *(Figure 5.15)*.

Figure 5.15 Landing page of the mobile application (source: *Developed by the authors*)

After that the Role Selection scren asks for a simple question whether a user wants to join the platform as a customer or as a driver. This makes sure the app shows you only the related tools you actually need.

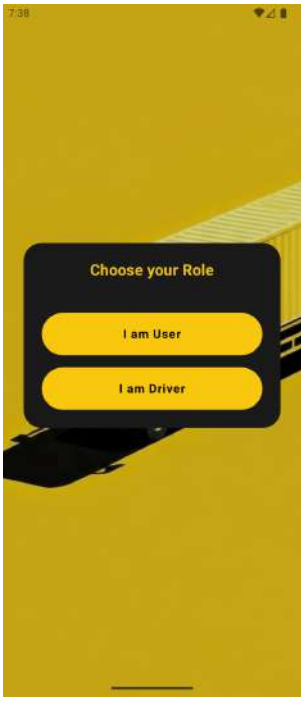


Figure 5.16 Choose your role page of the mobile application (source: *Developed by the authors*)

In order to keep things well organised, the registration page lists everything you need to fill in as a user in a simple vertical row. The login page is even simpliler and designed to get you into your account as fast as possible.

Figure 5.17 Registration pages of the mobile application (source: *Developed by the authors*)

When a user is willing to create a driver account, one of the additional things on this stage is providing a city of service by using Google inserted mapping utility.

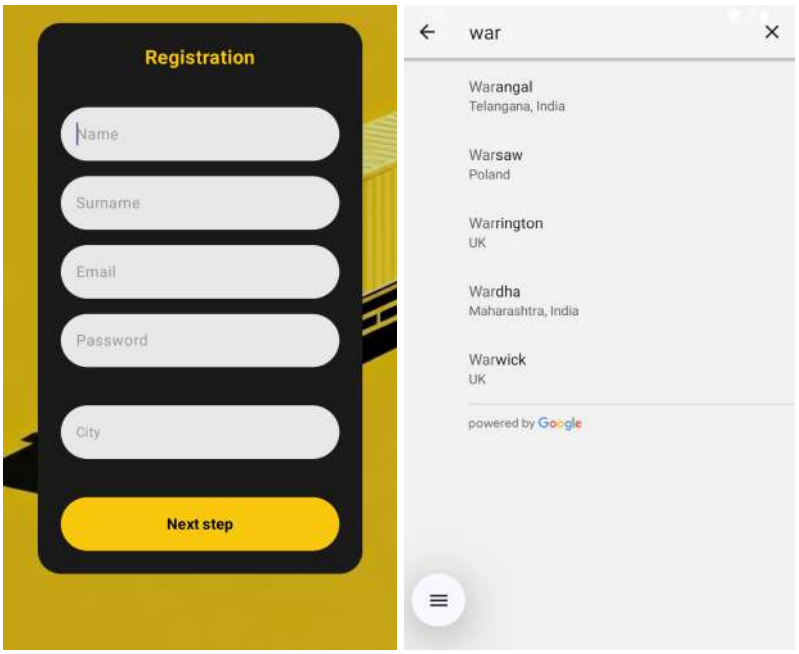


Figure 5.18 Driver registration flow of the mobile application (source: *Developed by the authors*)

The next step of driver registration is providing the neccessary information about their vehicle. It includes filling in certain fields of vehicle description. Each of the input parameters can be chosen via the list occurring after hitting a certain input.

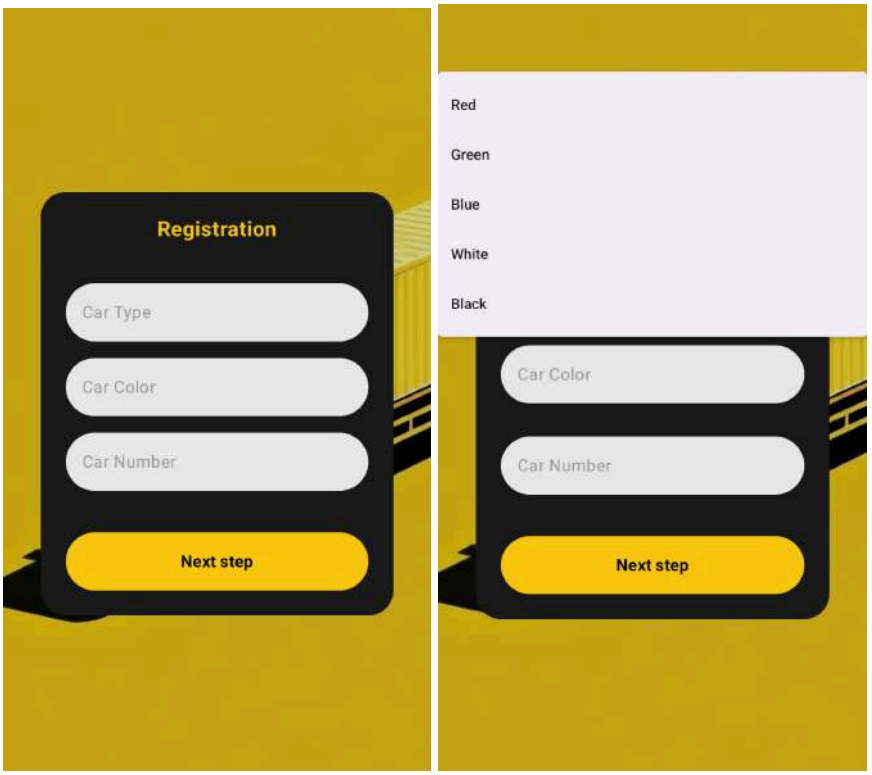


Figure 5.19 Vehicle registration in the mobile application (source: *Developed by the authors*)

If you forget your login details, the Password recovery screen walks you through a couple of quick steps represented by entering your email, then verification code and picking a new password.

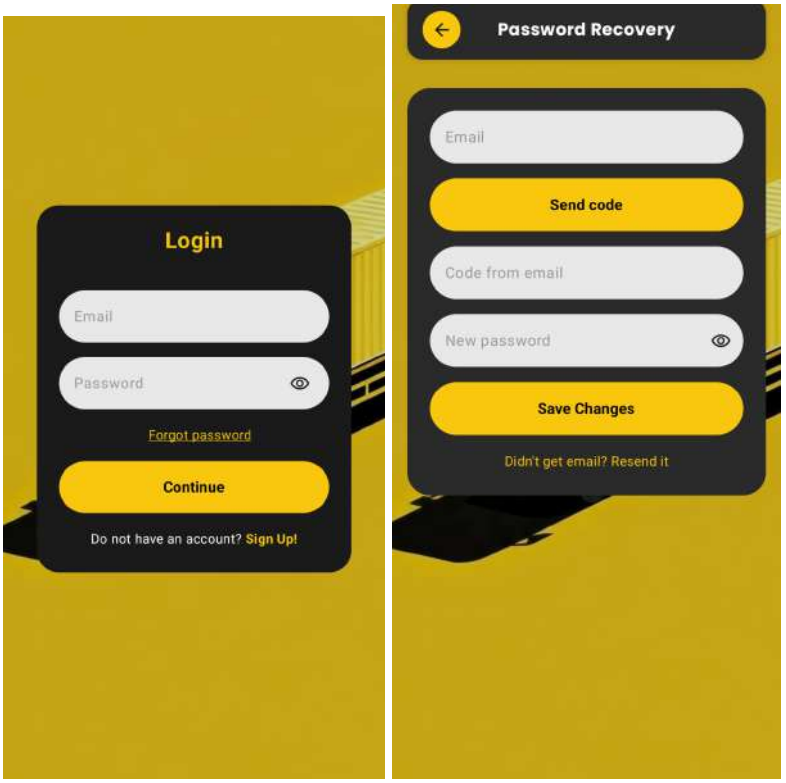


Figure 5.20 Login page of the mobile application (source: *Developed by the authors*)

Creating order is the most detailed part of the app. The order creation form provides the same utilities to add an order accordingly to the web application. In the mobile application we kept graphic elements the same for higher similarity with the web application.

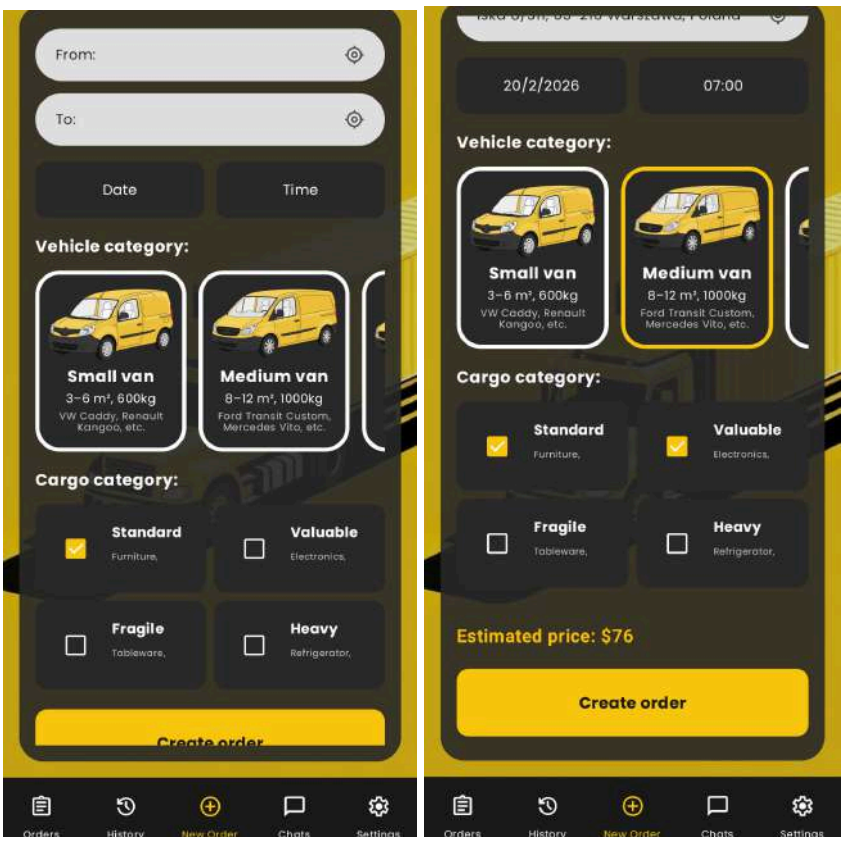


Figure 5.20 Order creation in the mobile application (source: *Developed by the authors*)

The dashboard is the home base for everything a customer needs to do in the app. At the bottom of the screen (Figure 5.21), there is a permanent menu bar. When a user places the first order, an interactive card appears on the Orders page. The customer then has the ability to tap on it in order to access order details.

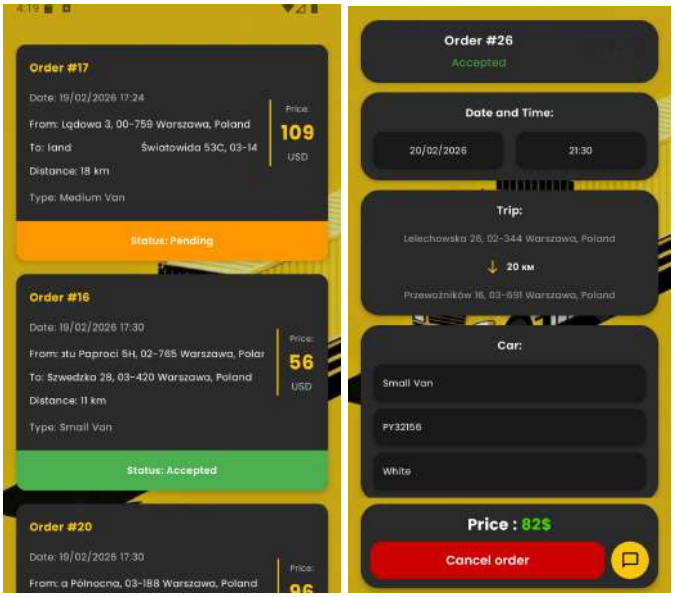


Figure 5.21 Order placement in the mobile application (source: *Developed by the authors*)

Drivers have their own specialized workspace called the Orders feed, which acts like a live marketplace for earnings. They look for jobs marked with an orange Pending status (Figure 5.22). Each job is shown on a simple card with order details, helping the driver quickly decide if the job is right for them. Once a driver clicks an order, they can see the full route amd specific cargo needs before hitting the *'Accept order'* button.

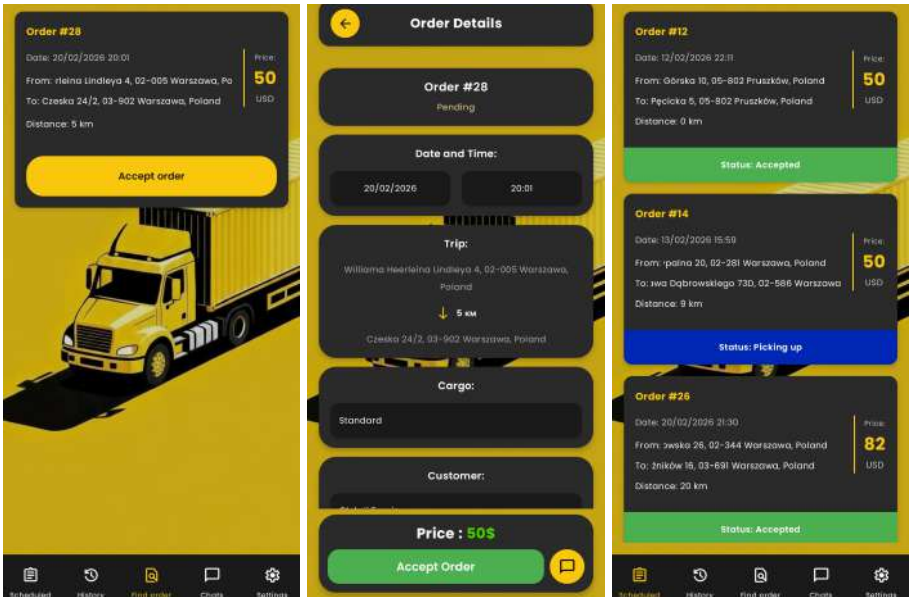


Figure 5.22 Driver's order flow of the mobile application (source: *Developed by the authors*)

The Chat module allows drivers and customers to talk to each other.

Figure 5.23 Chat order flow of the mobile application (source: *Developed by the authors*)

When the driver arrives at drop-off locations previously marked by the customer, it means the end of the transportation process. The driver then has to confirm delivery and this is the process that changes the order status on Completed from both sides.

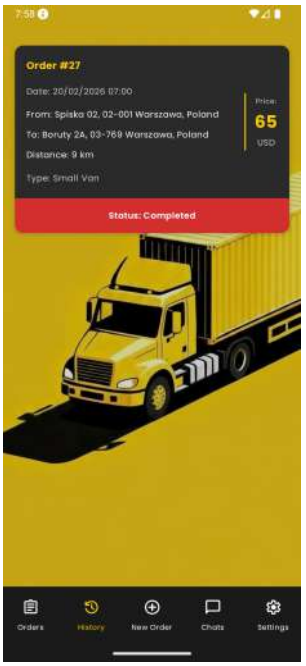


Figure 5.24 Completed order in the mobile application (source: *Developed by the authors*)

Both drivers and users can use the Edit mode in their settings. This module allows them to update their personal information that is extremely important from the perspective of keeping ways of communication like email and phone number up to date.

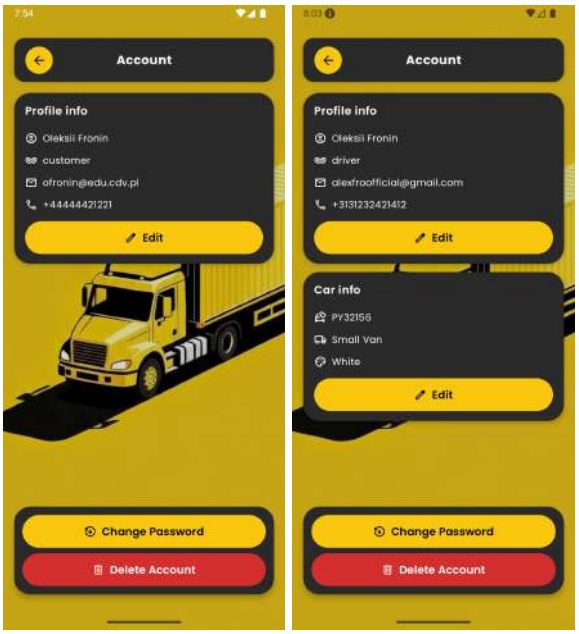


Figure 5.25 Edit profile flow for customer (left) and driver (right) of the mobile application (source: *Developed by the authors*)

Changes in personal information are made through the inputs placed inside the personal information form. Additionally, the driver is able to actualize their vehicle details.

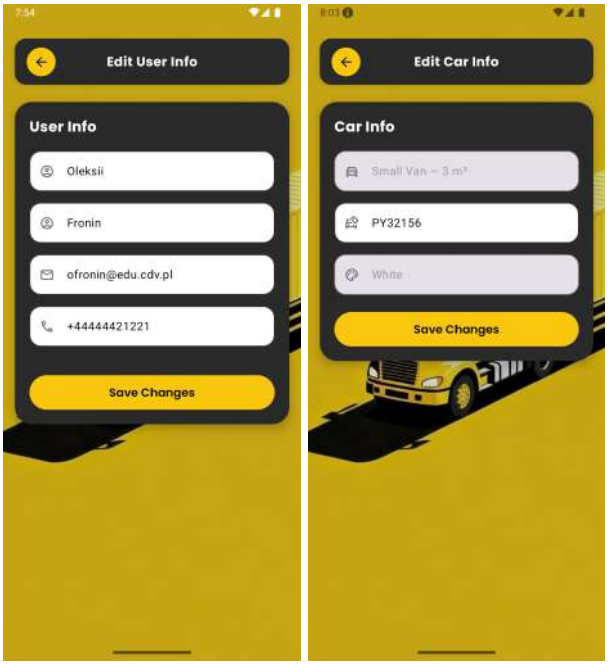


Figure 5.26 Edit profile form in the mobile application (source: *Developed by the authors*)

Apart from deletion of the account, each user is able to change their password by simply hitting the corresponding button. Setting up a new password basically requires a verification code sent via email.

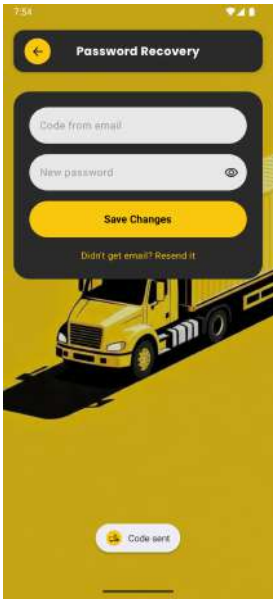


Figure 5.27 Change password flow of the mobile application (source: *Developed by the authors*)

On the settings page both of the users can also play with notifications settings and eventually turn the toggle for certain options they would like to be notified about. Moreover, notifications regarding upcoming orders are set by choosing a time interval from the list.

Figure 5.28 Notifications page in the mobile application (source: *Developed by the authors*)

The last interesting element is the About us page that consists of a brief description of 'Gazelka' purpose. It also lists the official email and phone number that a user can click. As a result of this function, they are redirected to the default email app or phonebook.

Figure 5.29 About us page in the mobile application (source: *Developed by the authors*)

6. Summary and conclusions

6.1 Achieved goals of the work

The project achieved several key goals, as outlined in the initial planning phase.

- A two-sided system was integrated by evolving a unified environment that works for both customers and drivers through shared back-end logic while sustaining distinct front-end roles. This was accomplished using Swagger for the server, ensuring secure data management along with role-based access via JWT tokens. Customers are now able to create orders with detailed specifications, while drivers view and accept them depending on their availability and vehicle affinity;
- A secure authentication mechanism was also implemented. This includes login, registration, password hash and multistage recovery. Email-based approval of new users was used in order to confirm accounts and reset password, protecting user data and order history;
- UX/UI designs were adapted. The application has a feature of polished transitions and animations. Both web and mobile applications emphasize map-based interactions with Google Maps [biblography] for location picking. This guarantess a steady experience across different devices with localStorage and sessionStorage catching tokens and draft orders to minimize API calls and enhance responsiveness.
- Real-time order statuses were also implemented. Orders progress through semantic color statuses and are visible for both roles. SignalR [biblography] allows live updates for order status changes and Firebase Cloud Messaging [biblography] delivers push notifications for events like new orders or reminders.
- Successful integration of a real-time messaging system. Each order has a dedicated chat for individual order coordination. Messages are sent via SignalR for immediate delivery when users are online with FCM pushes notifying offline participants.

6.2 Conclusions

This project has demonstrated that a hybrid web-mobile platform can productively uphold urban moving services. The application carries out well in simulated networks with real-time features like chat and notifications.

Integration of third-party services proved its importance for features like location-based order creation. The project also emphasised the significance of equal complexity, because real-time systems, in fact, require careful handling.

Problems encountered during writing

- Integrating real-time features with SignalR and FCm required debugging connection stability, specifically in low-network environments that lead to delays in testing chat scenarios. This was determined by adding retry logic and offline fallbacks that extend the timeline a little bit;

- Handling Google Maps API for distance calucation and location picking by coordinates encountered issues with API quotas testing that created occasional failures. The problem was moderated by caching results in sessionStorage;
- Cross-platform consistency between web and mobile applications was quite problematic for UI elements like carruosels and forms, requiring multiple repetitions for adaptive design.

Future plans

Short-term intentions contain an integrating payment system for smooth transactions, it also intends to implement professional matching algorithms. Mid-term goals include implementation of live driver tracking via background location services. Improving the chat with pictures and voice messages support will definitely enhance UI. The longest goals according to their terms are administration dashboard and partnership with e-commerce platforms that will turn ‘Gazelka’ into a bigger moving ecosystem.

List of bibliography

1. AnyVan, https://www.anyvan.com/ (access: 07.01.2026);
2. Anytime Estimate (2025) survey, https://journalrecord.com/2025/02/20/is-moving-more-stressful-than-divorce-study-says-yes/ (access: 09.01.2026);
3. BCrypt, https://www.npmjs.com/package/bcrypt (access 22.01.2026)
4. Bearer, https://www.bearer.com/ (access 22.01.2026)
5. CSS, https://www.w3schools.com/css/ (access 14.01.2026)
6. C#, https://www.w3schools.com/CS/index.php (access 29.01.2026)
7. Dispatchers IO, https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-dispatchers/ (access 10.02.2026)
8. Docker, https://www.docker.com/ (access 24.02.2026)
9. DTO, https://www.baeldung.com/java-dto-pattern (access 27.01.2026)
10. Figma, https://www.figma.com (access: 10.01.2026);
11. Firebase, https://firebase.google.com/docs/cloud-messaging?hl=pl (access 24.01.2026)
12. Geocoder, https://developers.google.com/maps/documentation/javascript/reference/geocoder?hl=pl (access 18.01.2026)
13. Gmail SMTP, https://kb.synology.com/pl-pl/SRM/tutorial/How_to_use_Gmail_SMTP_server_to_send_emails_for_SRM (access 25.01.2026)
14. Google Maps API, https://mapsplatform.google.com/lp/maps-apis/ (access 17.01.2026)
15. Google PLaces API, https://developers.google.com/maps/documentation/places/web-service/overview?hl=pl (access 23.01.2026)
16. HTML, https://www.w3schools.com/html/ (access 13.01.2026)
17. HTTP, https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages (access 13.02.2026)
18. Human factors of visual and cognitive performance during driving, https://www.inviewmedical.pl/wp-content/uploads/2015/08/Human-factors-of-visual-and-cognitive-performance-in-driving-2.pdf (access 17.01.2026)
19. Ikea, https://www.ikea.com/pl (access: 12.01.2026);
20. JavaScript, https://www.w3schools.com/js/ (access 15.01.2026)
21. JSON, https://www.json.org/json-en.html (access 20.01.2026)
22. JWT authentication, https://www.jwt.io/introduction#what-is-json-web-token (access 15.01.2026)
23. Kotlin, https://kotlinlang.org/ (access 23.01.2026)
24. Lalamove, https://web.lalamove.com (access: 06.01.2026);
25. LocalStorage, https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage (access 16.01.2026)
26. Research and Markets 2026, https://www.researchandmarkets.com/reports/6082460/ride-hailing-market-global-forecast?srsltid=AfmBOopAoTa2Dc7TsKYW_EQUDMiVsz5mf4IpVZnJDXTz0klKiu0YmYFY, *Ride Hailing Market Report: Trends and Forecast.*
27. Restful API, https://boringowl.io/tag/restful (access 20.01.2026)

28. SharedPreferences, https://developer.android.com/training/data-storage/shared-preferences?hl=pl (access 26.01.2026)
29. SignalR, https://dotnet.microsoft.com/en-us/apps/aspnet/signalr (access 19.01.2026)
30. Single-page application, https://boringowl.io/blog/spa (access 15.01.2026)
31. Swagger UI, https://swagger.io/why-swagger/?utm_source=google-paid&utm_medium=ppcg&utm_campaign=G+-+API+Hub+-+EMEA+-+BR&utm_term=swagger%20ui%20online&utm_content=p&gclsrc=aw.ds&gad_source=1&gad_campaignid=22683279341&gbraid=0AAAAAD_lD12M_KLl56lPzjan3Mx5f40hg&gclid=CjwKCAiAnoXNBhAZEiwAnItcG8w75oL3WZ29OCw-D11Au98BPpCR3YyGYPZM7_U43Rdb_2T6egnr0hoCAWEQAvD_BwE (access 20.02.2026)
32. Taniguchi E., Thompson R. G. & Yamada T., *City Logistics: Network Modelling and Intelligent Transport Systems.* CRC Press. (access: 05.01.2026)
33. Uber Freight, https://www.uberfreight.com (access: 05.01.2026)
34. ViewModel, https://developer.android.com/topic/libraries/architecture/viewmodel?hl=pl (access 16.01.2026)
35. WebSocket, https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (access 20.01.2026)

List of figures

List of tables

Abstract/Streszczenie

The thesis presented the development of a web and mobile applications by ‘Gazelka’, designed to simplify moving and transportation services for customers by connecting them with drivers in real time. The system includes user login, registration and email authentication along with password hash and recovery and role-based profiles for both of the sides. Customers are able to create and cancel orders, specify vehicle and cargo types as well as to track status order during its progress. Drivers can receive real-time notifications of available orders, accept or decline them, update their statuses and communicate with their customers through chat.

Praca dyplomowa przedstawia opracowanie aplikacji webowej i mobilnej o nazwie ‘Gazelka’, której celem jest ułatwienie świadczenia usług przeprowadzkowych i transportowych poprzez połączenie klientów z kierowcami w czasie rzeczywistym. System umożliwia rejestrację i uwierzytelnianie użytkowników z weryfikacją email, bezpieczne odzyskiwanie hasła oraz profile zróżnicowane dla klientów i kierowców. Klienci mogą tworzyć zlecenia, wybierając lokalizację odbioru i dostawy na interaktywnej mapie Google, a następnie śledzić status zlecenia. Kierowcy otrzymują powiadomienia o dostępnych zleceniach w czasie rzeczywistym, akceptują je, aktualizują statusy i komunikują się z klientami za pomocą wbudowanego czatu.

Keywords: moving service, real-time chat, push notifications, order management, map integration

COLLEGIUM DA VINCI

UCZELNIA LUDZI CIEKAWYCH

BREAKDOWN SHEET FOR WORK CARRIED OUT BY STUDENTS WITH REGARDS TO MULTI-AUTHOR (TEAM) ENGINEERING WORKS AT THE FACULTY OF APPLIED SCIENCES AT COLLEGIUM DA VINCI IN POZNAN.

Topic of the thesis: Web and mobile application for providing moving services to the customers.

	name	surname
CO-AUTHOR 1:	Vitalii	Sakhno
CO-AUTHOR 2:	Artem	Dzhula
CO-AUTHOR 3:	Oleksii	Fronin
CO-AUTHOR 4:		
CO-AUTHOR 5:		

	title/degree	name	surname
SUPERVISOR:	DR INŻ	ADAM	CZAJKA

Collegium Da Vinci
ul. Gen. Tadeusza Kutrzeby 10
61-719 Poznań
61 271 10 10 / info@cdv.pl

1. Overall contribution to the thesis determined on a percentage basis by the student as verified by the supervisor during the diploma awarding process:

	PERCENTAGE CONTRIBUTION OF THE CO-AUTHOR TO THE THESIS	TOTAL IN %
CO-AUTHOR 1	30%	100%
CO-AUTHOR 2	35%	
CO-AUTHOR 3	35%	
CO-AUTHOR 4		
CO-AUTHOR 5		

2. Percentage contribution of the co-authors of the thesis to the tasks arising from the requirements set for the engineering thesis*:

NO.	DESCRIPTION OF THE TASK	PERCENTAGE SHARE OF THE CO-AUTHOR IN THE PERFORMANCE OF THE TASK					TOTAL IN %
		CO-AUTHOR 1	CO-AUTHOR 2	CO-AUTHOR 3	CO-AUTHOR 4	CO-AUTHOR 5	
1	Drafting an introduction to the thesis	100%	0%	0%			100%
2	Determination of the state of the art	100%	0%	0%			100%
3	Definition of purpose, scope of the project and distribution of tasks in the project	80%	0%	20%			100%
4	Development of methodology of the thesis	70%	0%	30%			100%
5	Front-end development (coding)	25%	40%	35%			100%
6	Back-end development (coding)	5%	60%	35%			100%
7	Preparation of the documentation compliant with the software engineering requirements	100%	0%	0%			100%
8	Preparation of the project description	70%	0%	30%			100%
9	Software testing procedure	10%	50%	40%			100%
10	Preparation of the description of the method of implementation/installation/operation.	50%	50%	0%			100%
11	Preparation of the summary, conclusions	90%	0%	10%			100%
12	Editing and proofreading of the paper (in accordance with the applicable document - Formal Requirements - Computer Science	90%	0	10%			100%

13	Other**						100%

* Please fill in only the tasks carried out as part of the thesis.

** In the line "other", please enter the task that has not been considered above.

The appendix forms the basis for the assessment of the individual contribution of each student to the thesis and is an integral part of the thesis as the last unnumbered page of the thesis.

The appendix is signed by the authors of the thesis and is approved by the thesis supervisor.

Performance of individual tasks by the authors depends on the nature of the thesis.

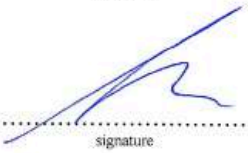
CO-AUTHOR 1: 
..... signature

CO-AUTHOR 2: 
..... signature

CO-AUTHOR 3: 
..... signature

CO-AUTHOR 4:
..... signature

CO-AUTHOR 5:
..... Signature

SUPERVISOR: 
..... signature

Poznan ....., 24.02.2026 .....
town date